



Diploma Thesis 2025/2026

School Association Management System

conducted at the
Higher Technical and Vocational College Neufelden
for Industrial Engineering,
Specialization in Business Informatics

by

David Niederleithner (Matr.Nr.: 20212012)

Fabian Schopf (Matr.Nr.: 20212019)

Paul Thumfart (Matr.Nr.: 20212024)

Supervisors

Dipl.-Ing. Werner Furtmüller

Dipl.-Ing. Gernot Stimpfl

Neufelden, April 9, 2026

Statutory Declaration

I hereby declare under oath that I have written this diploma thesis independently and without unauthorized outside assistance, and that no sources or aids other than those indicated have been used.

All passages adopted verbatim or in substance from other works, including sources from electronic databases or the internet, have been identified as such, with proper indication of the source and in compliance with the applicable rules of academic citation. This declaration also extends to all visual representations, tables, sketches, and drawings contained in the thesis.

In the preparation of this thesis, the generative AI tools ChatGPT, Gemini, Claude, and Perplexity were used as auxiliary tools for checking grammar and spelling, improving academic style and phrasing, structuring ideas, retrieving background information, generating images, and supporting iterative refinements of text and code. Furthermore, parts of the development process followed a pair-programming-oriented approach, with AI being used to enhance code quality, support validation, and improve technical implementation.

The use of these tools has been declared fully and truthfully, including the respective product versions and the prompts used.

Neufelden on Thursday, April 9, 2026

David Niederleithner

Niederbumberg 3
4133 Niederkappel
Born on October 6, 2006

0677 / 628 651 73
niederleithnerdavid@gmail.com



.....
David Niederleithner

Fabian Schopf

Schlägerbergweg 14
4162 Julbach
Born on August 9, 2007

0677 / 612 252 43
rootgives@gmail.com



.....
Fabian Schopf

Paul Thumfart

Eberhardschlag 24
4191 Vorderweißenbach
Born on August 5, 2007

+43 670 4092121
paul.thumfart@gmail.com



.....
Paul Thumfart

Acknowledgements

At this point, we would like to thank all those who contributed to the creation of this diploma thesis through their professional and personal support.

Special thanks go to our supervisors, Dipl.-Ing. Werner Furtmüller and Dipl.-Ing. Gernot Stimpfl, for their continuous support, valuable advice, and constructive feedback throughout the entire project phase. Their technical expertise and willingness to help have significantly contributed to the successful implementation of the Neufelden Organization and Administration Application (NOVA).

We also thank the HTL Neufelden and the entire teaching staff of the Business Informatics department. Profound education over the past years has provided us with the necessary tools and expertise to realize this comprehensive software project.

Gender Disclaimer

For the sake of readability, the generic masculine form is used in this diploma thesis. Unless otherwise indicated, all personal terms apply equally to all genders. This simplification serves exclusively for better comprehensibility and implies no judgement.

Client

Association:	School Association of HTL Neufelden
Address:	Höferweg 47, 4120 Neufelden 4120 Neufelden
Website:	www.htl-neufelden.at
Contact Person:	Dipl.-Ing. Rainhard Prieschl
E-Mail Address:	schulverein@atn.ac.at
Phone:	+43 (0) 678 7829316

Zusammenfassung

Ausgangslage: Die Administration vom Schulverein der HTL Neufelden ist durch einen hohen organisatorischen Aufwand und manuelle Datenpflege geprägt. Die bisherigen Prozesse zur Mitgliederverwaltung, Beitragsverrechnung und Veranstaltungsorganisation benötigen viel Zeit und bargen ein erhöhtes Risiko für fehlerhafte oder redundante Daten.

Zielsetzung: Ziel dieser Diplomarbeit ist die Konzeption und Entwicklung der *Neufeldner Organisations- und Verwaltungs-Anwendung (NOVA)*. Dabei handelt es sich um ein zentralisiertes, maßgeschneidertes Web-System für den Schulverein der HTL Neufelden, welches die administrativen Kernprozesse umfasst, digitalisiert und weitestgehend und spezifischen Anforderungen des Auftraggebers nachkommt.

Umsetzung: Die Realisierung des Projekts erfolgte in Form einer modernen Single-Page-Application (SPA). Als Client-Technologie kam das Frontend-Framework Angular zum Einsatz, um eine intuitive und reaktionsschnelle Benutzeroberfläche zu schaffen. Das serverseitige Backend wurde mit C# und dem .NET-Framework implementiert, wobei Entity Framework Core für eine sichere und strukturierte Datenbankbindung sorgt. Die Systemarchitektur ist modular aufgebaut und ermöglicht die nahtlose Interaktion verschiedener Service-Komponenten.

Funktionsumfang und Ergebnisse: Das fertiggestellte System umfasst mehrere voll integrierte Kernmodule: Eine dynamische Mitglieder- und Kontaktverwaltung bildet die Basis. Ein automatisiertes Rechnungswesen ermöglicht die effiziente Erstellung von Einzel- und Sammelrechnungen (Bulk-Invoices) inklusive automatischer PDF-Generierung. Darüber hinaus verfügt das System über einen flexiblen Formular-Builder zur Datenerfassung, ein Event- und Kalendermodul sowie ein umfassendes E-Mail-Service-System für gezielte Kommunikationskampagnen. Das Ergebnis ist eine maßgeschneiderte, benutzerfreundliche Verwaltungsplattform, die den administrativen Aufwand für den Schulverein der HTL Neufelden reduziert.

Abstract

Initial Situation: The administration of school associations is faced with a high organizational workload, manual data maintenance, and decentralized software solutions. At the HTL Neufelden, the existing processes for member management, fee invoicing, and event organization required a considerable amount of time and carried an increased risk of incorrect or redundant data.

Objectives: The objective of this diploma thesis is the design and development of the *Neufeldner Organisations- und Verwaltungs-Anwendung (NOVA)*. This involves a centralized, tailor-made web-based system for the school association of the HTL Neufelden, which bundles, digitizes, and largely automates the core administrative processes.

Implementation: The project was realized in the form of a modern Single-Page Application (SPA). The frontend framework Angular was utilized as the client technology to create an intuitive and responsive user interface. The server-side backend was implemented using C# and the .NET framework, with Entity Framework Core ensuring a robust and structured database connection. The system architecture is designed to be modular, enabling seamless interaction between various service components.

Features and Results: The completed system encompasses several fully integrated core modules. A dynamic member and contact management module form the foundation. An automated accounting system facilitates the efficient creation of individual and bulk invoices, including automated PDF generation. Furthermore, the system features a flexible form builder for data collection, an event and calendar module, and a comprehensive email service system for targeted communication campaigns. The result is a user-friendly management platform that significantly reduces the administrative burden for the HTL Neufelden school association, ensures data consistency, and sustainably increases the efficiency of the entire association's operations.

Table of contents

Introduction	10
1.1 Initial Situation	11
1.2 Goal	12
2. Project Concept	14
2.1 Functional Requirements	14
2.2 Non-Functional Requirements	15
2.3 System Architecture Overview.....	15
2.4 Database Concept	16
2.5 User Interface Concept	18
2.6 Wireframes.....	18
2.7 Prototypes	21
3. Project Management	23
3.1 Team Coordination.....	23
3.2 Milestones	23
3.3 Tech Stack.....	25
3.4 Project Structure & Responsibilities.....	26
3.5 Version Management.....	27
3.5.1 Azure DevOps.....	27
3.5.2 Feature Branching.....	27
3.5.3 Work Item Management	27
4. Technologies and Architectural Foundations	28
4.1 Libraries & NuGet Packages	28
4.2 Design Patterns	29
4.2.1 Repository Pattern.....	29
4.2.2 Generic Repository Pattern	30
4.2.3 Service Layer Pattern	30
4.2.4 Facade Pattern	31
4.2.5 DTO Pattern (Data Transfer Object)	32
4.2.6 Middleware Chain Pattern.....	33
4.2.7 Soft Delete Pattern	34
4.2.8 Template Method Pattern	34

4.2.9	Strategy Pattern.....	34
4.2.10	Dependency injection.....	35
4.3	Design Principles	37
4.3.1	Development & Deployment Principles	37
4.3.2	Architectural Principles.....	38
4.3.3	Frontend Principles.....	38
4.3.4	Other Principles	39
5.	Database	40
5.1	Concept	40
5.2	Entities.....	42
5.3	Access Management	57
5.4	Development Database.....	57
5.4.1	Docker	57
5.4.2	Starting Database.....	57
5.5	Produktion Database.....	58
5.5.1	School Server	58
5.5.2	Back Ups.....	58
5.6	Database Connection	59
5.6.1	Environmental Variables.....	62
6.	Backend	64
6.1	Web API with ASP.NET Core.....	64
6.2	Backend Structure	65
6.2.1	Controllers	66
6.2.2	Services	68
6.2.3	Core.....	70
6.2.4	Infrastructure.....	71
6.2.5	Tests.....	72
6.2.6	Logging and Monitoring.....	75
6.3	Backend Workflow	79
6.4	Main Features	82
6.4.1	Authentication and Authorization	82
6.4.2	Member Management.....	87
6.4.3	Product Management.....	93

6.4.4	Formular System	97
6.4.5	Import System.....	106
6.4.6	Invoice Builder	109
6.4.7	Email System with Templates	112
6.4.8	Invoice system with Templates.....	120
6.4.9	Kalender System	123
6.4.10	Settings System.....	126
6.5	Secondary Features.....	136
6.5.1	Dashboard Service	136
6.5.2	Export System	140
6.5.3	PDF Generation.....	144
6.5.4	QR-Code Generation.....	148
7.	Frontend	151
7.1	Introduction	151
7.2	Technology	151
7.2.1	Angular and Typescript	151
7.2.2	Reactive State with RxJS	151
7.2.3	Angular Material and Styling	151
7.3	Design Patterns and Architecture	152
7.3.1	The Facade Design Pattern (State Management).....	152
7.3.2	Modularization and Lazy Loading	152
7.4	Data Handling and Security	153
7.4.1	Route Guards	154
7.5	Routing and Navigation.....	154
7.5.1	Global Routing Configuration	154
7.5.2	Public vs. Protected Routes	155
7.6	Authentication and Authorization	155
7.6.1	NOVA Identity Integration	155
7.6.2	Route Guards and Interceptors	156
7.7	Core Feature Implementation.....	157
7.7.1	Unified Entity Management.....	157
7.7.2	Table Personalization and Persistent Tables	158
7.7.3	Dynamic Form Builder UI	158

7.7.4	Invoice Management and PDF Generation.....	159
7.7.1	Dynamic Email System and Templates	159
7.7.1	Product and Fee Management	160
7.8	General UI Layout and Interface Patterns.....	160
7.8.1	Three Viewing Zone Skeleton Layout	160
8.	Deployment	162
8.1	Evolution of the Deployment Strategy.....	162
8.1.1	Legacy Deployment: Manual IIS Provisioning.....	162
8.1.2	8.1.2 Transition to Containerization (Portainer).....	163
8.2	CI/CD Pipeline Implementation.....	163
8.2.1	Pipeline Workflow.....	163
8.3	Conclusion on Infrastructure.....	164
9.	Results and Outlook.....	165
9.1	Benchmarks.....	165
9.2	Feedback from Client	166
9.3	Unit testing.....	167
9.4	Points for Improvement.....	170
10.	Table Collection	177
11.	List of Figures	182
12.	List of Tables	184
13.	Bibliography.....	191

Glossary

Due to the advanced technical terminology used throughout this thesis, a comprehensive glossary is provided at the end of the document for reference.

Introduction

The administration of a school association involves numerous organizational tasks, ranging from the registration of new members and the collection of annual fees to the coordination of events. Furthermore, continuous communication with members requires the regular dispatch of informational emails and newsletters. On the financial side, administrators must also handle the accurate generation of individual and bulk invoices, track incoming payments, and manage payment reminders.

1.1 Initial Situation

At the Higher Technical and Vocational College (HTL) Neufelden, the school association previously used the third-party administration software campai (campai, 2026) to manage the school association. However, this system proved to be overly complex and not tailored to the specific needs of the association. Managing member data and organizing school-related events required significant manual effort, this made many processes highly inefficient and demanded substantial time commitments. Furthermore, the software enforced rigid accounting rules that severely restricted flexibility, rendering the editing or deletion of incorrect invoices entirely impossible. Additionally, the short session times were frustrating to deal with for users as they had to keep re-entering the login credentials within very short time windows.



Figure 1: Campai Logo



Figure 2: NOVA-Logo

1.2 Goal

The primary goal is to create a centralized web application, called NOVA, that is fully tailored to the specific operational requirements of the school association. To eliminate the inefficiencies of the previous software, this project aims to achieve the following core objectives:

- **Reduce Administrative Effort:** Automate and simplify the management of member data and the coordination of school-related events.
- **Implement Flexible Accounting:** Develop a fully functional, flexible invoicing module that allows administrators to easily generate, edit, and delete individual or bulk invoices.
- **Improve Communication:** Integrate a reliable email service system to handle mass emails and news letters
- **Enhance User Experience:** Provide a frustration-free workflow by implementing improved session management with extended login times, preventing constant interruptions.

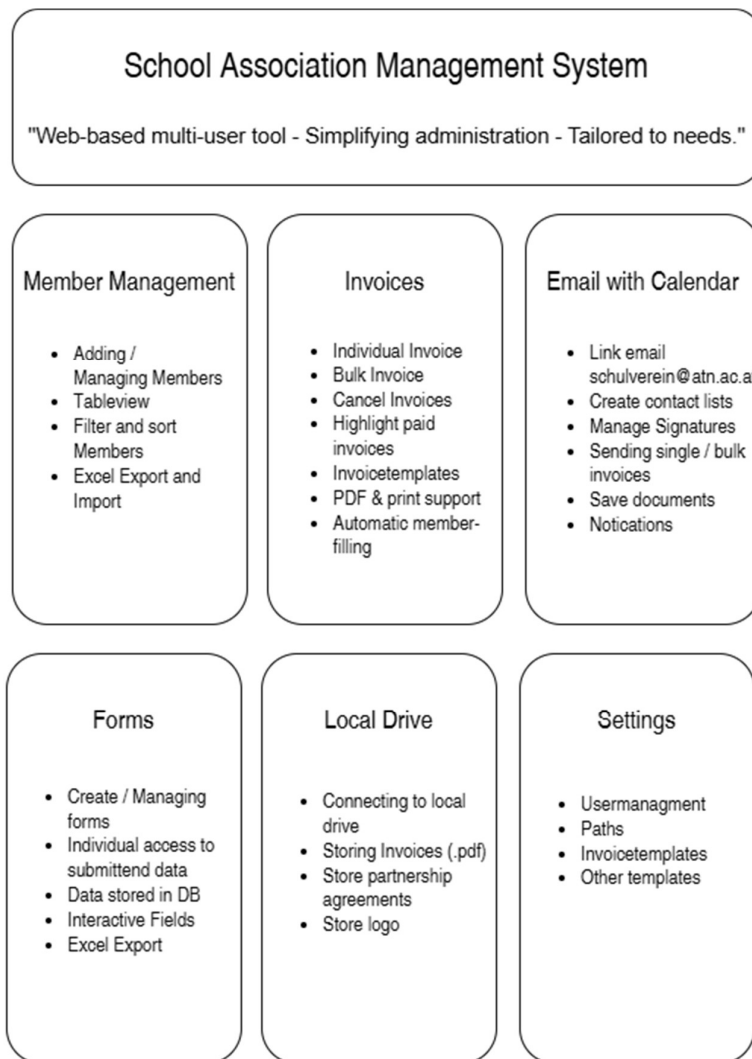


Figure 3: Task Overview

2. Project Concept

The project concept defines the structural and functional foundation of the Neufeldner Organisations- und Verwaltungs-Anwendung (NOVA). It serves as a blueprint for the development phase by detailing what the system must accomplish and how it is technically conceptualized to solve the inefficiencies of the previously utilized software

2.1 Functional Requirements

Functional requirements define the specific behaviors and features the system must support to fulfill the administrative needs of the school association. The system is divided into several core modules:

- **Member and Contact Management:** The system must provide comprehensive features to create, read, update, and delete (CRUD) member and contact records. It must support the categorization of individuals into dynamic groups to facilitate targeted administration. Furthermore, it requires functionalities to import existing data and handle soft-deletions to maintain historical integrity.
- **Financial Management and Invoicing:** A core requirement is the automated handling of association fees. The system must allow administrators to define products and generate both individual and bulk invoices. It requires an integrated PDF generation engine based on customizable invoice templates. Additionally, a debtor management feature is necessary to track payment statuses and outstanding balances.
- **Dynamic Data Collection (Form Builder):** To digitize paper-based forms and surveys, the system must include a dynamic form builder. Administrators must be able to design custom forms with various input fields, define submission limits, and manage the visibility of these forms. The system must collect, store, and display form submissions comprehensively.
- **Communication and Event Scheduling:** The application must integrate a fully functional email service to send notifications, invoices, and targeted email campaigns to specific member groups. Furthermore, an event scheduling module is required to organize association events, track invitees, and optionally synchronize with external calendar services (e.g., Google Calendar).

2.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance constraints, and security standards the system must adhere to:

- **Usability:** As the system replaces an overly complex third-party solution, a highly intuitive and user-friendly interface is mandatory. The application must be designed as a Single-Page Application (SPA) to ensure fast, seamless navigation without page reloads.
- **Security:** The system handles sensitive personal and financial data. Therefore, it must employ robust security measures, including stateless authentication via JSON Web Tokens (JWT), role-based access control (RBAC), and secure password hashing algorithms. Furthermore, password reset functionalities via secure email tokens are required.
- **Performance and Reliability:** The system must be capable of executing bulk operations—such as generating hundreds of invoices or sending mass email campaigns—efficiently without blocking the user interface or causing server timeouts.
- **Maintainability:** The software architecture must follow a strict separation of concerns, ensuring that the frontend and backend remain entirely decoupled. This modular approach guarantees that future extensions or updates can be implemented with minimal friction.

2.3 System Architecture Overview

To satisfy the aforementioned requirements, NOVA is conceptualized utilizing a modern, decoupled client-server architecture. This approach separates the presentation layer from the business logic and data access layers.

The frontend (Client) is built as a Single-Page Application using the Angular framework, which runs entirely in the user's web browser. It is responsible for rendering the user interface, handling user interactions, and performing client-side validation.

The backend (Server) is implemented as a RESTful Web API using the .NET Core framework (C#). It serves as the central hub for processing business rules, authenticating requests, and interacting with the database. The client and server communicate statelessly over HTTPS, exchanging data exclusively formatted as JavaScript Object Notation (JSON).

NOVA — System Architecture Overview

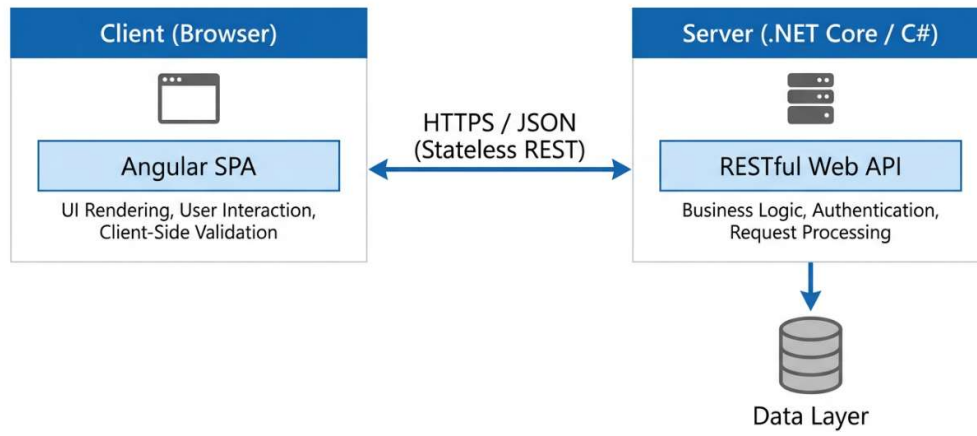


Figure 4: System Architecture Overview (Generated by Perplexity with NOVA's data)

2.4 Database Concept

The system utilizes a relational database structure to ensure data consistency and enforce referential integrity. The database schema is designed and managed using Entity Framework Core through a Code-First approach. This allows the database tables to be generated directly from the C# entity models defined in the backend infrastructure.

The core of the data model revolves around the `Member` and `Contact` entities, which hold relational ties to `Groups`, `Invoices`, and `Form Submissions`. To prevent accidental data loss of critical financial or personal records, a soft-delete mechanism is conceptually integrated across primary entities. Instead of permanently removing a record from the database, a deletion flag is set, hiding the record from standard application queries while preserving it for audit and historical purposes.

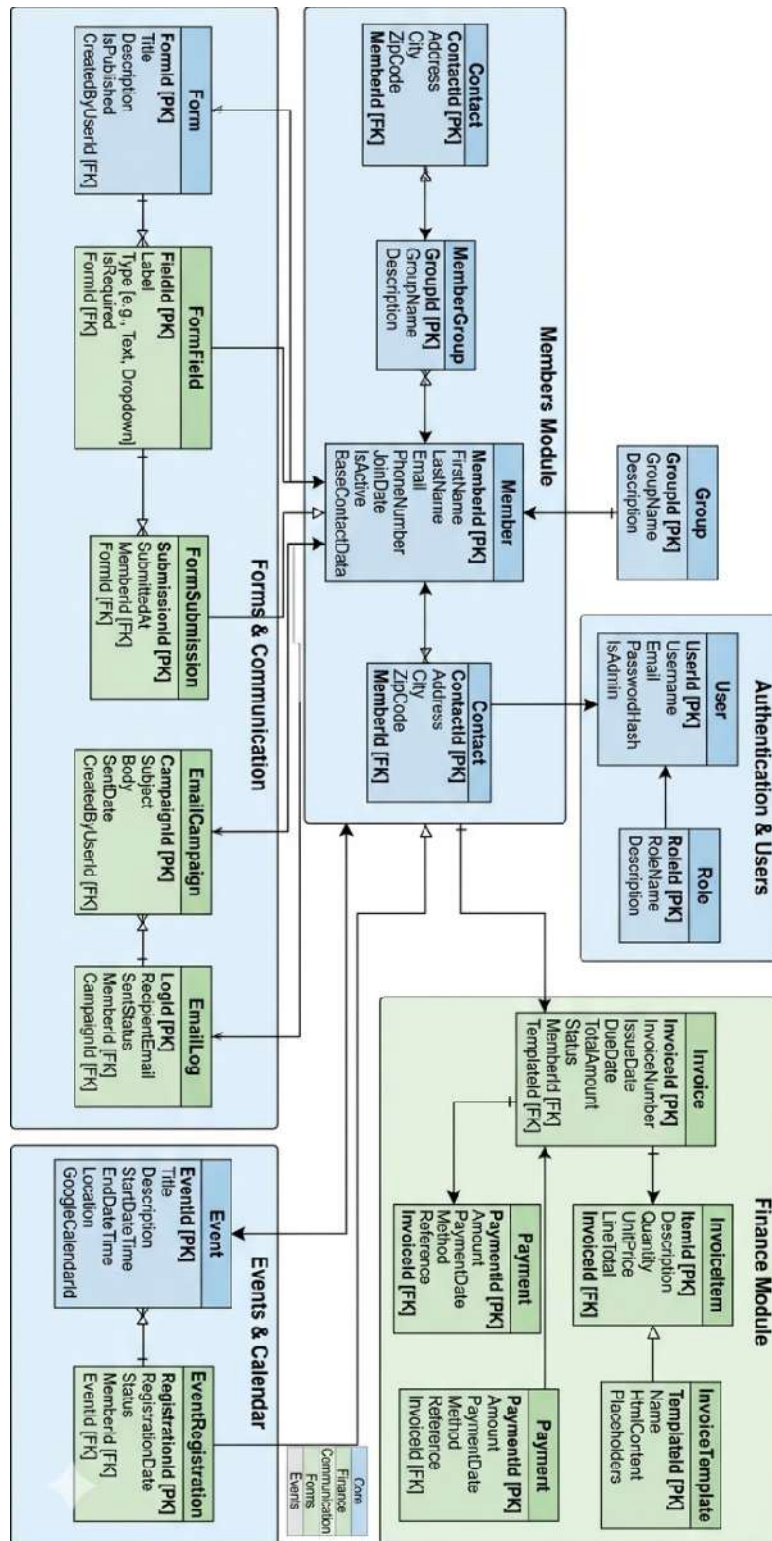


Figure 5: Database overview (Generated by Gemini's Nano Banana 2 using NOVA-Data)

2.5 User Interface Concept

The User Interface (UI) is designed with a strong emphasis on clarity, accessibility, and modern web design principles. To counteract the steep learning curve of the previous administration software, NOVA utilizes a clean, dashboard-centric layout.

Upon authenticating, users are greeted by a centralized dashboard providing an immediate overview of critical metrics, such as recent form submissions, upcoming calendar events, and outstanding debtor balances.

The application utilizes a persistent navigation bar to allow quick switching between the core modules (Members, Invoices, Forms, Emails, Settings). Data is consistently presented using standardized data tables featuring built-in pagination, sorting, and filtering capabilities. To streamline user workflows, entity creation and modification processes (e.g., adding a new member or creating a product) are predominantly handled via modal dialogs, ensuring the user does not lose their current context within the application.

2.6 Wireframes

Before the actual technical implementation of the user interface commenced, wireframes and UI prototypes were created to conceptualize the visual layout and user flow of the NOVA application. This step was crucial for aligning the design with the functional requirements outlined in section 2.1 and ensuring a highly intuitive User Experience (UX), thereby directly addressing the usability issues of the previously utilized software.

The prototyping phase focused on establishing a consistent design language across the entire Single-Page Application. This included defining the core structural elements, such as the persistent side navigation, the header bar, and the standardized layout for data tables and modal dialogs. By creating these visual representations early in the project lifecycle, the team was able to validate design decisions, refine the navigation paths, and ensure that complex administrative tasks—such as bulk invoicing or form creation—were presented in a clear, accessible manner.

Key screens that were conceptualized during this phase include:

- **The Dashboard:** Designed to act as the central hub upon login, providing administrators with an immediate overview of critical metrics, recent form submissions, and upcoming calendar events.
- **Data Management Views:** Standardized wireframes for the member, contact, and debtor lists, featuring integrated search, filtering, and pagination controls to handle large datasets efficiently.

- **Entity Creation Dialogs:** Mockups detailing the modal windows used for creating or editing entities (e.g., adding a new member or configuring an email campaign) without forcing the user to navigate away from their current context.

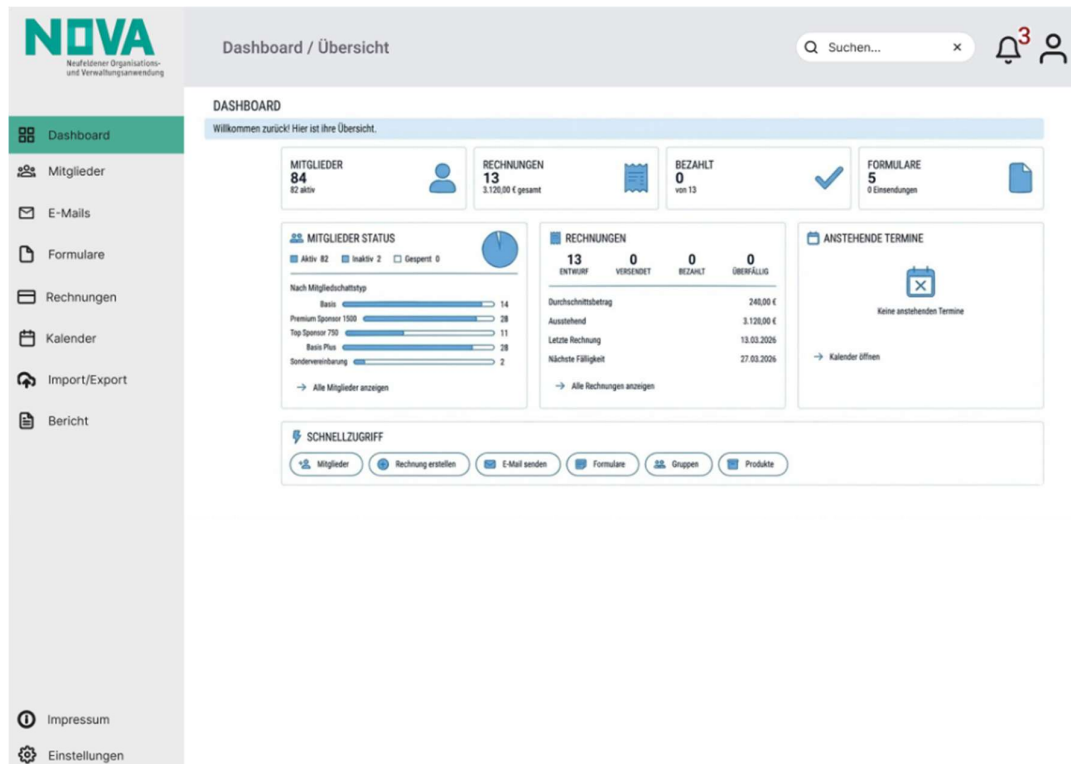


Figure 6: Wireframe of NOVA-Dashboard

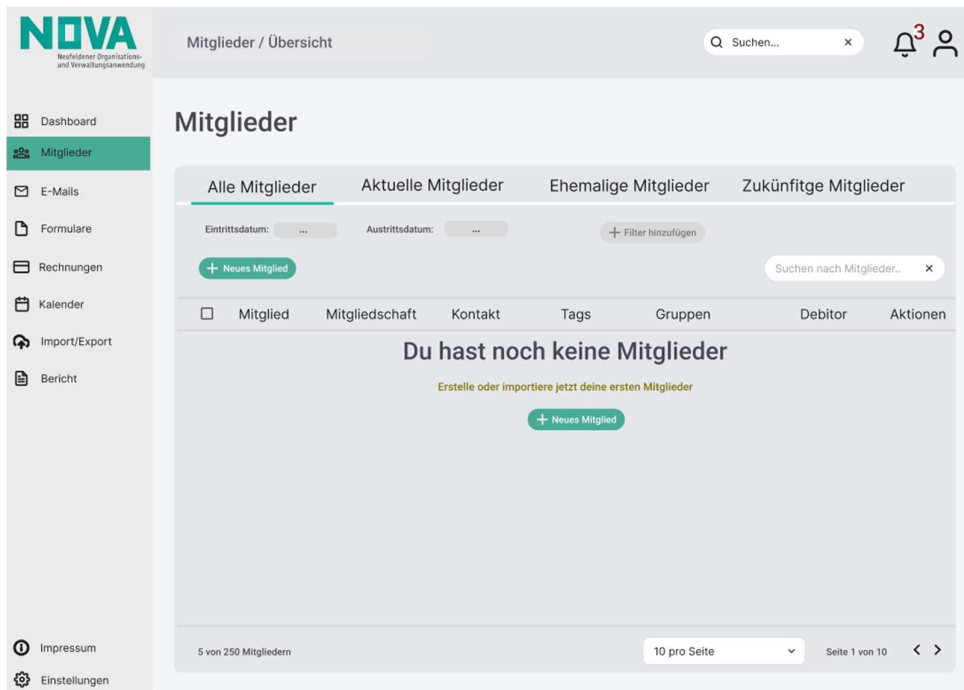


Figure 7: Wireframe of NOVA-Members-Page

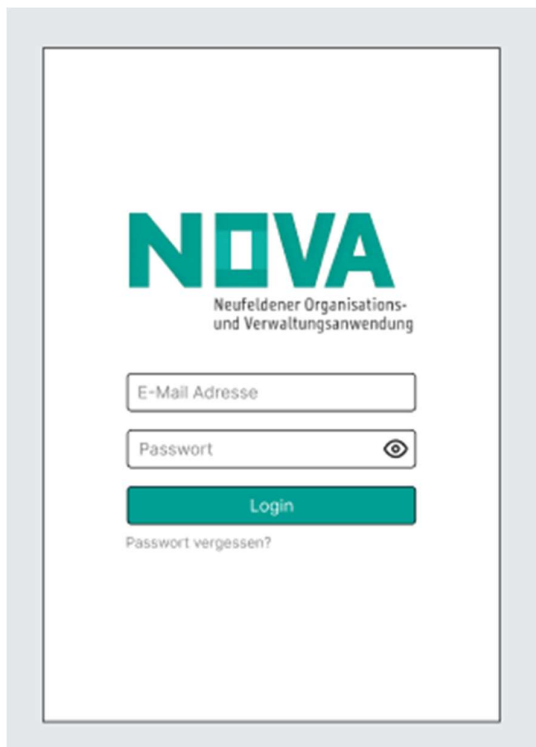


Figure 8: Wireframe of NOVA-Login-Page

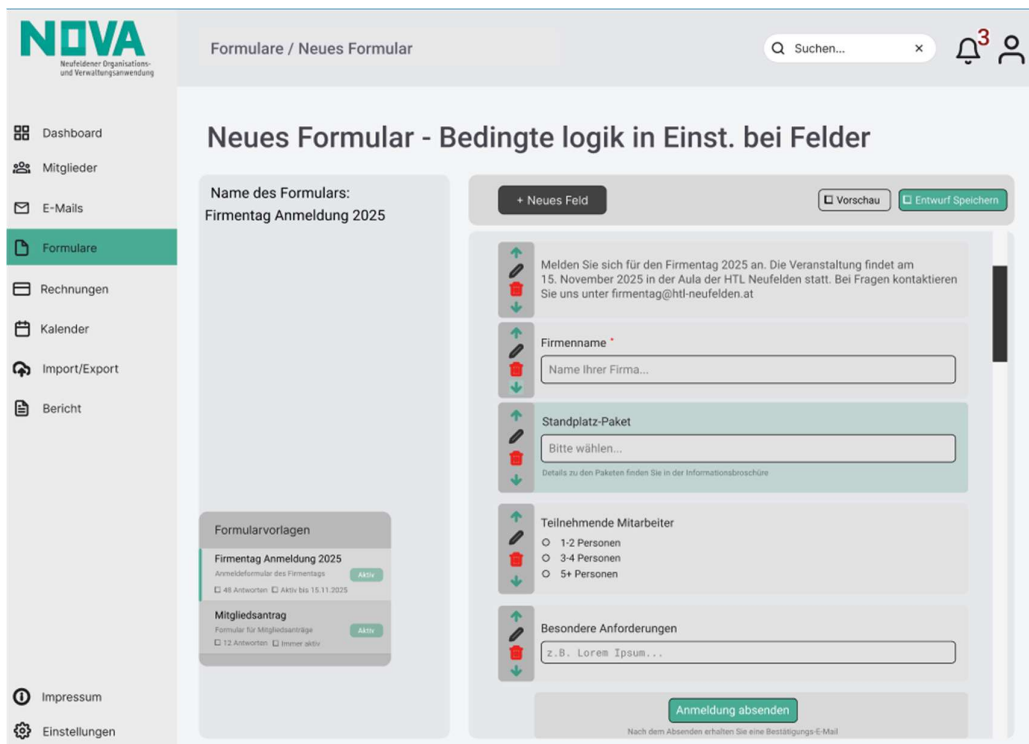


Figure 9: NOVA-Form-BUILDER

Additional wireframes were created during the design phase; these will be presented in detail in Chapter 2 Frontend.

The transition from these initial wireframes to the final implemented Angular components ensured that the visual hierarchy and the intended user interactions were maintained throughout the development process.

2.7 Prototypes

Before fully integrating complex features and third-party libraries into the main system architecture, the project team developed several technical prototypes, also known as Proof of Concepts (PoCs). The primary objective of this phase was to mitigate technical risks by evaluating the feasibility, performance, and API structures of critical components in an isolated environment. By testing these functionalities outside the main NOVA codebase, the team could ensure that the chosen libraries met all project requirements without compromising the integrity of the core application.

The most significant technical prototypes developed during this phase included:

- **PDF Generation (iText7):** A core requirement of the financial module is the automated generation of bulk invoices. A standalone prototype was built utilizing the *iText7* library to verify that dynamic data (such as member details, product items, and calculated totals) could be reliably mapped into a structured, well-formatted PDF document. This prototype ensured that font handling, table generation, and layout styling worked flawlessly before database integration.
- **Mail Client Integration:** To fulfill the communication requirements, the system needed a robust mechanism to dispatch emails (e.g., sending out the generated invoices or form invitations). A prototype was implemented to test the SMTP and IMAP protocols. This ensured that automated emails could be constructed, authenticated, and dispatched successfully to various email providers without causing synchronous blocking issues in the backend.
- **QR Code Generation:** A prototype for generating QR codes was developed to explore dynamic graphical data encoding. This feature was tested to ensure that generated codes—which can be used for quick form access, event check-ins, or standardized payment slips—were instantly scannable, correctly formatted, and easily embeddable into the frontend or exported PDFs.

The insights and functional code snippets gained from these isolated technical prototypes significantly accelerated the subsequent implementation phase. They provided a proven technical foundation, preventing architectural bottlenecks during the core development of the NOVA backend services.

3. Project Management

This section outlines how communication with stakeholders was handled, how milestones were organized, how milestones were planned and monitored and how the project structure supported efficient collaboration. All team members communicated regularly, held meetings with the project advisors and the client and continuously aligned progress with the project's goals.

3.1 Team Coordination

Several communication channels were used to coordinate work and keep all stakeholders informed. In general, emails were used to notify project advisors about upcoming meetings with the client. In case urgent support was needed, contacting the advisors directly at the school was possible.

Internal communication took place through a **Discord** (Discord, 2026) server, regular inperson meetings and **Azure DevOps** (Azure DevOps). Initially a strict separation of responsibilities was planned, although many development stages, such as deployment or resolving merge conflicts, required close teamwork and support.

Throughout the project, the project advisors continuously acted to motivate and helped communicating with the stakeholders. Their support was imperative for coordinating deadlines as well as meetings with the client. Thanks to their continuous support, permanent improvements could be made, significantly contributing to the success of NOVA.

3.2 Milestones

NOVA was specifically developed to simplify the organization of school association events, such as Company Day. As a result, several key milestones had to be implemented early in the project, for example the form system used for invitations to invite companies. The project was divided into 4 critical phases:

- Phase 1: The project started by designing the database structure (see Database) and building the API base structure. These elements established the foundation for the UI and further development. Authentication and Authorization mechanisms were added to protect critical API routes.

- Phase 2: This phase focuses on building core functionality. The base UI was created, giving NOVA its characteristic appearance. Additionally, modules such as member management, as well as an email system were implemented to enable first functional demonstrations for the client.
- Phase 3: Invoices, forms and calendar features were developed, expanding NOVA into a more complete system.
- Phase 4 – Testing and finalization: The period beyond October was planned to serve the purpose of testing and deploying.

Generous buffer times were planned to account for contingencies during deployment and to ensure qualitative code. Prototypes (see chapter 2.7) were developed in April to analyse project feasibility. All remaining open questions were resolved by June. The NOVA logo (Figure 2: NOVA-Logo) and milestone plan were presented on June 24, 2025, during the project kick-off meeting (HTL-Neufelden-Calendar).

Following Gantt chart illustrates all development stages, estimated timelines and task dependencies:

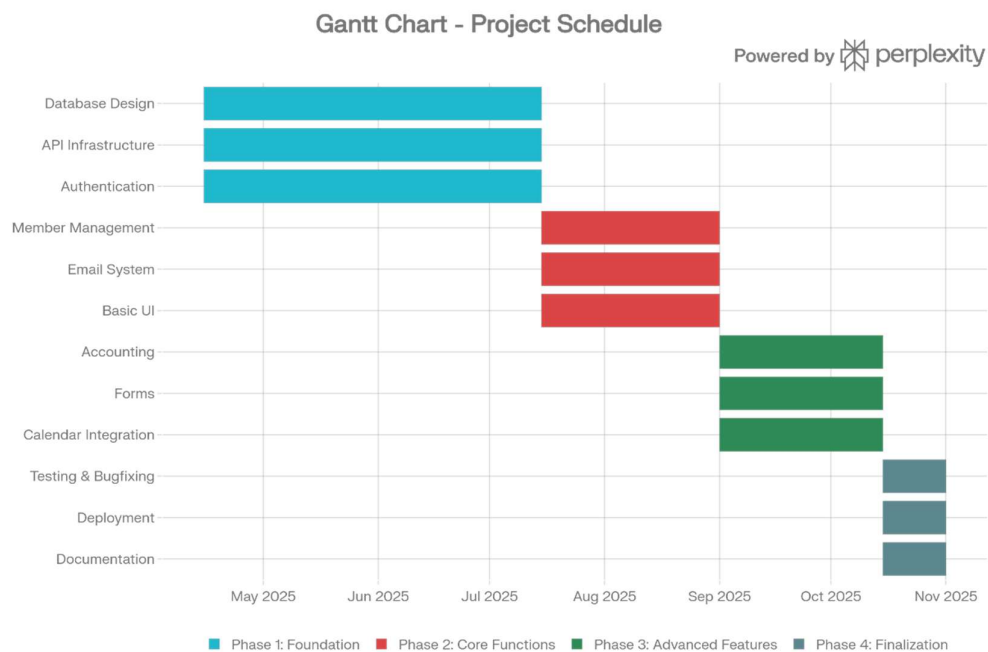


Figure 10: Gantt Chart – Timeline (*Generated by Perplexity with NOVA's data*)

3.3 Tech Stack

NOVA was implemented using a modern web application stack consisting of Angular in the frontend, ASP.NET in the backend and Microsoft SQL Server (MSSQL) as the database system. Several frameworks for identification, data access and deployment were used to accelerate the development process. This tech-stack is commonly used in professional web application solutions.

Figure 11 shows technologies used in NOVA's repository, grouped by categories. In total, twelve essential technologies were used.

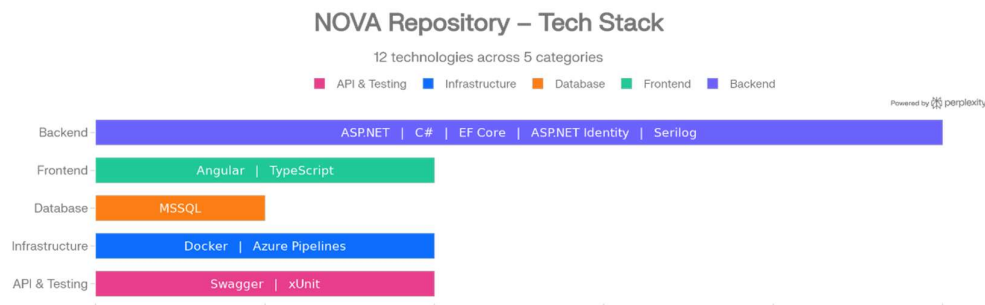


Figure 11: Tech Stack (Generated by Perplexity.ai with NOVA's data)

This tech stack required the use of multiple programming languages, including C#, TypeScript and T-SQL for MSSQL. At the beginning of the project, all team members had solid experience with .NET (C#), docker and MSSQL, while Angular was completely new to the group.

Due to his internship experience with Angular, Mr. Schopf initially took over all frontend development tasks. In coordination with the team, Mr. Niederleithner was responsible for the database setup, while Mr. Thumfart focused on the API. Later, both Mr. Niederleithner and Mr. Thumfart learned Angular with the help of Mr. Schopf, resulting in a setup, where Mr. Schopf concentrated on frontend development and Paul and David handling Full stack tasks.

3.4 Project Structure & Responsibilities

Based on the tech stack, a solution based on a ASP.NET Core Web API and Angular was created. Furthermore, class libraries and a xUnit test project was added. Below

Backend (.NET Solution):

- SchulvereinVerwaltung.API: ASP.NET Core Web API with REST endpoints, controllers, configurations and authentication/authorization
- SchulvereinVerwaltung.Core: Class library containing models, interfaces for services/repositories, business logic and validation.
- SchulvereinVerwaltung.Infrastructure: Classlibrary integrating external services, such as Entity Framework Core (see chapter 4.1) and database access.
- SchulvereinVerwaltung.Services: Class library for implementing business logic, such as form management and invoice generation.
- SchulvereinVerwaltung.Tests: xUnit (xUnit, 2026) test project for automated testing.

Frontend (Angular):

- SchulvereinVerwaltung-Client: Angular application with components, services, routing, assets google authentication, and Typescript interfaces mirroring API models.

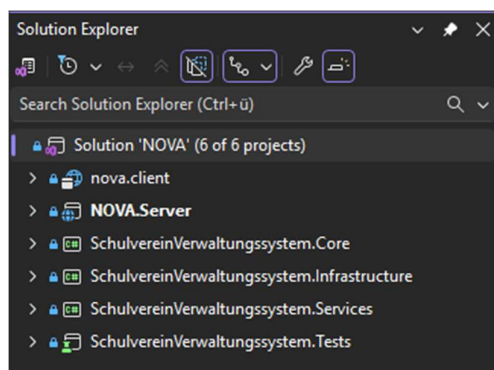


Figure 12: Project- Solution in Visual Studio

Team Responsibilities

This table shows all responsibilities grouped by team members:

Responsibilities	Initially	Latest
David Niederleithner	Infrastructure and Services	Full stack
Fabian Schopf	Frontend	Frontend
Paul Thumfart	Backend	Backend and additional Full stack tasks

Table 1: Project Responsibilities

3.5 Version Management

This project used Azure DevOps for source control, task management and pipelines.

3.5.1 Azure DevOps

Azure DevOps served as the central platform, providing version control and documentation tools.

(DevOps, 2026)

3.5.2 Feature Branching

Development followed a feature branch style, ensuring that new features are developed isolated before being merged into the main branch.

3.5.3 Work Item Management

Tasks were tracked with work items in DevOps, linking commits and pull requests to project requirements and progress tracking.

4. Technologies and Architectural Foundations

The technological foundation of NOVA is built upon modern frameworks, libraries, and design principles that collectively ensure maintainability. This chapter explores the core technologies, external dependencies, and architectural patterns that shaped NOVA's implementation across both backend and frontend environments.

4.1 Libraries & NuGet Packages

NOVA makes extensive use of external libraries and NuGet packages to accelerate the development process. The extensions ease complex tasks, such as authentication or database access. In case of authentication and authorization, ASP.NET Identity (below) requires minimal code in NOVA, thereby needing to be credited like other libraries as well.

Backend

ASP.NET Identity was a random sample of several key NuGet packages used in NOVA's backend. Yet, more critical packages contributed to domain features, like Entity Framework Core, Google Apis, Serilog, QuestPDF, QRCode, Moq, EPPlus and many others. These well-established frameworks enabled the rapid develop key features and contributed to NOVA's overall success.

Table 2 provides a comprehensive overview of all external dependencies integrated into the NOVA .NET assembly, detailing the specific version, primary purpose, and licensing information for each NuGet package.

Table 2: NuGet Packages with Versions, Purpose and Licenses (Refined by Perplexity.ai using existing table/ NOVA's data)

Frontend

NOVA's frontend also relies on a set of packages to deliver a qualitative and responsive user experience. Angular Material serves as the main UI library, providing components such as data tables and dialogs. Beyond Angular Material, packages like Angular CDK, RxJS and NgRx handling, for example, asynchronous data streams. These libraries allowed the NOVA team to singly focus on application logic, yet providing decent user experience.

The following table lists all third-party Packages included in NOVA's Angular assembly, with version, purpose and license information please refer to Table 3: Angular Dependencies (Generated with Perplexity.ai using NOVA's data) at the end for detailed information.

Licenses

Since NOVA is a non-commercial diploma project, all libraries are free to use under their respective terms. Most licenses, predominantly MIT, Apache 2.0 and BSD, explicitly permit use for academic purposes and noncommercial purposes without restrictions. Full compliance is ensured, with commercially restricted or proprietary license, such as EPPlus and FluentAssertions, being free for non-commercials.

Licenses, their terms and reference URLs are listed in Table 4: License Analysis (Refined by Perplexity.ai using NOVA's data).

4.2 Design Patterns

The code of NOVA follows well-established software design patterns that enforce separation of concerns and enable high test coverage. The patterns below are applied consistently across the solution.

4.2.1 Repository Pattern

Every domain entity, such as members or invoices, has an interface (e.g., `IMemberRepository`, `IInvoiceRepository`) and a concrete implementation class (e.g., `MemberRepository`, `InvoiceRepository`). This abstracts all data access logic from the service layer, making this layer swapable and testable. Data access logic can represent database access in backend or API calls in frontend.

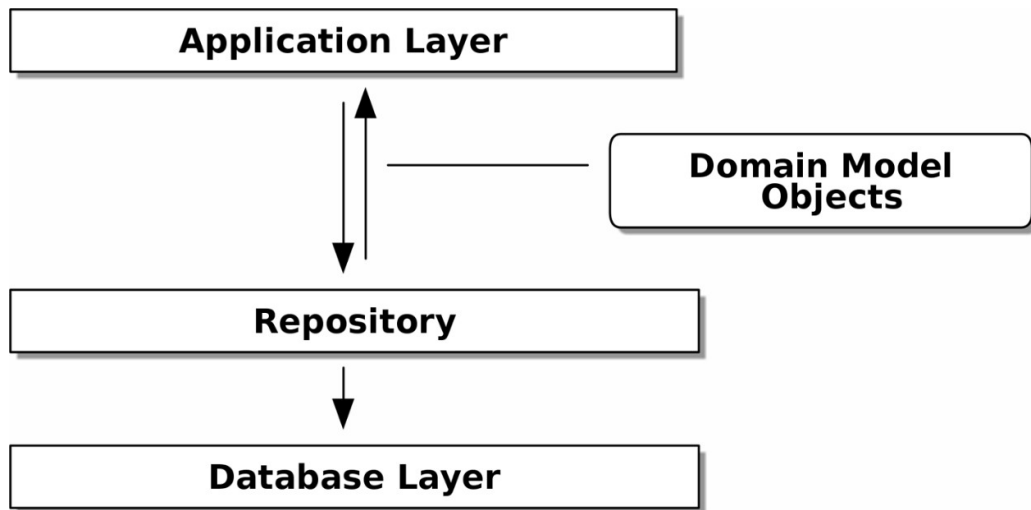


Figure 13: Repository Pattern (The Blue Book, 2026)

4.2.2 Generic Repository Pattern

`IGenericRepository<T>` provides a generic repository interface for `GenericRepository<T>`, containing methods like `GetAllAsync`, `AddAsync`, Due to containing essential CRUD operations, Domain specific repositories extend this repository, overriding only domain specific logic. This reduces duplicate code across more than a dozen backend repository implementations.

4.2.3 Service Layer Pattern

A Service layer, such as `MemberService`, separates vital domain business logic from API controllers. Numerous services coexist in the `NOVA.Services` class library with their corresponding interfaces. This enforces the Dependency Inversion Principle, keeps controllers slim and enables critical business logic testing.

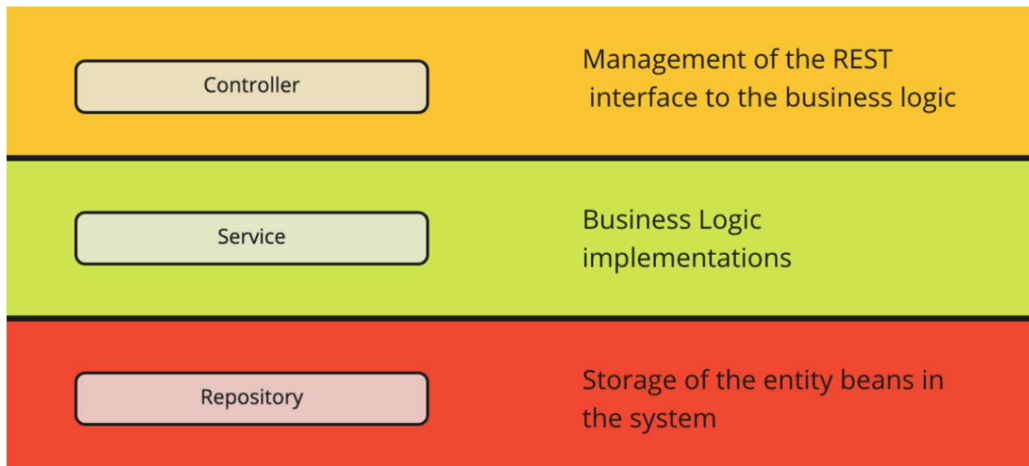


Figure 14: Service Layer Pattern (LinkedIn, 2026)

4.2.4 Facade Pattern

High-level domain service classes, such as `MemberFacades` or `InvoiceFacade`, act as facades and exposing only refined data. These facades manage one or more subsystems, simplifying access to complex features in external services. For example, angular pages use these facades for API calls while remaining unaware of the underlying complexity. This principle contributes to avoid duplicate code and strictly enforces separation of responsibilities.

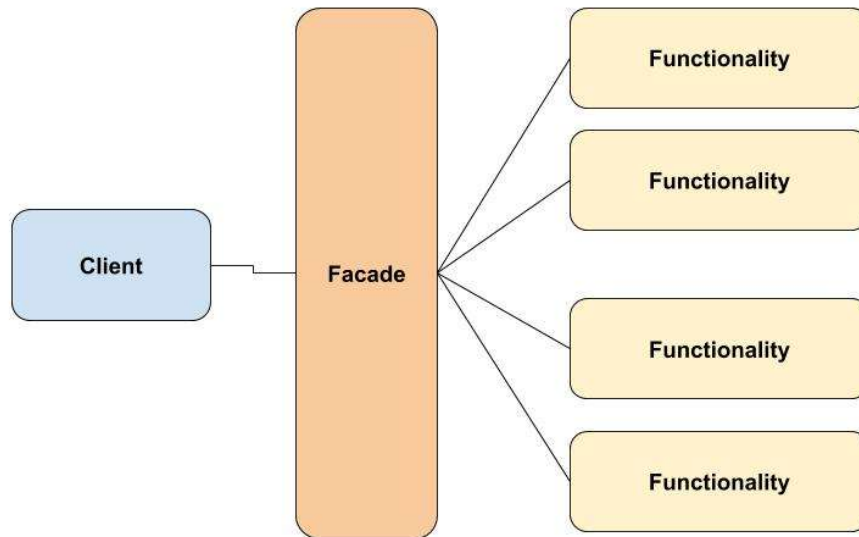


Figure 15: Facade Pattern (Peiretti, 2026)

4.2.5 DTO Pattern (Data Transfer Object)

The `NOVA.Core` class library provides DTOs separating the API's data context from internal domain entities. DTOs like `MemberDto` are used by `IMemberService`, mapping only necessary properties without revealing database entities or sensitive fields. Dedicated extension classes handle entity-DTO conversions, ensuring stability even if the underlying schema evolves. This keeps controllers thin and protects NOVA's routes with consistent, secure responses.

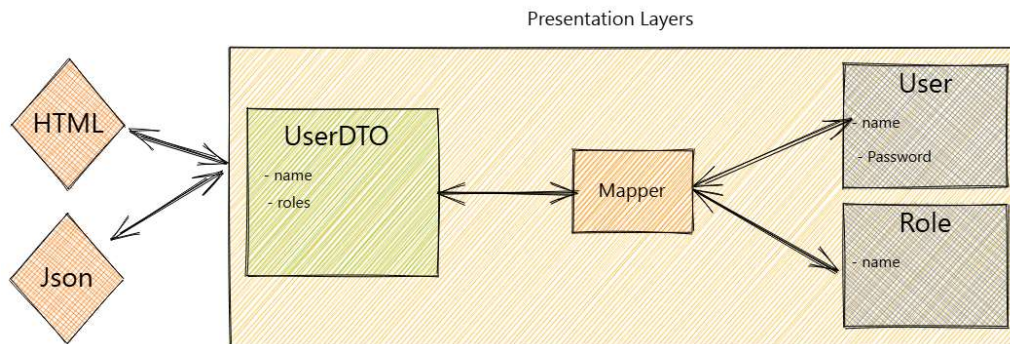


Figure 16: Data Transfer Objects Pattern (Baeldung, 2026)

4.2.6 Middleware Chain Pattern

In ASP.NET Core each middleware component can inspect, change the HTTP context or pass it onto the next handler. A custom `ExceptionHandlerMiddleware` plugs into request pipeline following the Chain of Responsibility Pattern (Dofactory, 2026), handling specific exceptions with `try / catch`. On success the request passes. On failure a response with a corresponding message and code is sent.

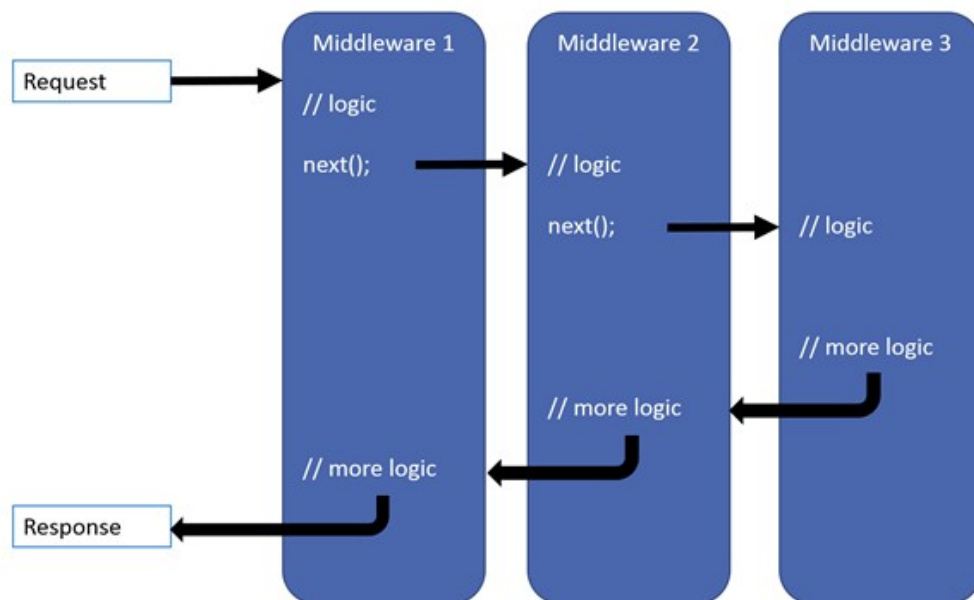


Figure 17: Middleware Pattern (Microsoft, 2026)

4.2.7 Soft Delete Pattern

The `ISoftDelete` interface marks entities as logically deletable, instead of being physically removing them from the database. A flag such as `IsDeleted` is set, preserving the existing record for audit and recovery. In NOVA, this pattern is applied to entities that support restore operations and is implemented via `SoftDeleteAsync()` in the respective domain repositories. This prevents accidental data loss while enabling full restore capabilities, supporting NOVA's usability-driven approach where safety and reversibility of user actions are prioritized.

4.2.8 Template Method Pattern

`AbstractExcelSheetProcessor` defines a skeleton of the Excel export algorithm through non-overridable methods, such as `WriteHeaders()`, which internally delegate to abstract methods like `WriteVerticalHeaders()` or `WriteHorizontalHeaders()`. Concrete classes, such as `ExcelSheetProcessor`, implement those abstract steps. This ensures excel-sheet-processors to share core logic, while remaining relatively independent.

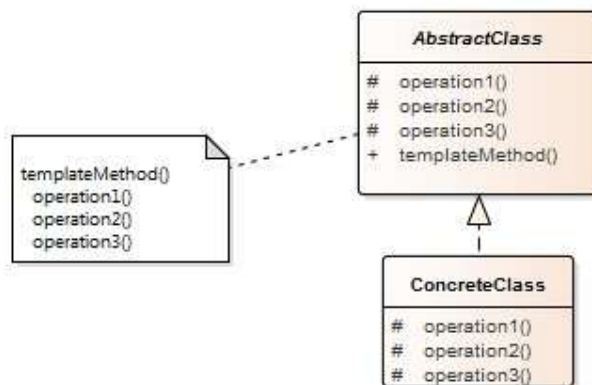


Figure 18: Template Method Pattern (Heise, 2026)

4.2.9 Strategy Pattern

An `IFilter<T>` interface defines a single `Filter(IEnumerable<T> values)` method, establishing a common contact for all filtering behaviour. Implementations, such as in

MemberFilter, GroupFilter, EmailFilter and FormFilter, implement specific filtering logic that can be swapped at runtime. This decouples the “what to filter” decision to “how to filter”.

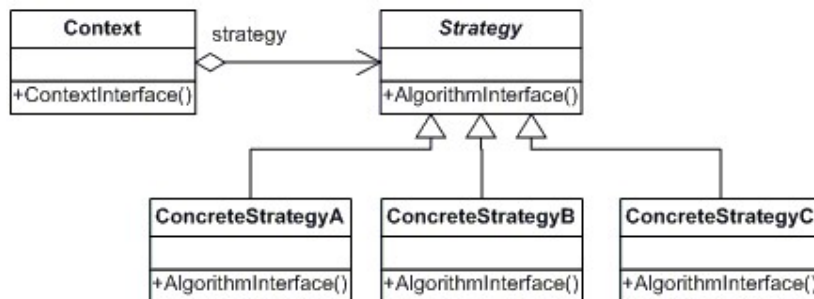


Figure 19: Strategy Pattern (Dofactory, 2026)

4.2.10 Dependency injection

Dependency Injection (DI) promotes loose coupling by delegating the creation and binding of dependent objects to an external container, the “injector”, rather than the client itself. Instead of classes constructing their own dependencies, required objects are “injected” at runtime. This allows implementation to be replaced or extended without modifying dependent code, thereby increasing maintainability and testability.

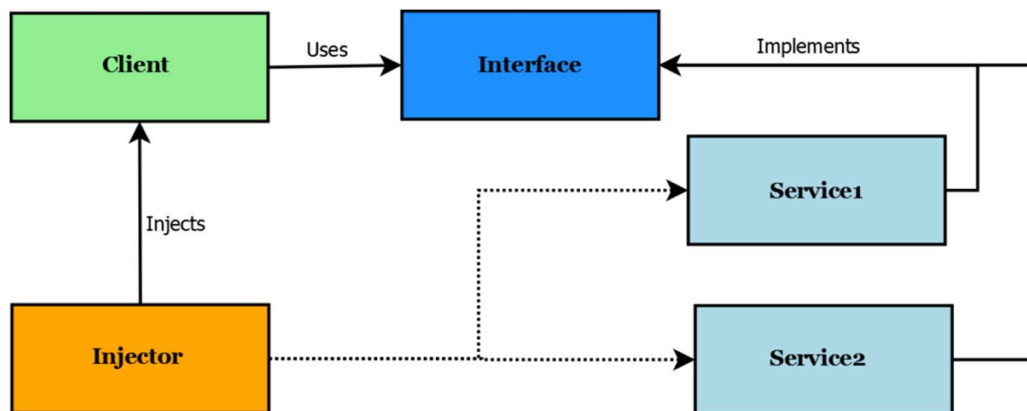


Figure 20: Dependency Injection (Sanjay, 2026)

Dependency Injection is used extensively throughout NOVA. Both the ASP.NET Core backend and the Angular frontend offer native support for DI. This reduces object construction overhead and allows services to be managed across the application's lifetime.

An example can be seen in NOVA's repository architecture. The `MemberRepository` class implements its corresponding interface:

```
public class MemberRepository(DatabaseContext context)
    : GenericRepository<Member>(context), IMemberRepository {
    ...
}
```

Within the ASP.NET Core pipeline, this repository is registered with its interface inside the dependency container:

```
services.AddScoped<IMemberRepository, MemberRepository>();
```

Once Registered, classes that depend on this repository can request it via constructor injection:

```
public class MemberServices :IMemberServices {
    private readonly IMemberRepository memberRepo;
    private readonly IExportServices exportService;

    public MemberServices(
        IMemberRepository memberRepo, IExportServices exportService) {
        this.memberRepo = memberRepo;
        this.exportService = exportService;
    }
    ...
}
```

4.3 Design Principles

NOVA's design principles guided the project towards an efficient state. Modularity, separation of concerns and modern practices were applied throughout the entire project. These principles are discussed below.

4.3.1 Development & Deployment Principles

CI/CD Pipeline

Committing onto the production branch triggers Azure Pipelines for automated testing and building and deployment. Deployment is further discussed in chapter 8.

Environment Separation and Data Seeding

To ensure a secure, maintainable, and efficient software lifecycle, the NOVA system strictly separates its development and production environments. This foundational architectural decision guarantees that sensitive, real-world association data remains entirely isolated from experimental development and testing processes.

Development Environment and Containerization

For the local development environment, a containerized infrastructure utilizing *Docker Compose* is employed. This approach ensures that all project members operate on identical system configurations, eliminating discrepancies between individual local setups.

Automated Data Seeding

To facilitate comprehensive and safe testing without the risk of exposing or corrupting real user data, the development environment is designed to automatically provision itself with a controlled set of mock data. Upon spinning up the Docker containers, an initialization process is triggered for the database container. This automated seeding is orchestrated through a dedicated shell script (`entrypoint.sh`), which utilizes the `sqlcmd` command-line utility to execute a predefined SQL script (`init.sql`).

This script sets up the baseline schema and populates the database with dummy records—such as test members, conceptual invoices, and generic calendar events. Consequently, developers are provided with a fully functional and populated system state immediately after the container deployment. This significantly streamlines the development workflow,

enabling the team to test complex edge cases—like bulk invoice generation or pagination—in a safe, isolated sandbox.

Production Environment

In contrast, the production environment strictly omits these initialization scripts to ensure that the live database starts with a clean slate and only accepts cryptographically secure, legitimate user input. By cleanly decoupling these environments, NOVA achieves a high standard of data privacy while maintaining maximum developer velocity.

4.3.2 Architectural Principles

Layered Architecture

NOVA follows a layered structure: `Core`, `Client`, `Database`, `Infrastructure`, `Services` and `Server`. Each layer has a distinct role, separating responsibilities. For more details see chapter 3.4.

Service oriented design

NOVA's functionality is divided into domain services like `MemberService` or `InvoiceService`. This logic grouping eases updates and testing. Further details in chapter 6.2.2..

Data Abstraction

Repositories facilitate database access via Entity Framework Core, abstracting raw SQL-queries from the business logic. For more details see chapter 5.6.1.

4.3.3 Frontend Principles

Component-Based Architecture

The NOVA client follows a strict component-based structure utilizing the Angular framework. The user interface is decomposed into distinct, modular building blocks divided into overarching page views (e.g., `members-page`, `dashboard`) and reusable, self-contained UI components (e.g., navigation bars, data tables, and modal dialogs). This approach encapsulates HTML, SCSS, and TypeScript logic per component, promoting reusability and simplifying UI maintenance.

Separation of Concerns

UI rendering and business logic are strictly decoupled within the client application. While components are kept lightweight and focused exclusively on the presentation layer, dedicated Angular Services (e.g., `MemberService`, `InvoiceService`) are utilized to handle complex data formatting and HTTP communication. This isolation aligns with the backend's service-oriented design and ensures highly testable frontend code. Further details in chapter 7.X.

Facade Pattern and Data Abstraction

To manage application state and streamline API interactions, the frontend implements the Facade design pattern combined with client-side repositories. Facades (e.g., `members.facade.ts`, `auth.facade.ts`) act as an intermediary abstraction layer between the UI components and the underlying data services. This shields the presentation layer from raw HTTP requests and complex data-fetching logic, ensuring a unidirectional and predictable data flow. For more details see chapter 7.X.

Component based UI

The angular component architecture is used to organize

Material Design

Angular Material is used for a consistent, modern look.

4.3.4 Other Principles

Logging & Observability

Serilog is used for structured logging and monitoring. For more details see chapter 6.2.6.

5. Database

The database layer forms the persistent foundation of NOVA, storing and managing every entity the system operates on — from member records and invoices to form submissions, audit logs, and GDPR consent. This chapter describes the database technology selected and the rationale behind that choice, the Code-First schema design philosophy adopted with Entity Framework Core, and the full domain entity model that reflects the administrative requirements of the Schulverein HTL Neufelden. Subsequent sections cover the access management strategy, the containerized development environment, and the production database deployment (see chapters 5.3 through 5.6).

5.1 Concept

Technology Selection: SQL Server

NOVA's persistence layer is built on Microsoft SQL Server (Developer edition for local development, Standard edition in production). The selection of SQL Server over alternative relational database systems was driven by three converging factors.

First, native ecosystem alignment: the entire backend is implemented on the .NET 8 platform using ASP.NET Core (see chapter 6.1). Microsoft SQL Server integrates natively with Entity Framework Core and offers first-class driver support through the Microsoft.Data.SqlClient package, eliminating any impedance mismatch that would arise with cross-vendor stacks. Second, enterprise feature set: SQL Server provides row-level locking, filtered unique indexes, check constraints, and reliable transaction management — all of which are actively exercised within the NOVA schema (see chapter 5.2). Third, institutional fit: the production instance runs on the internal server infrastructure of HTL Neufelden, where SQL Server licences are already available through the school's Microsoft agreement, reducing operational overhead.

Alternative systems such as PostgreSQL or MySQL were considered but rejected. PostgreSQL, while technically comparable, would have introduced an additional operational dependency in an environment where SQL Server administration skills are already present. MySQL lacks native support for the `nvarchar(max)` column type, which is required for storing the JSON-serialised field values of dynamic form submissions (see chapter 5.2, Form domain).

Code-First Approach with Entity Framework Core

The schema is managed exclusively through EF Core's Code-First workflow. Under this approach, C# entity classes residing in **SchulvereinVerwaltungssystem.Infrastructure** are treated as the single source of truth for the database schema. Migrations — generated via the EF Core CLI — translate incremental model changes into idempotent SQL scripts that are applied to the target database at startup.

This strategy yields several architectural advantages. Schema changes are version-controlled alongside application code, making the full evolution of the database schema auditable through Git history. The dotnet ef migrations add workflow enforces an explicit, reviewed

step between model change and schema mutation, reducing the risk of accidental destructive alterations in production. At the time of writing, 28 migration files reside in **Schulverein-Verwaltungssystem.Infrastructure/Migrations/**, capturing the full lifecycle of the schema from the initial ASP.NET Core Identity tables through the introduction of the bulk invoice system, the dynamic forms engine, and the invoice template subsystem.

Schema creation and migration application are performed automatically on application startup via **InitializeDatabase.cs** in **NOVA.Server/**, which calls `Database.MigrateAsync()` against the configured **DatabaseContext**. This guarantees that any environment — including freshly provisioned Docker containers (see chapter 5.4.1) — is always in the correct schema state without manual intervention.

Schema Design Philosophy

The schema design follows a domain-grouped, normalised structure. Entities are partitioned into nine functional domains — Person, Finance, Email, Form, Calendar, Group, Settings, GDPR, and Audit — each mapping directly to a feature area of the application. Foreign keys enforce referential integrity at the database level, with delete behaviours chosen per relationship: CASCADE where child entities have no independent meaning, SET NULL where the relationship is optional and the parent may be independently removed, and RESTRICT or NO ACTION where orphan prevention is paramount (see chapter 5.2 for per-entity detail).

Unique indexes are placed on all business-key columns — **InvoiceNumber**, **MemberNumber**, **CustomerNumber** on **Debtor**, **TemplateName** on **InvoiceTemplate** — ensuring data integrity independently of application-layer validation. Composite indexes on frequently filtered column combinations (e.g., (FormId, SubmittedAt) on **FormSubmission**, (EntityType, EntityId) on **Document**) support query performance without requiring denormalisation.

All timestamps default to `GETUTCDATE()` at the SQL level, making time zone handling consistent across environments regardless of host clock configuration.

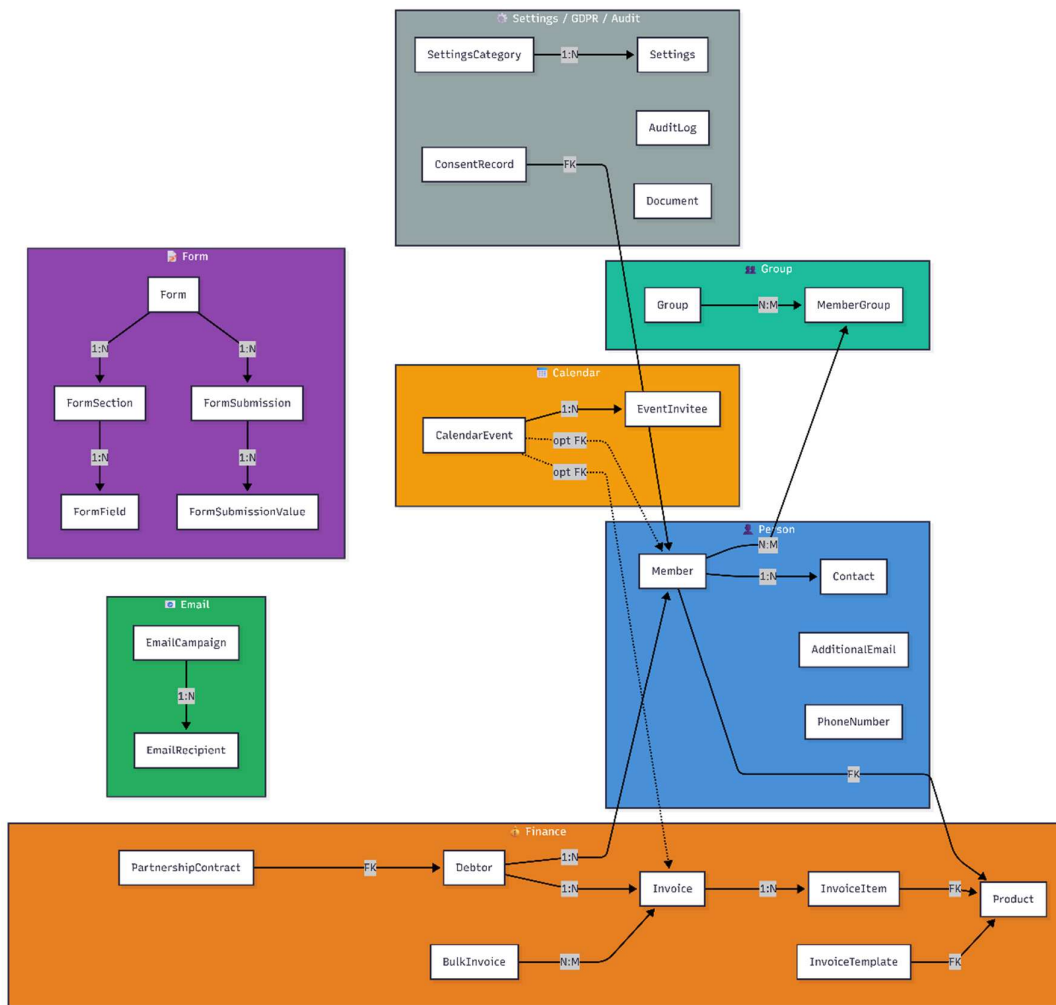


Figure 21: Entity Relationship Diagram (Generated by Claude using NOVA Data)

5.2 Entities

DatabaseContext

The **DatabaseContext** class in **SchulvereinVerwaltungssystem.Infrastructure/Data/Data-baseContext.cs** is the central EF Core component bridging the C# domain model and the SQL Server database. It extends **IdentityDbContext<NovaUser>**, inheriting the full ASP.NET Core Identity schema — users, roles, claims, and tokens — in addition to NOVA's own 31 **DbSet** properties.

```
// DatabaseContext.cs - DbSet declarations (excerpt)
public class DatabaseContext : IdentityDbContext<NovaUser>
{
    // Person domain
```

```

    public DbSet<Member>           Members           { get; set; }
    public DbSet<Contact>          Contacts          { get; set; }
    public DbSet<AdditionalEmail> AdditionalEmails    { get; set; }
    public DbSet<PhoneNumber>      PhoneNumbers      { get; set; }

    // Finance domain
    public DbSet<Invoice>           Invoices          { get;
set; }
    public DbSet<InvoiceItem>       InvoiceItems       { get;
set; }
    public DbSet<Debtor>            Debtors            { get;
set; }
    public DbSet<BulkInvoice>       BulkInvoices       { get;
set; }
    public DbSet<Product>           Products          { get;
set; }
    public DbSet<InvoiceTemplate>   InvoiceTemplates   { get;
set; }

    // Form domain
    public DbSet<Form>              Forms              { get;
set; }
    public DbSet<FormSection>       FormSections       { get;
set; }
    public DbSet<FormField>         FormFields         { get;
set; }
    public DbSet<FormSubmission>    FormSubmissions    { get;
set; }
    public DbSet<FormSubmissionValue> FormSubmissionValues { get;
set; }

    // ... remaining domains
}

```

`OnModelCreating` applies all Fluent API configuration — table names, indexes, constraints, foreign key behaviours, and enum conversions — in a single, centralised location. This keeps entity classes free of configuration attributes wherever possible, separating the domain model from its persistence mapping (Repository Pattern, see chapter 4.2.1).

Automatic timestamp maintenance is implemented by overriding both `SaveChangesAsync` and `SaveChanges`. Before delegating to the base implementation, the override iterates the `ChangeTracker` for modified **Member** and **Contact** entries and sets `UpdatedAt` to `DateTime.UtcNow`:

```
// DbContext.cs - automatic timestamp update on save
public override Task<int> SaveChangesAsync(
    CancellationToken cancellationToken = default)
{
    foreach (var entry in ChangeTracker.Entries<Member>())
    {
        if (entry.State == EntityState.Modified)
            entry.Entity.UpdatedAt = DateTime.UtcNow;
    }
    foreach (var entry in ChangeTracker.Entries<Contact>())
    {
        if (entry.State == EntityState.Modified)
            entry.Entity.UpdatedAt = DateTime.UtcNow;
    }
    return base.SaveChangesAsync(cancellationToken);
}
```

Soft Delete Pattern and `ISoftDeletable`

Several NOVA entities must never be physically removed from the database. Invoices must remain traceable for accounting purposes; member records may be required for historical reporting; documents may be referenced by active invoices. These entities implement the **`ISoftDeletable`** interface defined in **`SchulvereinVerwaltungssystem.Infrastructure/Data/Repositories/ISoftDeletable.cs`**:

```
// ISoftDeletable.cs - soft delete contract
public interface ISoftDeletable
{
    bool IsDeleted { get; set; }
    DateTime? DeletedAt { get; set; }
}
```

Entities implementing this interface — **Member, Contact, Debtor, Invoice, Bulk-Invoice, Document, EmailTemplate, and InvoiceTemplate** — are never removed with a DELETE statement. Instead, a global query filter registered in `OnModelCreating` silently excludes soft-deleted rows from all EF Core queries:

```
// DbContext.cs - global query filter for soft delete (Member example)

entity.HasQueryFilter(m => !m.IsDeleted);
```

The **GenericRepository** (see chapter 4.2.2) enforces this contract through its `SoftDeleteAsync` method, which checks for `ISoftDeletable` at runtime and sets both flags before persisting:

```
// GenericRepository.cs - soft delete implementation

public virtual async Task SoftDeleteAsync(T entity)
{
    if (entity is ISoftDeletable softDeletableEntity)
    {
        // Mark as deleted without issuing a physical DELETE
        softDeletableEntity.IsDeleted = true;
        softDeletableEntity.DeletedAt = DateTime.UtcNow;
        await UpdateAsync(entity);
    }
    else
    {
        // Fall back to hard delete for non-auditable entities
        await DeleteAsync(entity);
    }
}
```

This design applies the Soft Delete Pattern (see chapter 4.2.7) uniformly across the entire data access layer without requiring individual service implementations to be aware of the deletion strategy.

Enum Handling

All C# enums are stored as integers in the database. EF Core value converters are registered in `OnModelCreating` for every enum property, ensuring that the integer representation is written and the typed enum is read back without relying on string storage:

```
// DatabaseContext.cs - enum-to-integer conversion (representative
  examples)

builder.Entity<Member>()
    .Property(e => e.Status)
    .HasConversion(v => (int)v, v => (MemberStatus)v);

builder.Entity<Invoice>()
    .Property(e => e.Status)
    .HasConversion(v => (int)v, v => (InvoiceStatus)v);

builder.Entity<FormField>()
    .Property(e => e.FieldType)
    .HasConversion(v => (int)v, v => (FormFieldType)v);
```

Twelve enum properties across eight entity types are configured this way. Integer storage reduces column width, avoids locale-sensitive string comparisons, and decouples the database representation from enum member naming, which may change during development without requiring a schema migration.

Domain Groupings

Person

The Person domain centres on **Member**, the core entity of the entire system. A **Member** represents either an individual or a business (IsBusiness flag) and carries personal data, a unique MemberNumber, membership status (MemberStatus enum: Active, Exited), payment tracking dates, and optional linkage to a **NovaUser** identity record when system access has been granted.

```
// Member.cs - key fields (excerpt)

public class Member : ISoftDeletable
{
    [Key]
    public int MemberId { get; set; }

    [Required, StringLength(50)]
    public string MemberNumber { get; set; }    // unique index
```



```

[Required]

public MemberStatus Status { get; set; }    // stored as integer

public int? MembershipProductId { get; set; }    // FK → Product

public DateTime JoinDate { get; set; }
public DateTime? ExitDate { get; set; }

// ISoftDeletable
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }

// Audit
public DateTime CreatedAt { get; set; }
public DateTime UpdatedAt { get; set; }

// Computed — not mapped to a column
[NotMapped]
public string FullName => $"{FirstName} {LastName}";
}

```

Each **Member** may own multiple **Contact** records (sub-contacts such as accountants or deputies), **AdditionalEmail** entries typed by the EmailType enum (Primary, Secondary, Billing), and **PhoneNumber** entries typed by PhoneType. Both **AdditionalEmail** and **PhoneNumber** carry a SQL-level check constraint that enforces exclusive ownership — a row must belong to either a **Member** or a **Contact**, never both:

```

// DatabaseContext.cs — ownership check constraint (AdditionalEmails)
entity.ToTable("AdditionalEmails", t => {
    t.HasCheckConstraint("CHK_AdditionalEmails_Owner",
        "( (MemberId IS NOT NULL AND ContactId IS NULL) " +
        "OR (MemberId IS NULL AND ContactId IS NOT NULL))");
});

```

The membership type is no longer stored as an enum on **Member**; it is instead expressed as a nullable foreign key (MembershipProductId) referencing the **Product** entity. This design

normalises membership classification against the product catalogue, allowing membership prices and descriptions to be managed centrally (see chapter 6.4.3).

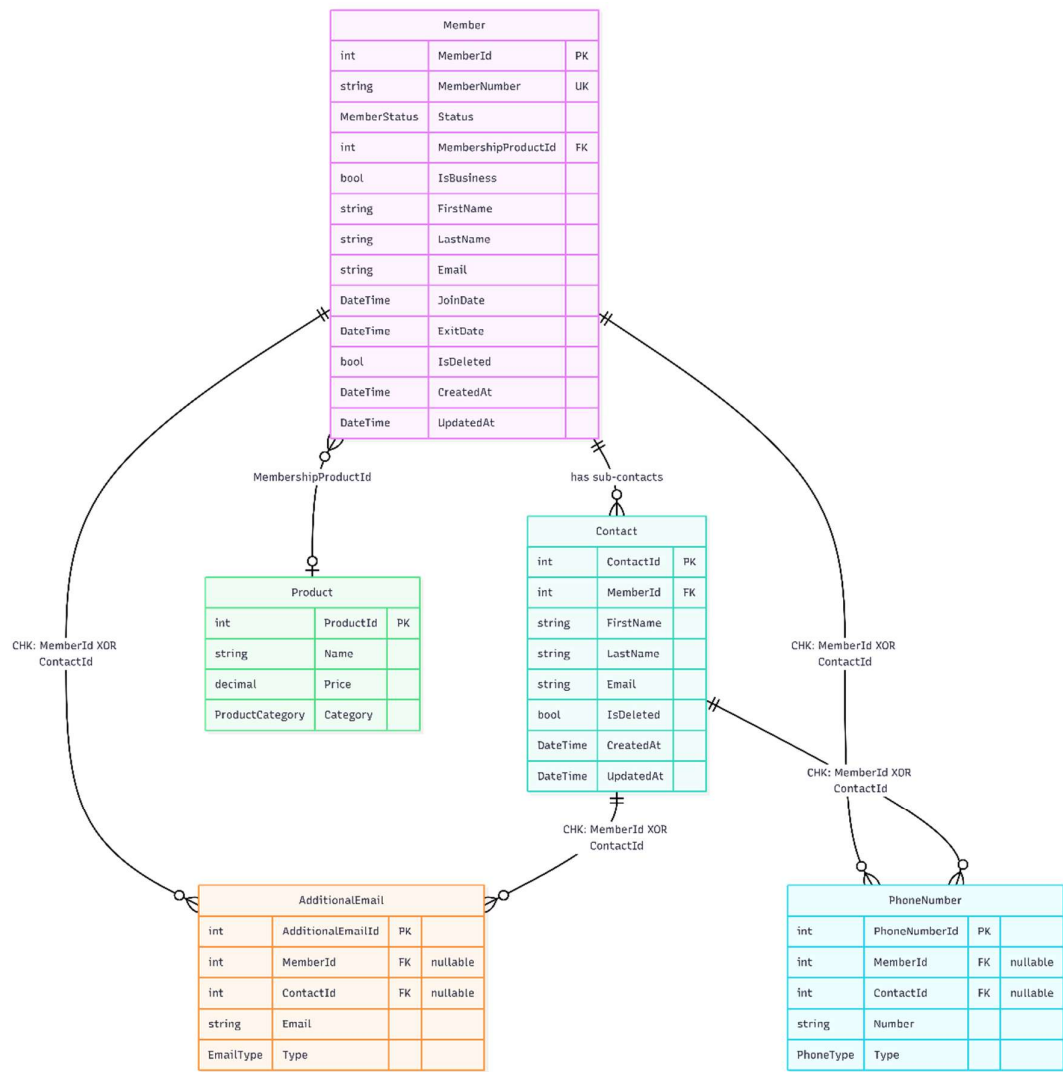


Figure 22: Person domain (Generated by Claude using NOVA Data)

Finance

The Finance domain manages all monetary entities. **Debtor** is the billing entity — it carries a unique CustomerNumber, billing and distribution contacts, optional logo document, and payment terms. A single **Debtor** may aggregate multiple **Members** (a company that sponsors several employees as members) and multiple **Invoices**.

Invoice represents an individual billing document, linked to a **Debtor** by foreign key. Its Status property (InvoiceStatus enum: Draft, Sent, Paid, Overdue, Cancelled) tracks lifecycle state. A computed TotalAmount is persisted as decimal(18,2), while NetAmount and VATAmount are derived at runtime from the associated **InvoiceItem** collection:

```
// Invoice.cs - computed VAT breakdown (not mapped to columns)
[NotMapped]
public decimal VATAmount
{
    get
    {
        if (InvoiceItems == null || !InvoiceItems.Any())
            return 0m;

        return InvoiceItems.Sum(item => item.TaxAmount * item.Quantity);
    }
}
```

BulkInvoice represents a consolidated billing run across multiple **Invoices**. Its many-to-many relationship with individual **Invoice** entities is resolved through the implicit join table **InvoiceBulkInvoiceMappings**, configured via EF Core's `UsingEntity` Fluent API with a composite primary key on (`InvoiceId`, `BulkInvoiceId`).

Product defines the catalogue of billable items — membership types, sponsorship tiers, event fees, donations. Both **InvoiceItem** and **PartnershipContract** reference **Product** by foreign key, replacing earlier enum-based type columns and enabling runtime price and category management without schema changes.

InvoiceTemplate and **InvoiceTemplateItem** form a reusable invoice blueprint system, carrying pre-configured line items, a default **Debtor**, and payment terms, from which concrete **Invoices** are instantiated (see chapter 6.4.8).

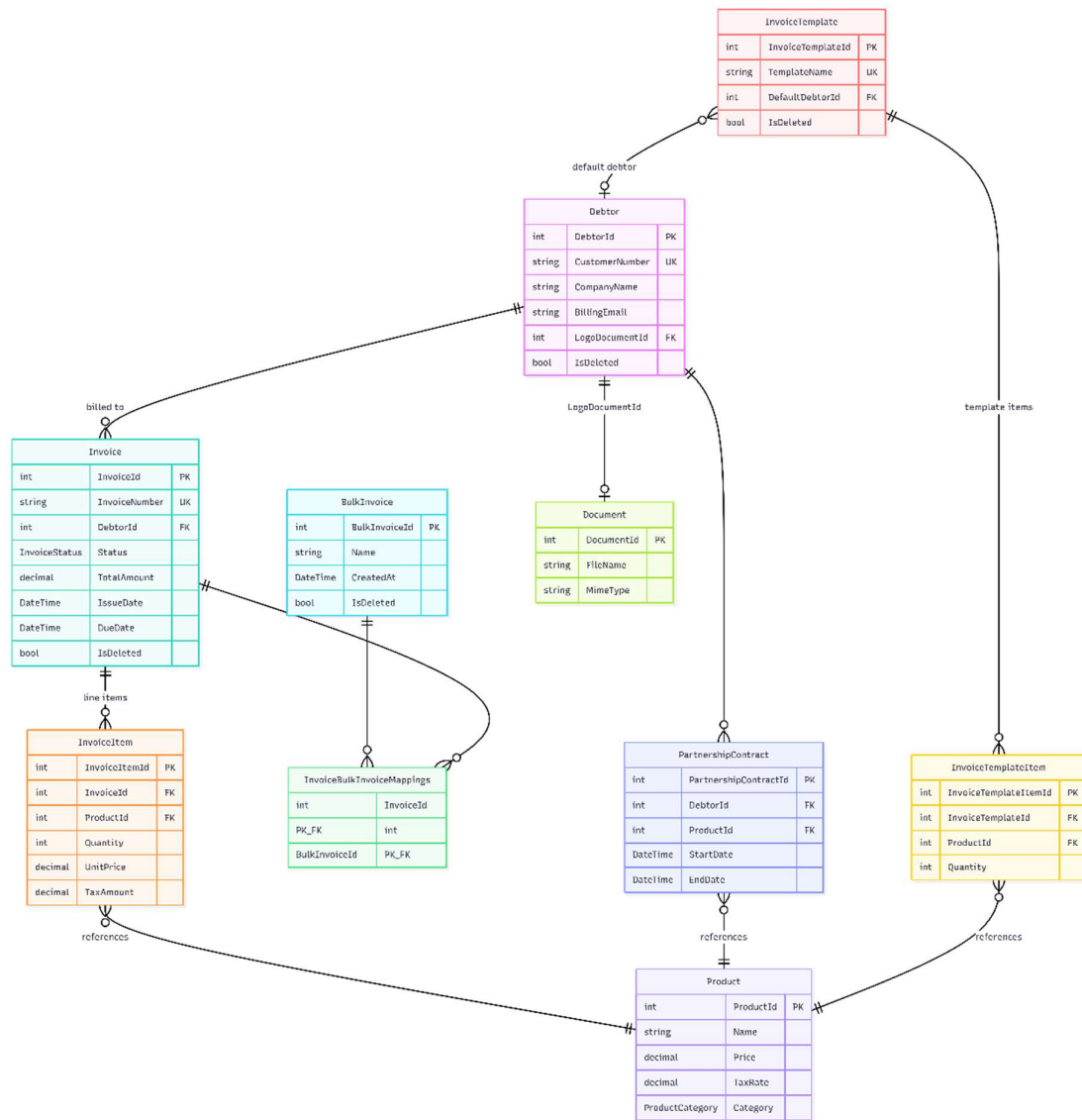


Figure 23: Finance domain (Generated by Claude using NOVA Data)

Email

The Email domain models targeted communication campaigns. **EmailCampaign** is the parent entity, carrying subject, body, send timestamp, EmailCampaignStatus (Draft, Scheduled, Sending, Completed, Failed), and a reference to the creating **NovaUser**. Each intended recipient is represented by an **EmailRecipient** row linked to the campaign, with its own EmailRecipientStatus (Pending, Sent, Failed) and an optional failure reason, enabling per-address delivery tracking.

EmailTemplate stores reusable HTML email bodies, keyed by an EmailTemplateEnum value. A unique index on TemplateType guarantees that at most one active template exists per purpose (e.g., welcome email, payment reminder), making template lookup deterministic.

Form

The Form domain implements the dynamic form builder. **Form** is the root entity, holding publication state (`IsPublished`, `PublishedAt`), an optional `MaxSubmissionCount` enforced by the `[PositiveInteger]` custom attribute (see **SchulvereinVerwaltungssystem.Core/Attributes/**), and a reference to the creating user. Its structural children follow a strict hierarchy:

- **FormSection** → ordered groupings within a form
- **FormField** → individual input definitions within a section, typed by the `FormFieldType` enum (`Text`, `Number`, `Select`, `Checkbox`, `Date`, etc.)
- **FormFieldOption** → selectable options for choice-type fields

FormLayout records versioned JSON snapshots of the rendered form layout, with a composite unique index on (`FormId`, `Version`) preserving the layout history across edits.

FormSubmission captures each completed response, and **FormSubmissionValue** stores the individual field answers as `nvarchar(max)` — accommodating free-text, serialised multi-select values, and structured JSON payloads within a single generic column type. The (`SubmissionId`, `FieldId`) composite index on **FormSubmissionValue** supports efficient per-submission value retrieval.

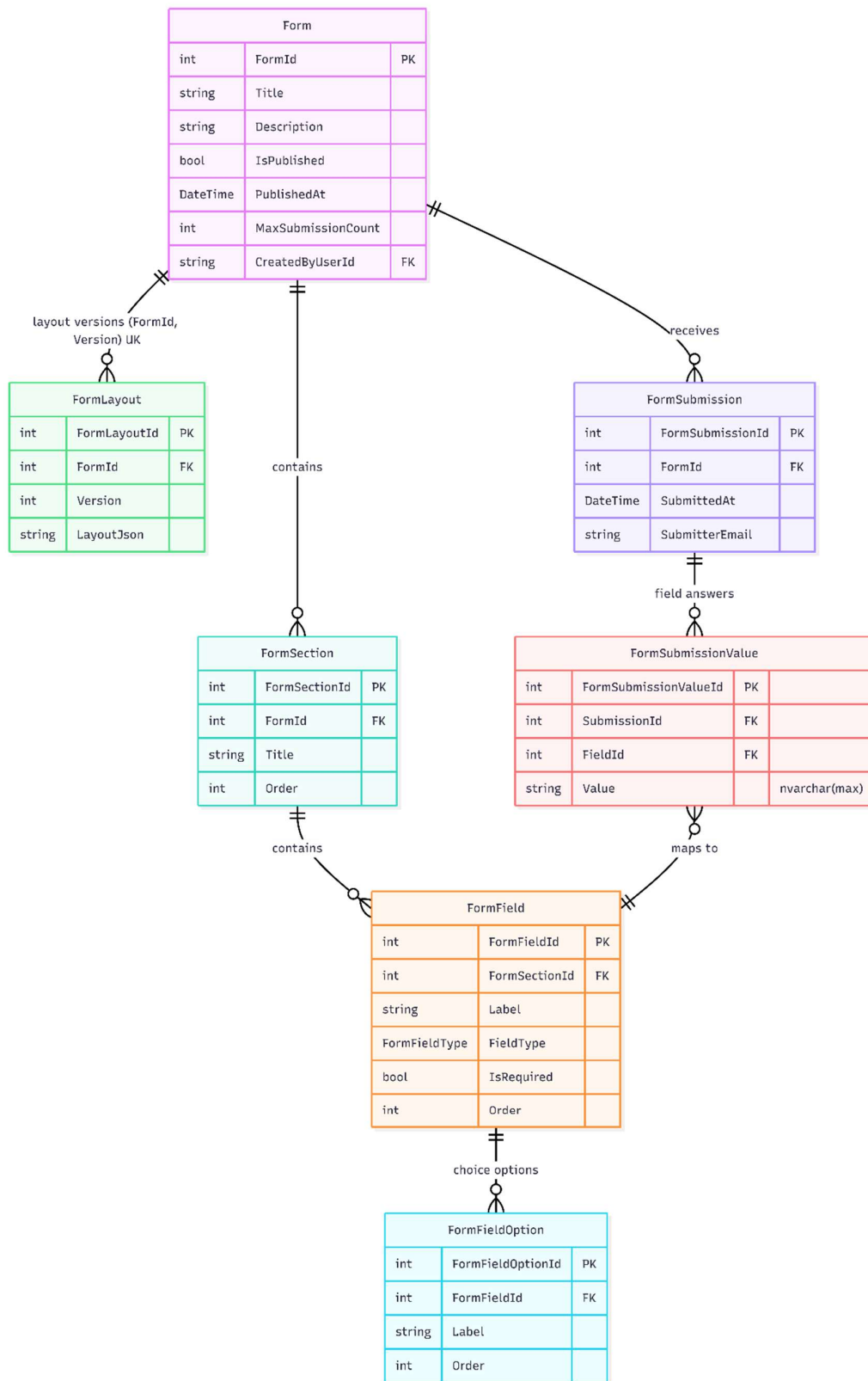


Figure 24: Form domain (Generated by Claude using NOVA Data)

Calendar

CalendarEvent persists events that are synchronised bidirectionally with Google Calendar via the Google Calendar API (see chapter 6.4.9). The `GoogleEventId` field stores the remote identifier returned by the API, enabling update and deletion operations against the external calendar. Optional foreign keys to **Invoice** and **Member** allow events to be contextually linked to billing or membership activity. **EventInvitee** records per-event attendees, each identified by email address with an optional **Member** or **Contact** link; a unique index on (`EventId`, `Email`) prevents duplicate invitations.

Group

Group categorises members into named collections typed by the `GroupType` enum (Administration, Representative, Sponsor, Communication, Alumni, Custom). The many-to-many relationship between **Member** and **Group** is modelled through the **MemberGroup** join entity, which carries a `JoinedAt` timestamp and uses a composite primary key on (`MemberId`, `GroupId`):

```
// DatabaseContext.cs - MemberGroup composite key and cascade behaviour
builder.Entity<MemberGroup>(entity => {
    entity.HasKey(e => new { e.MemberId, e.GroupId });

    entity.HasOne(e => e.Member)
        .WithMany(m => m.MemberGroups)
        .HasForeignKey(e => e.MemberId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(e => e.Group)
        .WithMany(g => g.MemberGroups)
        .HasForeignKey(e => e.GroupId)
        .OnDelete(DeleteBehavior.Cascade);
});
```

Cascading deletes are applied in both directions: removing a **Member** removes all their group memberships, and removing a **Group** removes all associated membership records, without affecting the underlying **Member** entities.

Settings

SettingsCategory and **Settings** form a two-level configuration hierarchy. Categories (e.g., "Email", "Invoice", "GDPR") group named settings entries. Each **Settings** row stores a Name (unique), a Value string, a Description, and a SettingType enum that informs the frontend of the expected input type (Text, Number, Boolean, Colour). Settings are loaded into memory at application startup and exposed to services via the **ISettingsRepository**, allowing runtime reconfiguration without redeployment (see chapter 6.4.10).

GDPR

ConsentRecord logs each instance of a member granting or withdrawing data processing consent, capturing the consent type, the timestamp (GrantedAt), and an optional expiry date. This entity directly supports the GDPR compliance requirements of the Schulverein and provides the audit trail required under Article 7 of the General Data Protection Regulation. A dedicated index on MemberId enables efficient retrieval of a member's full consent history.

Audit

AuditLog provides a tamper-evident record of all state-changing operations performed in the system. Each row captures the affected table name, the integer record identifier, the CRUD action type, JSON snapshots of the old and new field values, the acting user's identifier and username, a timestamp, the originating IP address, and a summary of which fields changed. Multiple indexes on TableName, RecordId, UserId, and Timestamp support efficient filtering across all query dimensions used by the audit reporting interface.

```
// AuditLog.cs – key audit fields
public class AuditLog
{
    [Key]
    public long    LogId          { get; set; }

    [Required, StringLength(100)]
    public string  TableName      { get; set; }

    public int     RecordId       { get; set; }

    [Required, StringLength(50)]
    public string  Action          { get; set; } // CREATE, UPDATE,
    DELETE

    public string? OldValues      { get; set; } // JSON snapshot
}
```



```
public string? NewValues    { get; set; } // JSON snapshot

public DateTime Timestamp   { get; set; }

// For DSGVO-compliant change tracking
[StringLength(1000)]
public string? ChangedFields { get; set; }
}
```

Document

Document stores metadata for uploaded files — file name, MIME type, file size, storage path, and upload timestamp. The `EntityType` and `EntityId` fields form a polymorphic reference, allowing documents to be attached to any domain entity (e.g., a **Debtor** logo, an invoice attachment) without introducing separate foreign key columns per entity type. A composite index on (`EntityType`, `EntityId`) makes document retrieval by owner efficient. **Document** implements **ISoftDeletable**, ensuring that files are not physically removed from the record until explicitly purged.

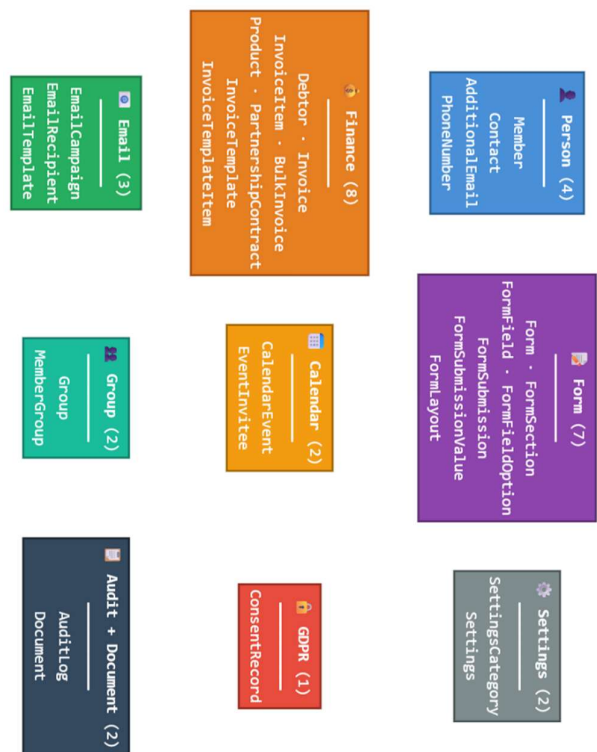


Figure 25: Domain Grouping Overview (Generated by Claude using NOVA Data)

5.3 Access Management

Securing the underlying data is a critical requirement, especially given the sensitive financial and personal information handled by the school association. Database access management is strictly regulated on two levels:

1. **Application-Level Access:** The NOVA backend API does not connect to the database using supreme administrative credentials (e.g., the sa account). Instead, a dedicated service account with restricted privileges is utilized. This account is only granted the necessary permissions to perform Data Manipulation Language (DML) operations (SELECT, INSERT, UPDATE, DELETE) and execute migrations.
2. **Environment Variable Injection:** Connection strings are never hardcoded into the source code or committed to the version control repository. During local development, connection strings are stored in `appsettings.Development.json` or local secrets. In the production environment, they are securely injected into the application via server environment variables, ensuring that production credentials remain entirely inaccessible to unauthorized personnel.

5.4 Development Database

To maintain high developer velocity and ensure consistency across different workstations, the local development database is entirely containerized and pre-configured.

5.4.1 Docker

The local SQL Server instance is hosted within a Docker container. By utilizing a declarative `docker-compose.yml` file, the exact version of the database engine, exposed ports, and volume mappings are standardized. This infrastructure-as-code approach guarantees that every project member operates in an identical data environment, effectively eliminating setup discrepancies and the classic "it works on my machine" paradigm.

5.4.2 Starting Database

Initializing the development database is a fully automated process. When a developer executes the container startup command, a customized initialization sequence is triggered to provision the database with necessary mock data for testing purposes.

This process is orchestrated by a shell script (entrypoint.sh), which waits for the database engine to become fully available. Once ready, it utilizes the sqlcmd utility to execute a comprehensive SQL script (init.sql). This script creates the initial schema, sets up default administrative users, and populates the tables with dummy records (e.g., test members, sample invoices, and calendar events). Consequently, developers are provided with a fully functional, populated system state within seconds of starting the project.

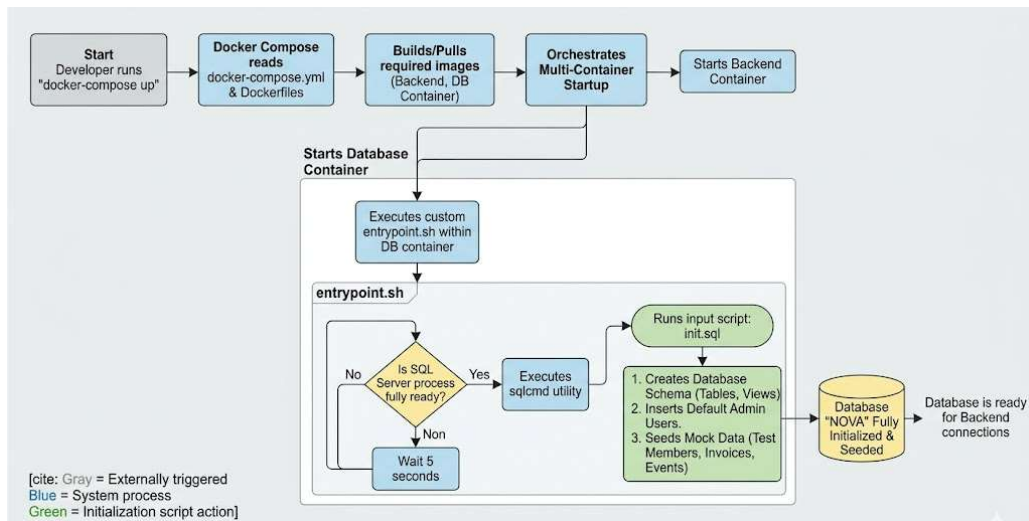


Figure 26: Flowchart Displaying Database Startup Process (Generated with Gemini Nano Banana 2 using NOVA Data)

5.5 Produktion Database

The production database architecture prioritizes data security, high availability, and strict isolation from the development lifecycle.

5.5.1 School Server

Unlike the ephemeral Docker containers used in development, the production database is hosted permanently on the internal IT infrastructure of the HTL Neufelden. Deploying the database on the school's internal servers ensures that all data remains geographically and legally within the institution's control, strictly adhering to General Data Protection Regulation (GDPR) requirements. The server is protected by the school's internal firewall, ensuring that the database port is not publicly exposed to the internet; it only accepts internal connections directly from the NOVA backend API.

5.5.2 Back Ups

To mitigate the risk of data loss due to hardware failures or critical system errors, a rigorous backup strategy is implemented on the production server. Automated cron jobs are configured to generate full database dumps on a scheduled basis (e.g., nightly). These backup files are securely transferred to off-site or redundant storage locations within the school's

network. Furthermore, transaction log backups are maintained to allow for point-in-time recovery, ensuring minimal data loss in the event of an unforeseen outage.

5.6 Database Connection

The database connection is handled by EF Core's `DbContext` abstraction, supplying all connection parameters at runtime from the ASP.NET Core Configuration System. Through this separation, it is ensured that environment-specific credentials are not hard-coded in application logic, and that switching to development, staging and production databases only requires a configuration change, rather than a code change. The two subsections below describe how the `DbContext` is registered within the Dependency Injection container, and how the connection string is supplied and overridden per environment.

5.6.1 Entity Framework Core

DbContext is registered with the ASP.NET Core DI container as a scoped service inside **NOVA.Server/Configuration/ConfigureServices.cs**. The registration uses `AddDbContext<T>`, which creates one **DbContext** instance per HTTP request — an appropriate lifetime for a unit-of-work pattern where all repository operations within a single request share the same EF Core change tracker:

```
// ConfigureServices.cs - DbContext registration
services.AddDbContext<DbContext>(options =>
    options.UseSqlServer(
        configuration.GetConnectionString("DefaultConnection"),
        b => b.MigrationsAssembly("SchulvereinVerwaltungssystem.Inf-
rastructure")
    )
);
```

Two details in this registration warrant attention. First, `configuration.GetConnectionString("DefaultConnection")` resolves the connection string by key from the active configuration source (see chapter 5.6.2 for how that source is determined per environment). Second, the `MigrationsAssembly` override is required because **DbContext** resides in **SchulvereinVerwaltungssystem.Infrastructure** while the entry-point project is **NOVA.Server**; without this override EF Core would search for migration files in the API project and fail to locate them.

Identity stores are bound to the same **DbContext** instance via the `AddEntityFrameworkStores<DbContext>()` call registered in **ConfigureIdentity.cs**, ensuring that the ASP.NET Core Identity tables (users, roles, claims, tokens) are managed by the same context and participate in the same transactions as NOVA's domain entities:

```
// ConfigureIdentity.cs - Identity stores wired to DbContext
```

```
services.AddIdentityApiEndpoints<NovaUser>()
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<DatabaseContext>();
```

On application startup, before the HTTP pipeline begins accepting requests, **InitializeDatabase.cs** in **NOVA.Server/Initialization/** performs an explicit connection check by resolving a **DatabaseContext** from a temporary DI scope and calling `CanConnectAsync()`. A failed connection throws an `InvalidOperationException`, preventing the application from starting in a partially initialised state:

```
// InitializeDatabase.cs - connection verification at startup
public static async Task InitializeDatabaseAsync(this WebApplication
    app)
{
    using var scope = app.Services.CreateScope();
    var context = scope.ServiceProvider
        .GetRequiredService<DatabaseContext>();

    var canConnect = await context.Database.CanConnectAsync();

    if (!canConnect)
        throw new InvalidOperationException("Database connection
            failed");
}
```

`InitializeDatabaseAsync` is called from **Program.cs** immediately after `app = builder.Build()`, before `ConfigureSettingsAsync()` and before `ConfigureAppPipeline()`. This ordering guarantees that the settings seeding logic (see chapter 6.4.10) and the full middleware pipeline are only initialised against a confirmed, reachable database.

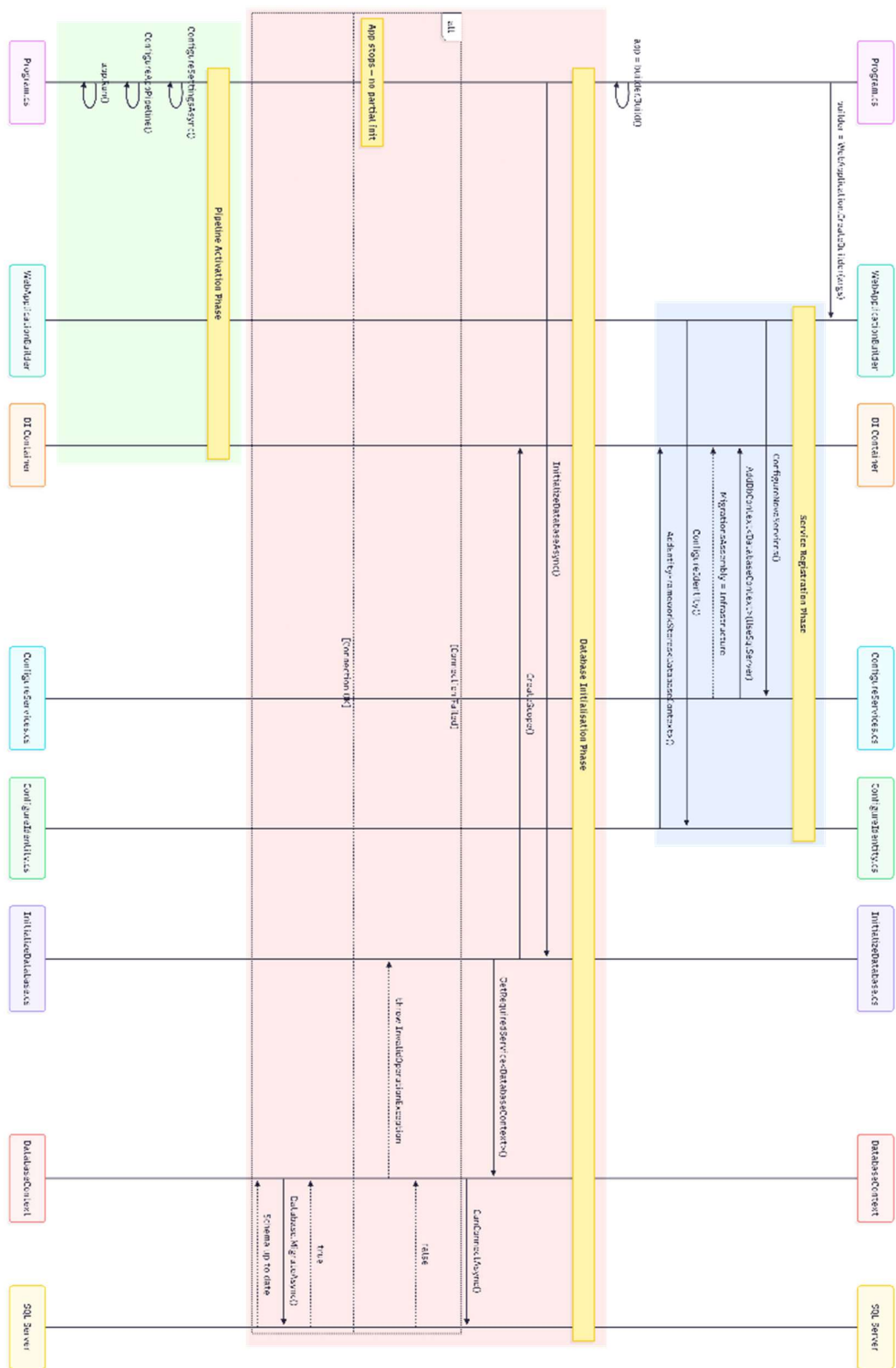


Figure 27: Startup Sequence (Generated by Claude using NOVA Data)

5.6.2 Environmental Variables

ASP.NET Core resolves configuration through a layered provider chain. For NOVA, the chain is composed as follows: **appsettings.json** supplies baseline defaults applicable to all environments; **appsettings.Development.json** and **appsettings.Production.json** overlay environment-specific overrides; and environment variables applied at process or container level take the highest precedence, overriding any value from the JSON files.

appsettings.json contains the connection string used when the application is run directly against the local SQL Server instance during initial development:

```
// appsettings.json - baseline connection string
"ConnectionStrings": {
  "DefaultConnection": "Server=127.0.0.1,1433;Initial Catalog=nova-
test-db;
    User ID=novaadmin;Password=...;MultipleActiveResultSets=True;
    Encrypt=False;TrustServerCertificate=True;Connection
    Timeout=30;"
}
```

appsettings.Production.json replaces the connection string with the school server target and switches to Windows Integrated Security, removing the need to store SQL credentials in any configuration file on the production host:

```
// appsettings.Production.json - production connection string
"ConnectionStrings": {
  "DefaultConnection": "Server=iWeb4;Initial Catalog=nova;
    Integrated Security=True;TrustServerCertificate=True;
    MultipleActiveResultSets=True;Connection Timeout=10;"
}
```

The `ASPNETCORE_ENVIRONMENT` environment variable controls which overlay file is loaded. When set to `Production` on the school server, ASP.NET Core merges **appsettings.Production.json** on top of the base file before **ConfigureServices.cs** reads `GetConnectionString("DefaultConnection")`, so no application code change is required between environments.

In the Docker Compose deployment (see chapter 5.4.1), the connection string is supplied as a process-level environment variable inside **docker-compose.yml**, using ASP.NET Core's double-underscore key notation to target nested JSON paths:

```
# docker-compose.yml - connection string injected as environment var-
iable
nova-server:
```



```
environment:
  - ConnectionStrings__DefaultConnection=Server=mssql,1433;
    Initial Catalog=nova-test-db;User ID=novaadmin;
    Password=...;Persist Security Info=False;
    MultipleActiveResultSets=True;Encrypt=False;
    TrustServerCertificate=True;Connection Timeout=30;
```

The key `ConnectionStrings__DefaultConnection` maps directly to the JSON path `ConnectionStrings.DefaultConnection`; the double underscore is the standard ASP.NET Core convention for expressing hierarchy in flat environment variable keys. Because environment variables take precedence over **appsettings.json**, the containerised API automatically targets the `mssql` service by its Docker Compose DNS hostname rather than `127.0.0.1`, without requiring a separate configuration file for the container environment.

This layered strategy means that the connection string read by `GetConnectionString("DefaultConnection")` in **ConfigureServices.cs** is always correct for the active deployment target — local development, Docker, or production — purely through environment-level injection, and no credentials are committed to source control beyond the development defaults in **appsettings.json**.

6. Backend

The backend forms the central processing and data management layer of NOVA. It is responsible for handling all business logic, persisting data, and exposing a well-defined interface through which the Angular frontend communicates. Each user interaction is processed by one of the backend's service components.

The backend operates as a RESTful Web API, ensuring no client session is maintained on the server (see chapter 2.3 System Architecture Overview). Each HTTP request originates from the Angular Single-Page Application and carries a JSON Web Token proving the caller's identity and permissions. The token is validated by the server, who executes the requested operation and returns a structured JSON response.

The solution is organized into four .NET class libraries and one API project, as introduced in Project Structure & Responsibilities (see chapter 3.4). This design follows Clean Architecture principles (see 4.3 Design Principles), ensuring that each layer has a single responsibility and that inner layers remain entirely independent of outer ones.

6.1 Web API with ASP.NET Core

The backend is implemented with ASP.NET Core, Microsoft's framework for building web applications and APIs. ASP.NET Core provides a configurable web server to handle API requests, routing, middleware processing and dependency injection (Microsoft, 2026).

In comparison to other frameworks, such as Node.js, ASP.NET Core offered three major advantages:

- Existing C# and .NET expertise for efficient development without introducing new languages.
- Native integration with Entity Framework Core, Azure DevOps and SQL Server.
- Support for dependency injection, middleware and JWT authentication.

REST Architecture

The backend (`SchulvereinVerwaltungs.API`) is designed as a REST API. REST maps operations to HTTP methods, such as `GET`, `POST`, `PUT` and `DELETE`. Domain entities, for instance, members or invoices, are exposed via controller endpoints (below) that exchange data as JSON objects. For more details on REST, refer to established documentations (Wikipedia, 2026).

Request Pipeline

An key feature of ASP.NET Core is its flexible middleware pipeline (see chapter 4.2.6 Middleware Chain Pattern for details), where requests and responses pass through a configurable chain of components. Each middleware element can intercept, modify or handle the incoming requests. This establishes a logical hierarchy and enables features like exception handling, logging and authentication (below).

Entry point and Configuration

The `SchulvereinVerwaltungs.API` project serves as the backend entry point. The `Program.cs` configures the middleware pipeline, registers services and initializes infrastructure components. Dependency injection (discussed in 4.2.10 Dependency injection) ensures that services receive their required dependencies automatically, without manual instantiation.

In addition:

- Entity Framework Core is configured using a connection string provided via environment variables, establishing the database context.
- Serilog is integrated for structured logging and monitoring, enabling reproducibility.
- JWT authentication is configured with the Bearer scheme, connecting to NOVA's identity provider for token validation (see Chapter 6.4.1).
- Swagger / OpenAPI support is enabled in the development environment, providing an interface for testing and documenting all available API routes.

6.2 Backend Structure

The NOVA backend follows a strict four-layer architecture implemented as separate .NET 8 class library projects within the **NOVA.slnx** solution. Each layer has a well-defined responsibility and a one-directional dependency graph: **NOVA.Server** depends on **SchulvereinVerwaltungssystem.Services**, which depends on **SchulvereinVerwaltungssystem.Infrastructure** and **SchulvereinVerwaltungssystem.Core**; **Core** has zero dependencies on any other project layer. This topology enforces the Dependency Inversion Principle at the project level,

making individual layers independently testable and replaceable. Dependency injection is the exclusive mechanism for wiring components across layers, with all registrations centralised in **NOVA.Server/Extensions/** and **NOVA.Server/Configuration/** (see chapter 6.2.1). The sections below examine each layer in detail, from the HTTP entry point down to the data access implementation.

6.2.1 Controllers

The **NOVA.Server/Controllers/** directory contains fifteen ControllerBase-derived classes that constitute the HTTP entry point for every API operation. All controllers are decorated with the [ApiController] attribute, which activates automatic model validation, binding source inference, and problem details responses without requiring explicit ModelState.IsValid checks in the majority of action methods. Route templates follow the api/{resource} convention enforced by [Route] attributes at the class level, with sub-routes defined on individual action methods.

Every controller receives its dependencies exclusively through constructor injection. The **ProductController** illustrates the canonical pattern:

```
// ProductController.cs - constructor injection and route declaration
[ApiController]
[Route("api/product")]
[Authorize]
public class ProductController : ControllerBase
{
    private readonly IProductService _productService;
    private readonly ILogger<ProductController> _logger;

    public ProductController(
        IProductService productService,
        ILogger<ProductController> logger)
    {
        _productService = productService;
        _logger = logger;
    }

    // GET api/product - returns active, non-deleted products
```

```

    [HttpGet]

    [ProducesResponseType(typeof(IEnumerable<ProductDto>), Status-
Codes.Status200OK)]

    public async Task<ActionResult<IEnumerable<ProductDto>>> Get-
AllProducts()

    {

        var products = await _productService.GetAllProductsAsync();

        return Ok(new { Products = products });

    }

}

```

The `[Authorize]` attribute at the class level enforces JWT bearer authentication on every endpoint; role-based restrictions are applied at the action level where finer-grained access control is required (see chapter 6.3 for the authentication model). HTTP verbs map to the CRUD operations expected by each resource: `[HttpGet]` for queries, `[HttpPost]` for creation, `[HttpPatch]` for partial updates, `[HttpPut]` for full replacement, and `[HttpDelete]` for removal. `[ProducesResponseType]` annotations declare the full set of possible response codes and body types, which are surfaced in the Swagger UI generated by **ConfigureSwagger.cs**.

Response construction is uniform: successful results are wrapped in an `Ok()`, `Created()`, or `CreatedAtAction()` result; domain exceptions propagate to the **ExceptionHandlerMiddleware** (see chapter 6.2.6), which translates them into standard HTTP status codes. Controllers themselves contain no business logic; their sole responsibilities are HTTP concern handling — parsing route parameters and request bodies, constructing response objects, and logging operation outcomes.

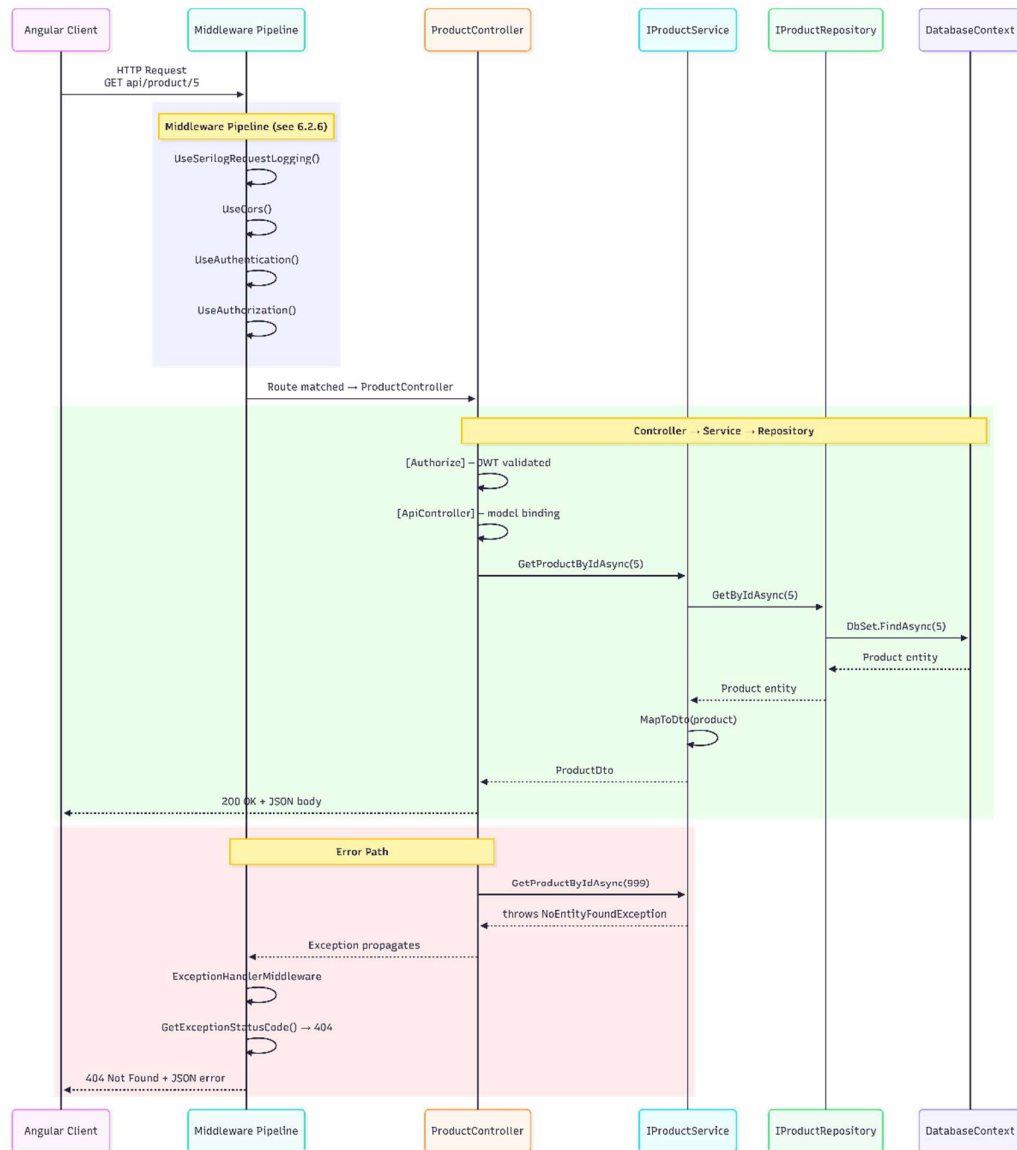


Figure 28: Controller Layer Interaction Diagram (Generated by Claude using NOVA Data)

6.2.2 Services

The service layer, located in **SchulvereinVerwaltungssystem.Services/**, implements all business logic and orchestrates operations that span multiple repositories. Each feature area is encapsulated in a matching interface/implementation pair following the naming convention **IXxxService / XxxService**. The complete set of services registered in **ConfigureNovaServices.cs** includes **PdfService**, **AuthService**, **EmailServices**, **MemberServices**, **InvoiceService**, **QrCodeService**, **FormServices**, **DebtorService**, **ProductService**, **ImportService**, **ExportServices**, **BulkInvoiceService**, **InvoiceBuilderService**, **GoogleCalendarService**, **CalendarEventService**, **DashboardService**, and **SettingsService**, among others.

Services are registered with `AddTransient` or `AddScoped` lifetimes depending on whether they carry state across a single request. Stateless services such as **ProductService** and **ImportService** use `AddTransient`. Services that wrap a scoped resource — for example, **InvoiceBuilderService** and **GoogleCalendarService**, which interact with the scoped **DatabaseContext** — are registered as `AddScoped`:

```
// ConfigureNovaServices.cs – representative DI registrations
services.AddTransient<IProductService, ProductService>();
services.AddScoped<IInvoiceBuilderService, InvoiceBuilderService>();
services.AddScoped<IGoogleCalendarService, GoogleCalendarService>();
services.AddTransient<IDashboardService, DashboardService>();
```

The service interface contract keeps controllers decoupled from any concrete implementation. **ProductService** exemplifies the validation and domain logic that belongs in this layer:

```
// ProductService.cs – business validation before repository delegation
public async Task<ProductDto> CreateProductAsync(CreateProductDto createDto)
{
    if (string.IsNullOrEmpty(createDto.ProductName))
        throw new EmptyStringException("Product name is required.");

    // Enforce uniqueness via repository query before persisting
    var isUnique = await _productRepository.IsProductNameUniqueAsync(createDto.ProductName);
    if (!isUnique)
        throw new DuplicateNameException("Product", createDto.ProductName);

    var product = new Product {
        ProductName = createDto.ProductName,
        Category     = createDto.Category,
        DefaultPrice = createDto.DefaultPrice,
        TaxRate      = createDto.TaxRate,
        IsActive     = createDto.IsActive
    };
}
```

```

        var created = await _productRepository.CreateAsync(product);
        return MapToDto(created);
    }

```

Domain exceptions such as `EmptyStringException` and `DuplicateNameException` are defined in **SchulvereinVerwaltungssystem.Core/Exceptions/** and are caught centrally by **ExceptionHandlerMiddleware** (see chapter 6.2.6), removing the need for try/catch blocks in service methods. The mapping from entities to DTOs is performed within the service, so no EF Core entity objects are ever returned to the controller layer.

6.2.3 Core

SchulvereinVerwaltungssystem.Core is the innermost project in the dependency graph. It defines the contracts and data shapes that all other layers use but never depends on any of them. Its content is organised into four directories: **DTOs/**, **DataTransferObjects/**, **Interfaces/**, and **Enums/**, plus **Exceptions/** and **Settings/**.

DTOs and **DataTransferObjects** together hold all request and response transfer objects. The split reflects historical growth: **DTOs/** contains invoice-specific and dashboard aggregates (e.g., **InvoiceDto**, **DashboardDto**, **InvoiceTemplateDto**) while **DataTransferObjects/** holds the majority of feature-specific objects (e.g., **ProductDto**, **CreateProductDto**, **ChangeProductDto**, **ImportResultDto**, **MemberGroupDto**). Both are plain C# classes with no logic, fully serialisable, and free of EF Core references:

```

// ProductDto.cs – representative DTO structure in Core
public class ProductDto
{
    public int    Id           { get; set; }
    public string ProductName { get; set; }
    public string? Description { get; set; }
    public ProductCategory Category { get; set; }
    public decimal DefaultPrice { get; set; }
    public decimal TaxRate      { get; set; }
    public bool   IsActive      { get; set; }
    public bool   IsDeleted      { get; set; }
    public DateTime? DeletedAt   { get; set; }
}

```

Interfaces in **Core** define the repository contracts (**IGenericRepository<T>**, **IMemberRepository**, **IInvoiceRepository**, etc.) and the service contracts (**IProductService**, **IDashboardService**,

etc.). Both services and infrastructure implementations depend on these interfaces, achieving inversion of control without Core depending on either. **Enums** centralises all enumeration types used across the system, including **ProductCategory** (Membership, Sponsorship, Event, Donation, Other), **MemberStatus**, **EmailType**, and **PhoneType**, preventing duplicated definitions across projects.

The zero-dependency constraint on Core is enforced at the project file level: **SchulvereinVerwaltungssystem.Core.csproj** declares no `<ProjectReference>` elements. This guarantees that Core entities and contracts can be unit-tested in total isolation and reused without introducing transitive dependencies.

6.2.4 Infrastructure

SchulvereinVerwaltungssystem.Infrastructure implements the data access layer using the Repository Pattern on top of EF Core 9.x. The layer is subdivided into **Data/DatabaseContext.cs**, **Data/Entities/** (per-domain entity classes), **Data/Repositories/** (generic and specialised repository implementations), and **Migrations/** (EF Core migration history).

Generic Repository

GenericRepository<T> in **Data/Repositories/GenericRepository.cs** provides a reusable, type-parameterised base implementation of `IGenericRepository<T>`. It wraps a **DatabaseContext** and a `DbSet<T>` instance and exposes the full set of common data operations — `GetByIdAsync`, `GetAllAsync`, `FindAsync`, `AddAsync`, `UpdateAsync`, `DeleteAsync`, `SoftDeleteAsync`, `GetPagedAsync`, `GetAllWithIncludeAsync`, and `ExistsAsync` — as virtual async methods:

```
// GenericRepository.cs – soft delete implementation
public virtual async Task SoftDeleteAsync(T entity)
{
    if (entity is ISoftDeletable softDeletable)
    {
        // Set flags rather than removing the row
        softDeletable.IsDeleted = true;
        softDeletable.DeletedAt = DateTime.UtcNow;
        await UpdateAsync(entity);
    }
    else
    {
        // Fall back to hard delete for entities that do not support
        soft delete
        await DeleteAsync(entity);
    }
}
```

```
}
}
}
```

The Soft Delete pattern (see chapter 5.2) is handled uniformly in **GenericRepository** through the **ISoftDeletable** interface check, so specialised repositories and services need not re-implement the deletion logic.

Specialised Repositories

Domain-specific repositories override or extend **GenericRepository<T>** with query methods that cannot be expressed generically. **MemberRepository**, for example, adds filtered queries by membership status and eager-loading of related contacts, phone numbers, and group memberships. **InvoiceRepository** provides paginated search with combined status, date-range, and debtor filters. **CalendarEventRepository** exposes **GetEventsByDateRangeAsync**, **GetUnsyncedEventsAsync**, and **GetByGoogleEventIdAsync** for the calendar synchronisation feature (see chapter 6.4.9).

All specialised repositories are registered in **ConfigureInfrastructure.cs** as scoped services:

```
// ConfigureInfrastructure.cs – specialised repository registrations
(excerpt)

services.AddScoped<IMemberRepository, MemberRepository>();
services.AddScoped<IInvoiceRepository, InvoiceRepository>();
services.AddScoped<ICalendarEventRepository, CalendarEventRepository>();
services.AddScoped<IInvoiceTemplateRepository, InvoiceTemplateRepository>();

Generic repositories are registered for entities that require only
the standard CRUD surface:

// ConfigureInfrastructure.cs – generic repository registrations (excerpt)

services.AddScoped(typeof(IGenericRepository<>), typeof(GenericRepository<>));
services.AddScoped<IGenericRepository<InvoiceItem>, GenericRepository<InvoiceItem>>();
services.AddScoped<IGenericRepository<Product>, GenericRepository<Product>>();
```

EF Core migrations reside in **Migrations/** and are applied automatically on startup via **Database.MigrateAsync()** called from **InitializeDatabase.cs** (see chapter 5.6.1).

6.2.5 Tests

The **SchulvereinVerwaltungssystem.Tests** project uses xUnit as the test runner, Moq for dependency mocking in service tests, FluentAssertions for expressive assertion syntax, and the

EF Core InMemory provider for repository tests that require a functioning **DatabaseContext** without a live SQL Server instance.

Test files are organised under two top-level directories that mirror the application structure:

- **RepositoryTests/** — seven test classes covering **MemberRepository**, **InvoiceRepository**, **GroupRepository**, **FormRepository**, **DebtorRepository**, **AuditLogRepository**, and **SettingRepository**.
- **ServiceTests/** — sub-directories for **AuthServiceTests**, **EmailService**, **GroupServiceTests**, **InvoiceServiceTests**, **MemberServiceTests**, **SettingServiceTests**, **TokenServiceTests**, and **CalendarServiceTests**.

Repository tests instantiate a real **DatabaseContext** against a unique in-memory database per test class, guaranteeing isolation between test runs:

```
// MemberRepositoryTests.cs - InMemory context setup and representative test
public class MemberRepositoryTests : IDisposable
{
    private readonly DatabaseContext _context;
    private readonly MemberRepository _repository;

    public MemberRepositoryTests()
    {
        var options = new DbContextOptionsBuilder<DatabaseContext>()
            .UseInMemoryDatabase(databaseName:
                Guid.NewGuid().ToString())
            .Options;
        _context = new DatabaseContext(options);
        _repository = new MemberRepository(_context);
    }

    [Fact]
    public async Task AddAsyncMemberCorrectly()
    {
        var member = new Member
        {
```

```

        MemberNumber = "M-2024-TEST",
        FirstName     = "Test",
        LastName      = "User",
        JoinDate       = DateTime.UtcNow,
        Status         = MemberStatus.Active
    };

    await _repository.AddAsync(member);
    await _repository.SaveChangesAsync();

    // ID is assigned by EF Core after persist
    Assert.True(member.MemberId > 0);
    var saved = await _repository.GetByIdAsync(member.MemberId);
    Assert.NotNull(saved);
}

public void Dispose() => _context.Dispose();
}

```

Service tests use `Mock<IXxxRepository>` instances to isolate business logic from data access. Each mock is configured with `Setup / ReturnsAsync` chains that define the repository behaviour for the specific scenario under test, and `Verify` calls assert that the service issued the expected repository calls in the expected order. The **IDisposable** pattern on test classes and the unique in-memory database name per `DbContextOptionsBuilder` ensure no shared state bleeds between tests.

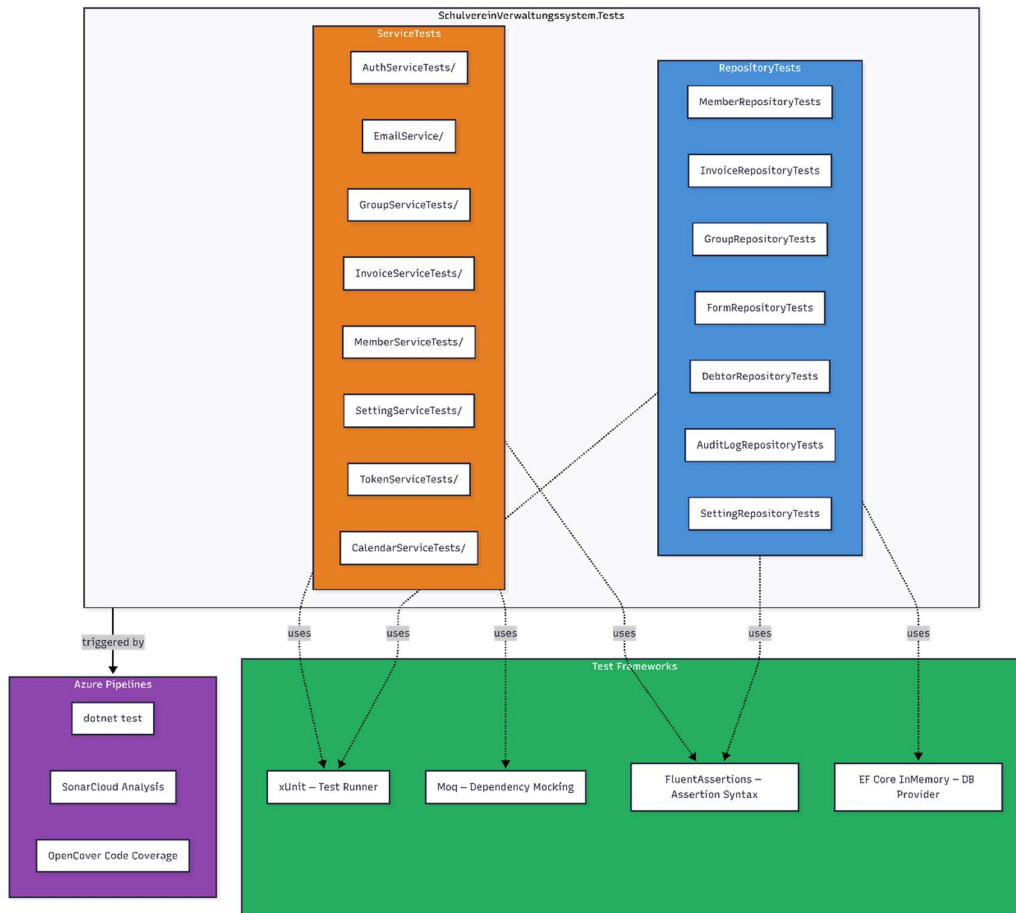


Figure 29: Test Project Structure (Generated by Claude using NOVA Data)

6.2.6 Logging and Monitoring

Structured logging in NOVA is provided by Serilog, configured in **NOVA.Server/Configuration/ConfigureLogging.cs** and activated via `builder.Host.UseSerilog(...)` in **Program.cs**. The configuration reads additional settings from the active `IConfiguration` source (e.g., minimum level overrides from **appsettings.json**), enriches every log event with contextual properties via `Enrich.FromLogContext()`, and writes to two concurrent sinks:

```
// ConfigureLogging.cs - Serilog dual-sink configuration
builder.Host.UseSerilog((context, services, configuration) => configuration
    .ReadFrom.Configuration(context.Configuration) // honours appsettings overrides
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.File(
```

```

        path: "logs/nova-log-.txt",
        rollingInterval: RollingInterval.Day,           // new file per
day
        retainedFileCountLimit: 30,                     // keep 30 days
of history
        fileSizeLimitBytes: 10_485_760,                // 10 MB cap per
file
        rollOnFileSizeLimit: true,
        shared: true
    });

```

HTTP request logging is added to the middleware pipeline via `app.UseSerilogRequestLogging()` in **ConfigureLogging.UseLogging()**. The custom message template records the HTTP method, path, response status code, and elapsed time in milliseconds for every request; the `EnrichDiagnosticContext` delegate additionally attaches the request host and user-agent string to each log event.

Exception handling is separated from logging through the **ExceptionHandlerMiddleware** defined in **NOVA.Server/CustomMiddleware/ExceptionHandlerMiddleware.cs**. Registered in **Program.cs** via `app.UseMiddleware<ExceptionHandlerMiddleware>()`, it wraps the remainder of the pipeline in a try/catch and, on exception, delegates to two static methods:

```

// ExceptionHandlerMiddleware.cs - exception interception and HTTP
response generation

public async Task InvokeAsync(HttpContext context)
{
    try { await _next(context); }
    catch (Exception ex)
    {
        LogException(ex, _logger);           // structured warning or
error log

        await HandleExceptionAsync(context, ex); // JSON error re-
sponse
    }
}

```

`GetExceptionStatusCode` maps each custom exception type to the appropriate HTTP status code using a switch expression. Domain validation exceptions (`NegativeIdException`, `EmptyStringException`, `InvalidEmailFormatException`) map to **400 Bad Request**; `NoEntityFoundException` and `UserNotFoundException` map to **404 Not Found**; concurrency and uniqueness

exceptions (`DuplicateNameException`, `EntityRelationException`, `SignInException`) map to **409 Conflict**; and `NoAccessException` maps to **403 Forbidden**. All other exceptions fall through to **500 Internal Server Error**. `LogException` assigns log levels per exception category: known domain exceptions are logged at Warning with structured message templates, while unexpected exceptions are logged at Error, allowing operations teams to filter noise from genuine failures in log aggregation tools.

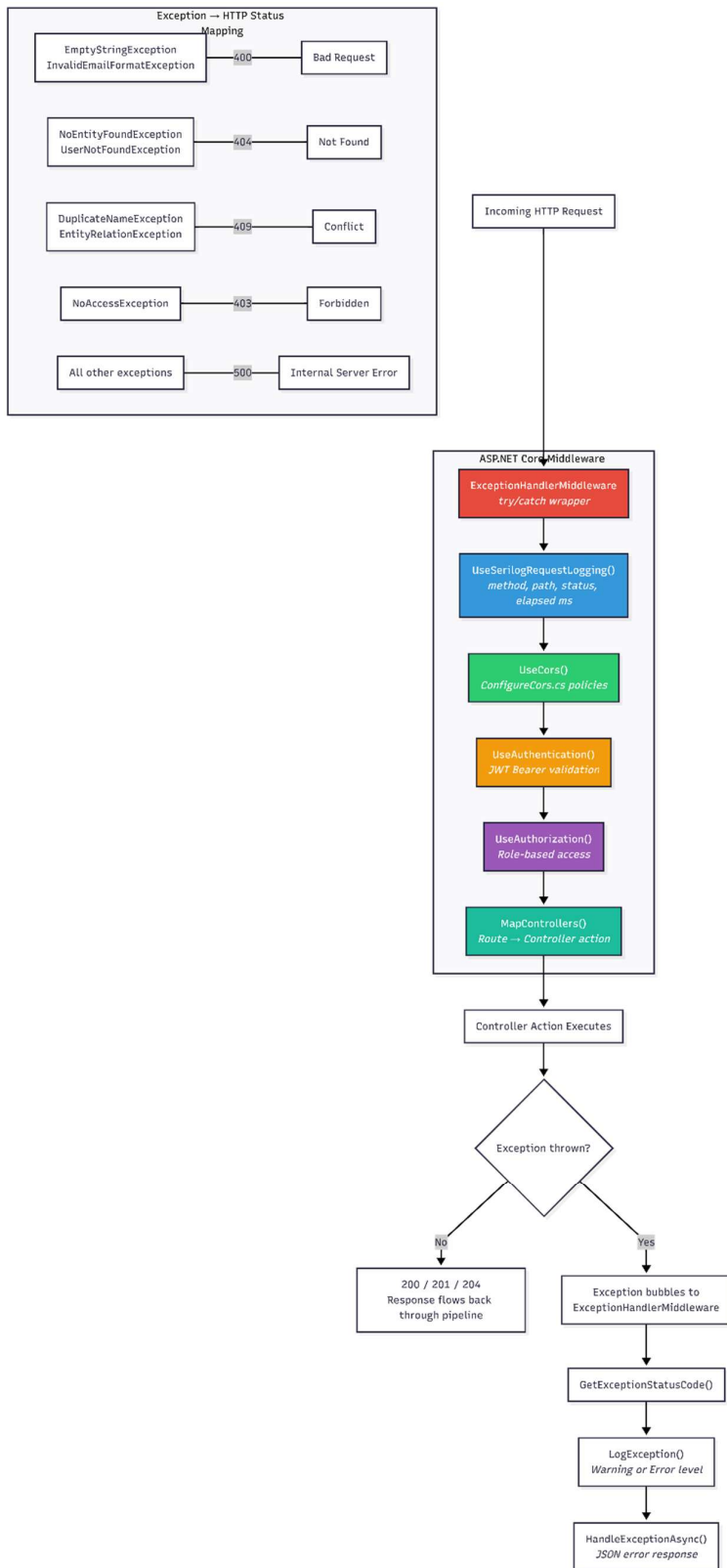


Figure 30: Middleware Pipeline Order (Generated by Claude using NOVA Data)

6.3 Backend Workflow

ASP.NET Core's middleware pipeline orchestrates HTTP requests through a sequence of components. This sequence ensures protected routes by restricting access to business logic. Furthermore, critical errors are intercepted to avoid server failure and ensure full availability.

Pipeline Overview

Incoming requests first encounter CORS middleware, followed by JWT authentication validation using the bearer scheme. Invalid tokens are rejected, while valid ones proceed to exception handling and routing.

The pipeline then matches endpoints to controllers, like `MemberController` or `InvoiceController`, injecting request entities for processing.

Core Processing Steps

- Controllers invoke domain services, such as `MemberService`, which coordinate with repositories and the `DatabaseContext` for data operations.
- Background services such as `EventBackgroundService` handle asynchronous tasks independently.
- Responses flow reversely through the pipeline with middleware adding headers or logging before JSON serialization for the client.

Figure 31: Backend Workflow - Part 1 and Figure 32: Backend Workflow - Part 2 illustrate this flow visually.

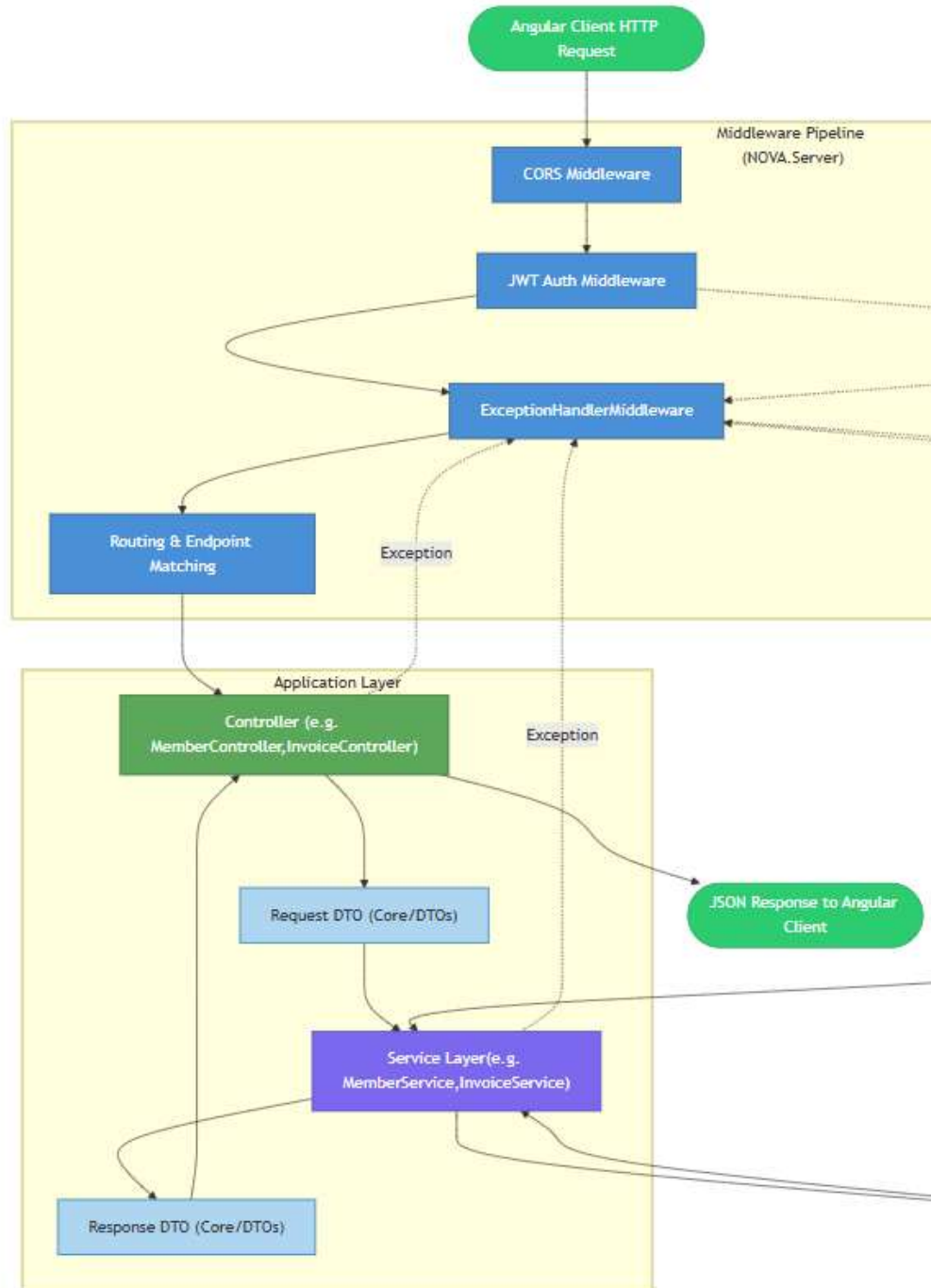


Figure 31: Backend Workflow - Part 1

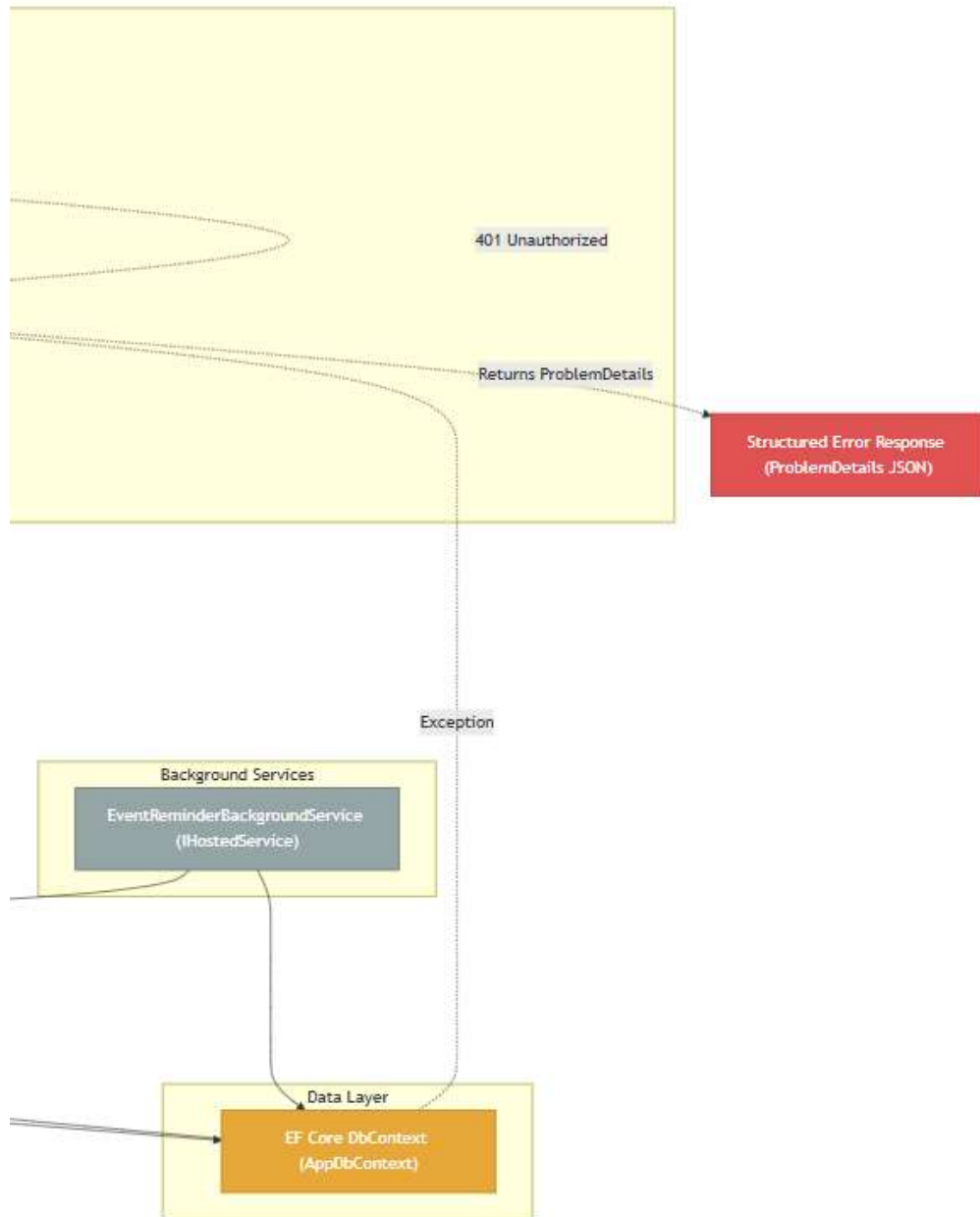


Figure 32: Backend Workflow - Part 2

6.4 Main Features

The main features of NOVA form the operational backbone. These “modules” directly address the key inefficiencies observed with the previous software: high manual effort, rigid accounting, and rough communication workflows.

Each feature is implemented as a dedicated service layer and exposed via REST-style endpoints, ensuring straightforward integration with the Angular frontend. The following sections describe the central use cases of the system, including user authentication, member and product administration, dynamic form and invoice generation, and automated email and calendar-based event management.

Together, these components enable administrators to register members, manage association fees, coordinate school-related events, and communicate with members through bulk emails and newsletters—all within a single, tailored web application.

6.4.1 Authentication and Authorization

Web applications must verify who users are and what they are allowed to do. First, authentication establishes the identity of the user. Second, authorization checks whether this identity has the required permissions, for example administrator vs. manager.

Access to the API is protected with HTTPS, ensuring that data is transmitted in encrypted form and cannot be read by others. To further limit data exposure, NOVA uses Data Transfer Objects for communication between the API and the Angular frontend. DTOs control which properties of database entities are sent, thereby reducing the risk of accidental leaks of sensitive fields.

Authentication with JWT

NOVA implements strict API protection by using JSON Web Tokens (JWT) passed via the HTTP *Authorization* header with the *Bearer* scheme. After a successful login, the backend response contains a JWT, which is then used in each subsequent request in HTTPS: *Authorization: Bearer <token>*.

A JWT consists of a header, a payload and a signature. The payload contains a set of claims, such as user identifier and username, while the signature is computed using the private secret key that is only known to the server. This signature allows the server to distinguish foul or modified tokens from valid tokens without storing any session state in the database.

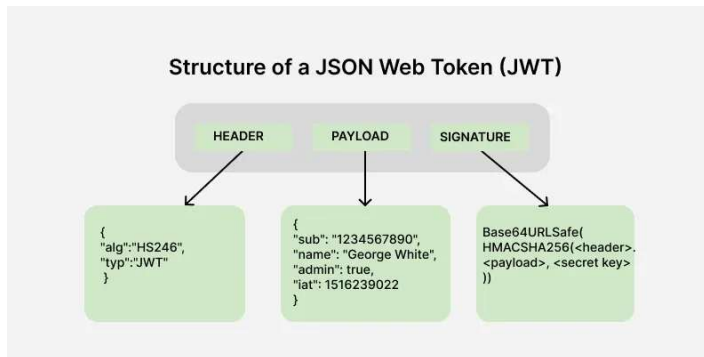


Figure 33: JWT Structure (Geeks For Geeks, 2026)

This token mechanic is used to validate incoming requests. Authentication middleware is added to the ASP.NET Core middleware pipeline, as each request must pass through this component, enforcing API protection. This middleware extracts the bearer token, validates the signature using the secret key or checks whether the token has expired.

If validation succeeds, the middleware passes the request. If validation fails, a *401 Unauthorized* response is returned without executing the target endpoint.

To limit the impact of token theft, tokens are generated with a limited lifetime. This lifetime is configurable by administrators through NOVA's Settings System (see chapter 6.4.10 Settings System). Best practices recommend short lifetimes of 15-60 minutes to minimize exposure, paired with refresh tokens for user experience.

The overall backend authentication flow, from HTTP request to response, is shown in Figure 59. The figure illustrates how valid tokens are handled and how invalid or unauthorized tokens are rejected:

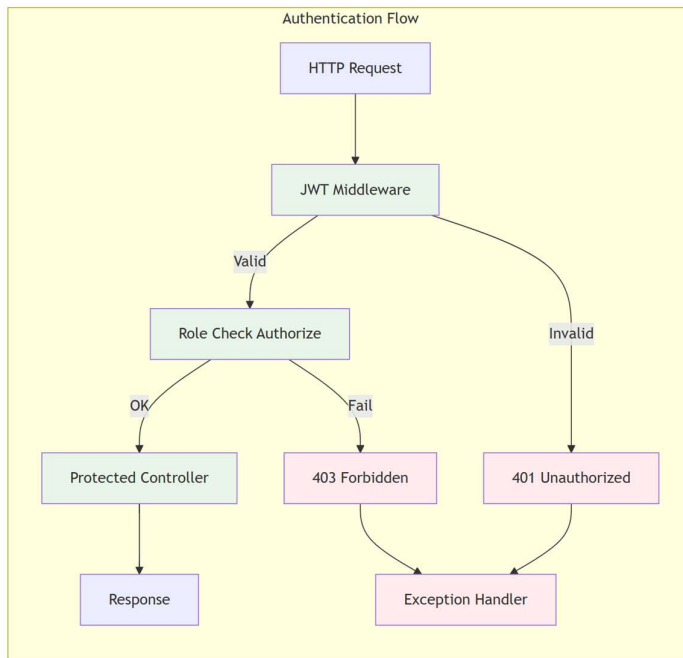


Figure 34: Authentication Flow – Backend

Implementation with ASP.NET Core Identity

To store user credentials, NOVA uses a dedicated *NovaUser* database entity, which derives from the ASP.NET Core Identity *IdentityUser* (Microsoft, 2026). ASP.NET Core Identity provides functionality for password hashing, password policies, lockout rules and role management. This well-tested framework reduces the risk of implementing security critical logic incorrectly significantly.

```

public class NovaUser : IdentityUser {
    ...
}
  
```

In Program.cs, JWT bearer authentication is configured in the builder service pipeline and sets the token validation parameters:

```

services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
  
```

```

options.SaveToken = false; // Token not stored server-side

options.TokenValidationParameters = new TokenValidationParameters() {

    ValidateIssuerSigningKey = true,

    IssuerSigningKey =

        new SymmetricSecurityKey(Encoding.UTF8.GetBytes(authSettings.TokenPrivateKey)),

    ValidateIssuer = false,

    ValidateAudience = false,

};

});

```

The app configuration stores and injects *TokenPrivateKey* via *AuthSettings*, loaded during startup-settings-configuration (see Settings System), while being able to be rotated. By disabling issuer and audience validation, NOVA treats the token as an internal credential, simplifying configuration while keeping signature verification enabled.

Authorization

In addition to authentication, NOVA uses ASP.NET Core's authorization system to enforce RBAC. A fallback policy requires that all API endpoints are accessed by an authenticated user unless explicitly marked. Publicly available endpoints require an *[AllowAnonymous]* attribute.

```

services.AddAuthorization(options => {

    options.FallbackPolicy = new AuthorizationPolicyBuilder()

        .AddAuthenticationSchemes(JwtBearerDefaults.AuthenticationScheme)

        .RequireAuthenticatedUser()

        .Build();

    // Define authority policies here

    options.AddPolicy("ManageEmailTemplates", policy => policy.RequireRole("Admin", "Superadmin", "Moderator", "Manager"));

});

```

Role-specific features are marked with an authorized attribute, for example with *[Authorize (Policy = "ManageEmailTemplates")]*. This approach makes the required authorization level explicit at the endpoint and allows policies to be reused across multiple controllers.

These roles and claims are stored in the database and attached to the user's JWT as claims during token creation. The authorization middleware evaluates incoming request's claims against the configured policies and either grants access or returns a 403 Forbidden response when the user is authenticated but lacks the required role.

User Operations

User authentication endpoints are grouped in a dedicated *AuthController* using *AuthService* logic. This controller implements the main authentication scenarios:

- Sign In – Validate user credentials; upon success, provide JWT
- Sign Up – Create a new *NovaUser* account; provide JWT

Grouping these endpoints within a single controller keeps the authentication logic cohesive. The Angular client communicates with this controller to sign in and sign up. Authentication logic is handled in *AuthService.cs* through ASP.NET Core Identity's *UserManager*, which abstracts user operations such as credential validation and token generation.

Account Recovery

NOVA supports secure password resets with help of the Email System (see 6.4.7 Email System with Templates). The password-reset-flow consists of two main steps: *Forgot Password* and *Reset Password*.

1. The user submits their email address. ASP.NET Core Identity's *UserManager* verifies that the email belongs to a user and then generates a password reset token, which is sent via email.
2. After receiving the email, the user follows the reset link, where they enter the token and the new password. The token is validated and if successful, the password is updated.

This recovery process is illustrated in more detail below (Figure 35).

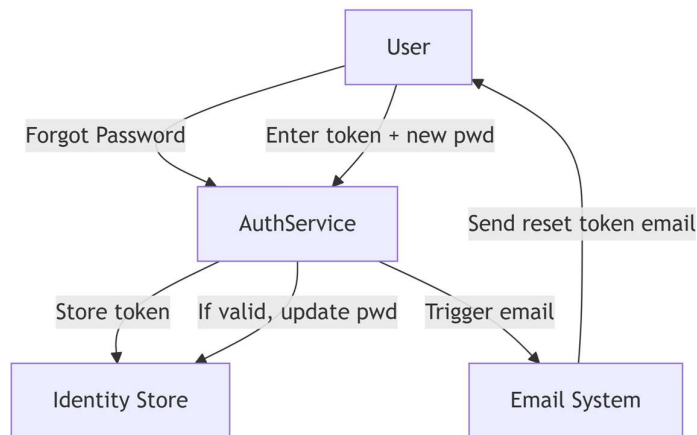


Figure 35:Password Reset Flow

6.4.2 Member Management

The Member Management System represents and models actual members of the school association, whose lifecycle includes being recruited, managed over several years and being eventually removed. It encapsulates membership rules and serves as information base for numerous modules, such as the Invoice System, Email Campaigns and Group Management.

Member Model

The foundation of this system is in the `Member` entity, representing either an individual or a business member of the school union. It stores the identity of members, address information, membership status and critical financial references to debtors, products and contacts.

```

public class Member :ISoftDeletable {
    [Key]
    public int MemberId { get; set; }
    [StringLength(450)]
    public string? AspNetUserId { get; set; }
    [Required]
    [StringLength(50)]
    public string MemberNumber { get; set; }
}

```

```

    [Required]
    [StringLength(100)]
    public string FirstName { get; set; }

    [Required]
    [StringLength(100)]
    public string LastName { get; set; }

    [Required]
    public MemberStatus Status { get; set; }

    public DateTime JoinDate { get; set; }

    public DateTime? ExitDate { get; set; }

    public bool IsDeleted { get; set; }

    public DateTime? DeletedAt { get; set; }

    ...
}

```

Each member optionally links to an ASP.NET Core Identity user via *AspNetUserId*. Lifecycle timestamps, such as *JoinDate*, *Exitdate*, *CreateAt* and *DeleteAt*, provide a history for memberships.

The *ISoftDeletable* interface and the *IsDeleted* and *DeletedAt* fields implement the Soft Delete Pattern (see chapter 4.2.7), which enables members to be deleted without losing the database record.

Member Lifecycle

School association engagement is tracked via the *MemberStatus* enum: *Active*, *Inactive*, or *Suspended*.

```

public enum MemberStatus {
    Active,
    Inactive,
    Suspended // Locked because of e.g. didnt pay
}

```

In practice, NOVA combines status with lifecycle timestamps and soft-delete flags to produce three member states shown in Figure 36:

- Active – New members start with this state.
- Exited – Members who have left the association temporarily have this state.
- Deleted – Members that have been removed have this state.

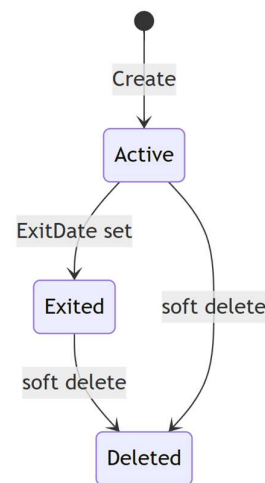


Figure 36: Member Object States

This system represents actual membership and prevents accidental member loss.

Data Layer

Member persistence and querying are handled in a dedicated *MemberRepository*, extending *GenericRepository<T>* and implementing *IMemberRepository*. This repository primarily uses the *DatabaseContext* with Entity Framework Core for all data operations.

```

public class MemberRepository(DatabaseContext context): GenericRe-
    pository<Member>(context), IMemberRepository {
    ...
}
  
```

Repositories abstract query logic in testable methods and can easily be replaced with mocks during unit testing. For a detailed description of this pattern, see chapters 4.2.1 Repository Pattern and 4.2.2 Generic Repository Pattern.

To protect database entities and control data exposure, NOVA follows the DTO Pattern (Data Transfer Object) (see chapter 4.2.5). Member-related API responses use DTOs such as *MemberDto*, *UpdateMemberDto* and *CreateMemberDto*, which flatten entity relationships and control what properties get exposed.

```

public class MemberDto : IExportable
{
    [ExportColumn("Mitglieds-ID", 0)]
    public int MemberId { get; set; }
    public string? AspNetUserId { get; set; }
    [ExportColumn("Mitgliedsnummer", 1)]
    public string MemberNumber { get; set; } = string.Empty;
    [ExportColumn("Vorname", 2)]
    public string FirstName { get; set; } = string.Empty;
    [ExportColumn("Nachname", 3)]
    public string LastName { get; set; } = string.Empty;
    ...
}

```

MemberDto implements *IExportable* so that the Export System (below) can generate Excel sheets directly from DTOs. Properties marked with the *[ExportColumn]* attribute define the column title and order in Excel sheets.

Service Layer

All business logic is encapsulated by the *MemberService*, which implements *IMemberService*. The service maps between DTOs and entities, performs validations and orchestrates repository operations. Key operations include CRUD, batch, and validation operations.

- CRUD operations are responsible for creating, reading, updating, and deleting member entities.
- Batch operations update statuses for several members at once.
- Validation operations sanitize user input and prevent invalid entity states.

Member Operations

The *MemberController* exposes the Member Management System via a RESTful API. Endpoints are grouped into logical regions:

- CRUD endpoints for individual members
- Search and filtering endpoints
- Group and status operations
- Batch update endpoints
- Statistics and export endpoints

Figure 37 illustrates the HTTP pipeline for member operations. Controllers receive requests, delegate logic to services, who receive data from repositories.

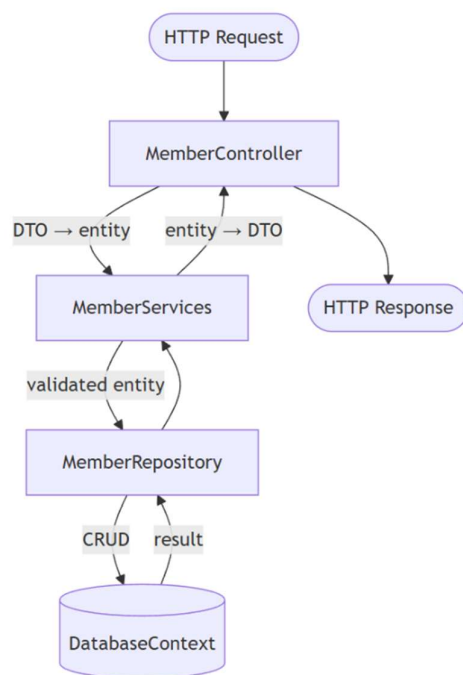


Figure 37: Member System Flow

Controllers focus on authentication, model binding and HTTP responses, while services implement the business rules and call repositories for data access. This layered architecture improves modularity, testability and performance.

Filtering

Advanced member filtering is implemented using *MemberFilterDto*. The filter bundles criteria such as name fragments, status, date ranges, products and groups, which are translated into composable LINQ predicates in the data access layer. DESIGNPATTERN HERE

Export

The

Export System (see chapter 6.5.1) orchestrates Excel exports. It operates on *IExportable* objects like *MemberDto*. First, the service fetches relevant members and converts them into a *WorksheetDataSource*, which represents a single Excel sheet. This sheet contains member credentials, status, debtor information and lifecycle timestamps. Column names and their order are defined by *[ExportColumn]* attributes on the DTO. Additional attributes on DTO properties enable simple modifications without changing export logic.

```
public async Task<byte[]> ExportMembersToExcelAsync() {  
    var members = await GetAllMembersAsync();  
    if (!members.Any()) return [];  
  
    return exportService.ExportToExcel(  
        new WorksheetDataSource<IExportable>(mem-  
bers, "Mitglieder", Alignment.Horizontal)  
    );  
}
```

The export system allows administrators to perform offline analyses, share data or archive annual membership states in a compact format.

6.4.3 Product Management

Products in NOVA represent the catalogue of purchasable or billable items that can be attached to invoices as line items. The **Product** entity, defined in **SchulvereinVerwaltungs-system.Infrastructure/Data/Entities/Finance/Product.cs**, implements **ISoftDeletable** and carries five business-relevant fields:

```
// Product.cs – entity definition  
public class Product : ISoftDeletable  
{  
    [Key]  
    public int ProductId { get; set; }  
  
    [Required, StringLength(200)]  
    public string ProductName { get; set; }
```

```

[StringLength(1000)]
public string? Description { get; set; }

[Required]
public ProductCategory Category { get; set; }    // enum: Member-
ship, Sponsorship, Event, Donation, Other

[Column(TypeName = "decimal(18,2)")]
public decimal DefaultPrice { get; set; }

[Column(TypeName = "decimal(5,2)")]
public decimal TaxRate { get; set; }

public bool IsActive { get; set; }
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }

public virtual ICollection<InvoiceItem> InvoiceItems { get; set;
}
}

```

The ProductCategory enum, defined in **SchulvereinVerwaltungssystem.Core/Enums/ProductCategory.cs**, classifies products into five categories (Membership = 0, Sponsorship = 1, Event = 2, Donation = 3, Other = 99), stored as integer values in the database via EF Core enum conversion. The DefaultPrice and TaxRate fields serve as pre-filled defaults that the **InvoiceBuilderService** copies into **InvoiceTemplateItem** records when a product is referenced from a template (see chapter 6.4.8), eliminating repetitive data entry.

The IProductService / ProductService pair in **SchulvereinVerwaltungssystem.Services/ProductService/** enforces business rules before delegating to **IProductRepository**. Name uniqueness is validated before every create and update operation:

```

// ProductService.cs – uniqueness enforcement on creation
public async Task<ProductDto> CreateProductAsync(CreateProductDto
createDto)
{
    if (string.IsNullOrEmpty(createDto.ProductName))

```



```

        throw new EmptyStringException("Product name is required.");

        var isUnique = await _productRepository.IsProductNameUniqueAsync(createDto.ProductName);

        if (!isUnique)
            throw new DuplicateNameException("Product", createDto.ProductName);

        var product = new Product {
            ProductName = createDto.ProductName,
            Description = createDto.Description,
            Category = createDto.Category,
            DefaultPrice = createDto.DefaultPrice,
            TaxRate = createDto.TaxRate,
            IsActive = createDto.IsActive
        };

        return MapToDto(await _productRepository.CreateAsync(product));
    }

```

The **ProductController** in **NOVA.Server/Controllers/ProductController.cs** exposes nine endpoints under `api/product` covering the full lifecycle: retrieval by ID, listing of active/all/including-deleted sets, filtering by category, free-text search, uniqueness check, create, partial update (PATCH), soft delete, restore, and permanent delete. The distinction between soft and permanent deletion is explicit: `DELETE api/product/{id}` sets `IsDeleted = true` and records `DeletedAt`; `DELETE api/product/{id}/permanent` removes the row entirely. Restoration is possible for soft-deleted products via `POST api/product/{id}/restore`, allowing administrators to recover accidentally archived catalogue entries. The `[Authorize]` attribute on the class requires authentication for all operations.

Transfer objects are defined in **SchulvereinVerwaltungssystem.Core/DataTransferObjects/**: **ProductDto** exposes all fields including soft-delete metadata; **CreateProductDto** captures the creation payload; **ChangeProductDto** carries the partial-update payload for PATCH semantics.

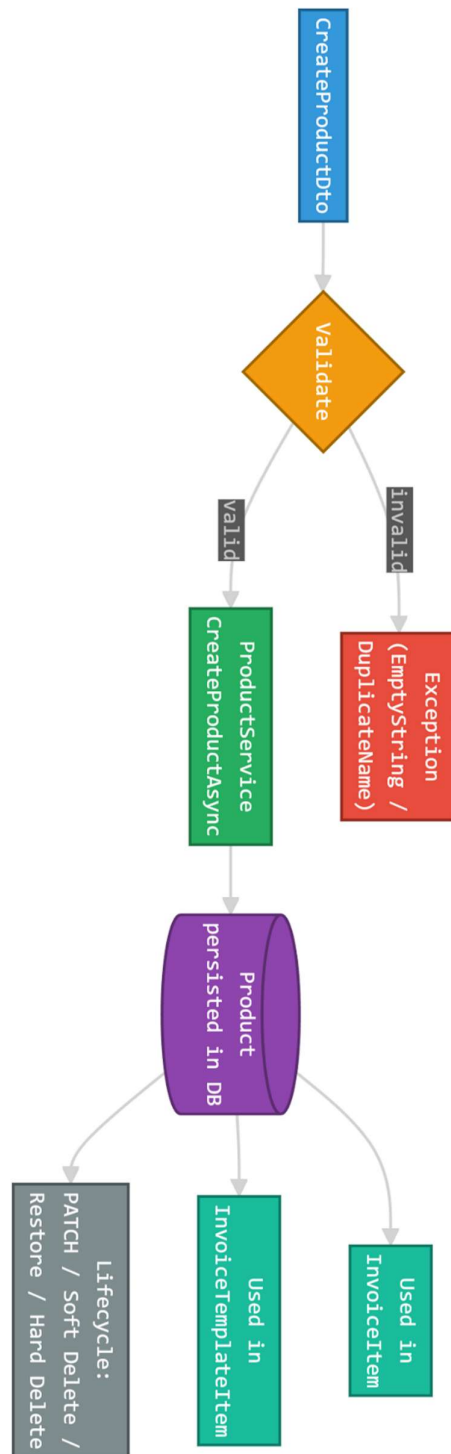


Figure 38: Product Management Flow (Generated by Claude using NOVA Data)

6.4.4 Formular System

A central feature of NOVA is the Formular System. The school association extensively uses forms to inform and recruit members for events, as well as invite new members. Formulars serve as communication channel between the association, its members and non-members who submit responses without being authenticated. To ensure privacy, forms are hashed into tokenized URLs that allow access without exposing internal data or requiring authentication. The Formular System is broken down into six functional areas: Formulars Management, Access Management, Formular Posting, Formular Submissions, Formular Builder and Formular Export.

Formular Management

The core entity is *Form*, which is the top-level object carrying all metadata. Fields like *FormSections*, *FormFields* and *FormFieldOptions* can contain structural definitions, enabling hierarchically grouped forms.

```
public class Form :ISoftDeletable {  
    [Key]  
    public int FormId { get; set; }  
    [Required]  
    [StringLength(200)]  
    public string Name { get; set; }  
    public string? Description { get; set; }  
    public DateTime CreatedAt { get; set; }  
    public string CreatedBy { get; set; }  
    public string? ImageUrl { get; set; }  
    public bool IsPublished { get; set; }  
    public int SubmissionCount { get; set; }  
    public int? MaxSubmissionCount { get; set; }  
    public ICollection<FormSection> Sections { get; set; }  
    public ICollection<FormLayout> Layouts { get; set; }  
    public bool IsDeleted { get; set; }  
    public DateTime? DeletedAt { get; set; }  
}
```

```
}
```

MaxSubmissionCount caps the number of submission responses, preventing over-submission on events. *IsPublished* controls external visibility and *SubmissionCount* tracks the number of submissions. Like the *Member* entity, *Form* implements *ISoftDeletable* (see chapter 4.2.7 Soft Delete Pattern), so forms are retained in the database after being deleted.

Form Hierarchy

Figure 39 shows the hierarchical composition of *Form*. It contains *FormSection* objects, each section containing *FormField* objects and fields optionally carrying selectable *FormFieldOption* entries. This enables a structured form with diverse input types.

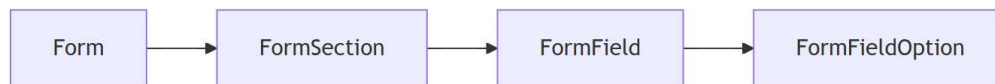


Figure 39: Form Entity Hierarchy

Each hierarchy level contains independent fields and an *Order* index, which controls the order of form elements in the frontend:

```
public class FormSection {
    public int FormSectionId { get; set; }
    public int FormId { get; set; }
    public string Title { get; set; }
    public int Order { get; set; }
    public ICollection<FormField> Fields { get; set; }
}

public class FormField {
    public int FormFieldId { get; set; }
    public int FormSectionId { get; set; }
    public int FieldType { get; set; } // text, select, checkbox, etc.
    public string FieldLabel { get; set; }
    public string? Placeholder { get; set; }
```

```

    public bool IsRequired { get; set; }

    public int Order { get; set; }

    public ICollection<FormFieldOption> Options { get; set; }
}

public class FormFieldOption {
    public int FormFieldOptionId { get; set; }

    public int FormFieldId { get; set; }

    public string Label { get; set; }

    public int Order { get; set; }
}

```

Data Layer

Formular persistence is handled in *FormRepository*, extending *GenericRepository<T>* and implementing *IFormRepository*. All data operations use the *DatabaseContext*.

```

public class FormRepository(DatabaseContext context)
    : GenericRepository<Form>(context), IFormRepository {
    ...
}

```

Like the *MemberRepository*, this abstraction isolates query logic in a testable, mockable units (see chapters 4.2.1 Repository Pattern and 4.2.2 Generic Repository Pattern). Advanced filtering uses *FormFilterDto*, which bundles filter criteria such as name fragments, publish state, date ranges and creator identity into composable LINQ predicates. All formular entities are exposed to the API via DTOs (see chapter 4.2.5) to control data exposure.

Service Layer

All business logic is encapsulated in *FormService*, which implements *IFormService*. The service maps between entities and DTOs, performs validation and orchestrates repository operations. Key operations include form CRUD, submission processing, hash generation and builder utilities.

Form Operations

The frontend interacts with Form Operations via a RESTful API. A dedicated *FormController* exposes all formular-related endpoints and delegates tasks to *FormService*. All interactions are grouped into seven functional regions, as shown in Figure 40:

- Read Operations
- Formular Posting
- Form Accessibility
- Submission Accessibility
- Submissions Operations
- CRUD for Forms
- Form Builder Operations
- Export Operations

```
[Route("api/form")]
[ApiController]
7 references
public class FormController :I
private readonly IFormServi
private readonly ILogger<Fo
1 reference
public FormController(ILogger
this.logger = logger;
this.formServices = formSi
}

Read
Posting Forms
Form Accessibility
Submission Accessibility
Submission
CRUD w.o. Read
Builder Usability Enhanceme
Export
}
```

Figure 40: FormController

Access Management

Because forms are used by non-authenticated users, both formulars and their submissions must be publicly accessible via unique URLs. NOVA solves this with form hashing, where *HashForm()* derives a unique string from the form entity. This string is embedded in the public URL and serves the sole purpose of external access, so that no internal ID or structure is ever exposed.

Alongside this access restriction, NOVA controls whether a form is accepting submissions through the *IsPublished* flag. Three endpoints exist to manage this state:

- *PublishForm()*
- *IsFormPublished()*
- *UnpublishForm()*

This means authenticated administrators can access forms via private endpoints, while external users access them only through their hashed versions, which are marked with `[AllowAnonymous]`:

```
// Private access

public async Task<IEnumerable<FormDto>> GetFormsAsync() {
    ...
}
```

```
// Public access
[HttpGet("hash/{hash}")]
[AllowAnonymous]
public async Task<ActionResult<GetFormByHashResponse>> GetFormByHash(string hash) {
    ...
}
```

Same process applies to submissions. Administrators access all submissions privately, while submitters can view their own specific submission through a uniquely hashed link generated by `HashSubmission()`.

Formular Posting

Forms are distributed to recipients via the Email System with Templates (see chapter 6.4.7), using a dedicated *Form Invitation* email template. Before sending, the form is hashed and the string is combined with the base URL for a direct access link. This link is embedded into the invitation email template, which means recipients can open the submission via the provided link and submit without any login:

```
// Send form via email
await emailService.SendEmailAsync(new SendFormInvitationDto() {
    ToAddress = email,
    ToName = member.FirstName + " " + member.LastName,
    FormName = form.Name,
    FormHash = await HashForm(formId),
    ExpiryDate = DateTime.UtcNow.AddDays(30)
});
```

Figure 41 illustrates the sending process to the supported posting targets through the *EmailService*:

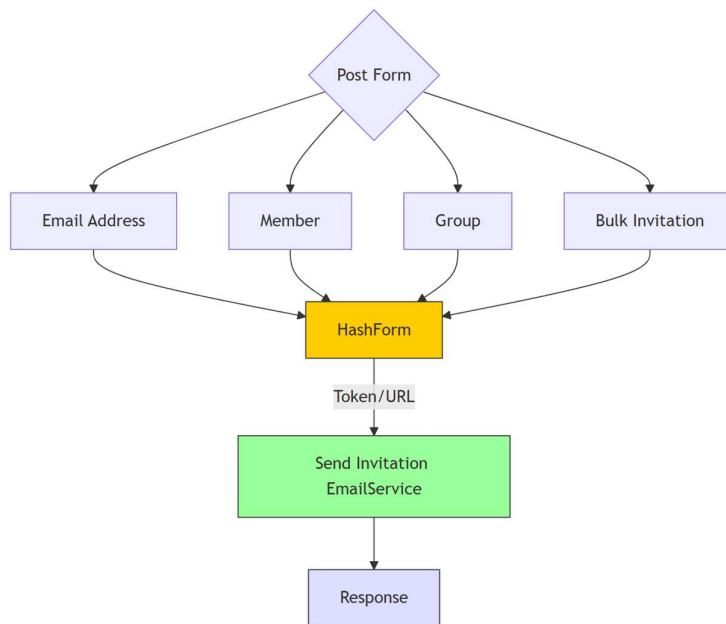


Figure 41: Form Posting Flow

Supported posting target being groups, members, email addresses and bulk invitations. The responses report success and failure details, enabling feedback in the Angular frontend.

Formular Submissions

When a recipient opens the invitation and submits a form, *ProcessSubmissionAsync()* orchestrates submission processing in the backend, as illustrated in Figure 42:

1. Validate the incoming submission
2. Enforce MaxSubmissionCount
3. Persist the submission and its values
4. Send a confirmation email to the submitter, if a designated email field is present in the submission.

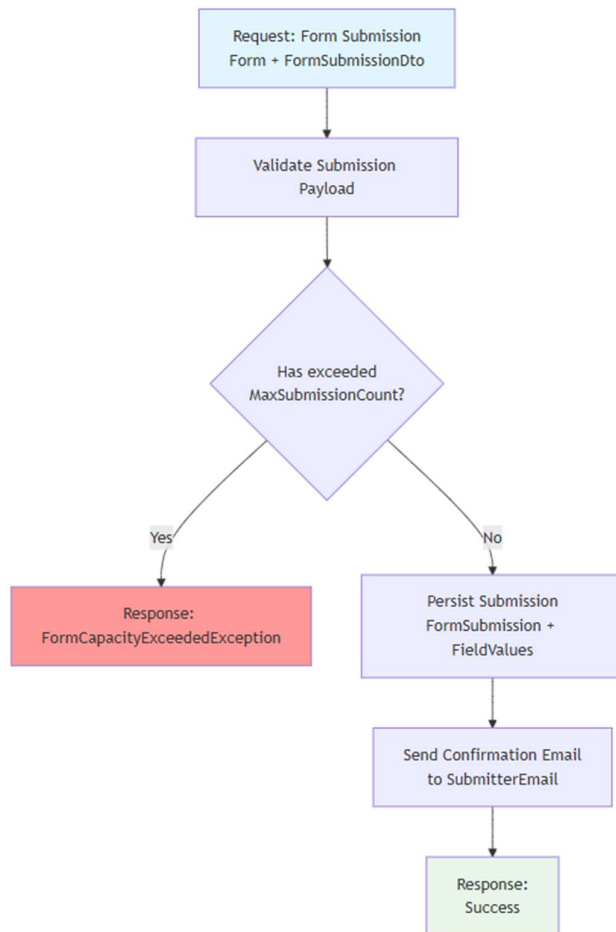


Figure 42: Form Submission Flow

The *FormSubmissionDto* captures all top-level fields for a submission. Answers are represented as *FormSubmissionValueDto* objects, each linking to submitted values and the original form field.

```

public class FormSubmissionDto : IExportable {
    [ExportColumn("Name", 0)]
    public string SubmitterName { get; set; }
    [ExportColumn("E-Mail", 1)]
    public string SubmitterEmail { get; set; }
    [ExportColumn("Eingereicht am", 2)]

```

```

    public DateTime SubmittedAt { get; set; }

    [ExportColumn("Bestätigungs-E-Mail gesendet", 3)]
    public bool ConfirmationEmailSent { get; set; }

    public ICollection<FormSubmissionValueDto> FieldValues { get; set; }
}

public class FormSubmissionValueDto : IExportable {
    public int SubmissionValueId { get; set; }

    public int SubmissionId { get; set; }

    public int FieldId { get; set; }

    [ExportColumn("Frage", 0)]
    public string FieldLabel { get; set; } = string.Empty;

    [ExportColumn("Antwort", 1)]
    public string FieldValue { get; set; } = string.Empty;
}

```

After submission, administrators retain full control with the ability to update accessibility status and correct data. If published, submitters can access their submission via the link sent over submissions post-receipt: `ChangeSubmissionStatusAsync` updates the review status, `SetSubmitterEmail` corrects contact data, and `PublishSubmissionAsync` (via `HashSubmission`) generates a read-only, shareable view of a single submission for the submitter.

Formular Export

Submissions are exported into Excel sheets via the

Export System(see chapter 1.1.1). Rather than producing a single flat sheet, NOVA generates multiple worksheets to provide both high-level view and full submission details:

- Overview Sheet – Briefly lists all submissions
- Detail Sheet – More detailed overview including submissions data
- Value Sheets – One sheet per submission to display all questions

```
// Sheet with all submissions

sheets.Add(new WorksheetDataSource<IExportable>(submissions, $"Formular_{formId}_Uebersicht", Alignment.Horizontal));

var overviewData = FormSubmissionOverviewDto.FromSubmissions(submissions);

if (overviewData.Any()) {
    sheets.Add(new WorksheetDataSource<IExportable>(
        overviewData, $"Formular_{formId}_Detailuebersicht", Alignment.Horizontal));
}

// One sheet per submission values
foreach (var submission in submissions) {
    ICollection<FormSubmissionValueDto> values = submission.Values;
    if (values == null || values.Count == 0)
        continue;
    sheets.Add(new WorksheetDataSource<IExportable>(
        values, $"Einreichung_{submission.SubmitterName}_{submission.SubmissionId}", Alignment.Vertical));
}

// Generate Excel
return exportService.ExportToExcel([.. sheets]);
```

Column names and their ordering are declared directly on the DTO via *[ExportColumn]* attributes, keeping export logic separate from DTO definitions like the approach in the Member Management System (see chapter 6.4.2).

Figure 43: Submission Export - Part1 and Figure 44: Submission Export - Part2 illustrate the full pipeline from request to HTTP download:

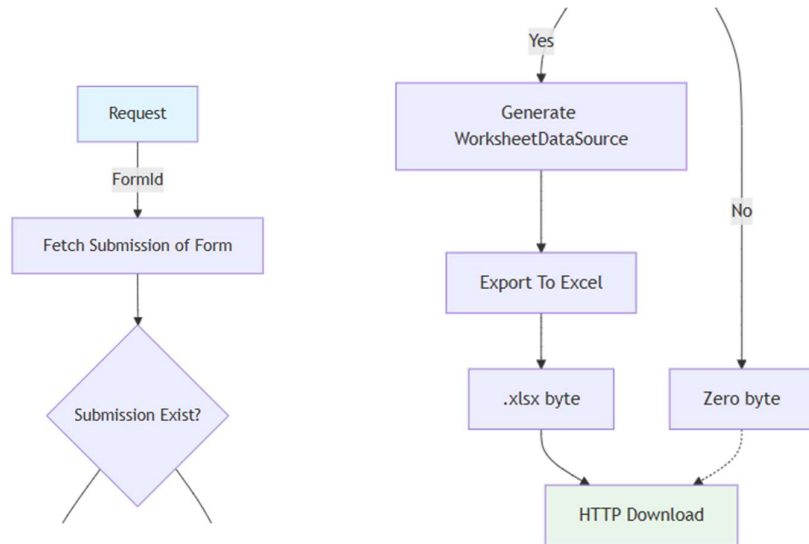


Figure 43: Submission Export - Part1

Figure 44: Submission Export - Part2

Formular Builder

The Formular Builder enables form authors to create and modify forms intuitively via the Angular frontend (see chapter 7 Frontend). The backend supports all necessary operations for live editing.

Core functionality is derived from CRUD operations, with sections and fields being creatable, updatable and removable. Furthermore, the backend provides quality-of-life operations for reordering and duplication.

Sections and fields are reordered by updating internal entity orders, thereby updating the frontend render order.

Duplication deep-copies the entire form and all its sections, fields and options. This enables template reuse for recurring events without modifying the original formular.

6.4.5 Import System

The import system enables bulk migration of member records from external sources into the NOVA database. It is exposed through **ImportController** at `api/import` and implemented in **SchulvereinVerwaltungssystem.Services/ImportService/ImportService.cs**.

Two file formats are supported. The primary format is a tab-separated values (TSV) export from the Campai member management system — the predecessor platform used by the Schulverein. The secondary format is an Excel workbook (.xlsx or .xls), processed via the EP-Plus library (OfficeOpenXml). Both code paths share the same validation and persistence logic; only the parsing stage differs.

The controller enforces a 50 MB request size limit and validates the file extension before delegating to the service:

```
// ImportController.cs – file validation and service delegation
[HttpPost("excel")]
[RequestSizeLimit(52_428_800)] // 50 MB
public async Task<ActionResult<ImportResultDto>> ImportFromExcel(
    IFormFile file, [FromQuery] bool validateOnly = false)
{
    if (file == null || file.Length == 0)
        return BadRequest("Keine Datei hochgeladen");

    var extension = Path.GetExtension(file.FileName).ToLower();
    if (extension != ".xlsx" && extension != ".xls")
        return BadRequest("Nur Excel Dateien (.xlsx, .xls) erlaubt");

    using var stream = file.OpenReadStream();

    var result = await _importService.ImportFromExcelAsync(stream,
        validateOnly);

    return Ok(result);
}
```

A dedicated /validate endpoint delegates to the same method with validateOnly = true, enabling a dry-run pass that reports validation errors without writing any records to the database. This two-phase approach allows administrators to review data quality before committing the import.

The Excel parser opens the first worksheet via package.Workbook.Worksheets[0], reads column headers from row 1, and maps each subsequent row to a **CampaiImportDto** using a header-name-keyed dictionary. Column positions are therefore not hard-coded; the format is tolerant of column reordering. The TSV parser applies equivalent header-driven mapping, splitting each line on the tab character:

```
// ImportService.cs – TSV header-driven mapping (excerpt)
```

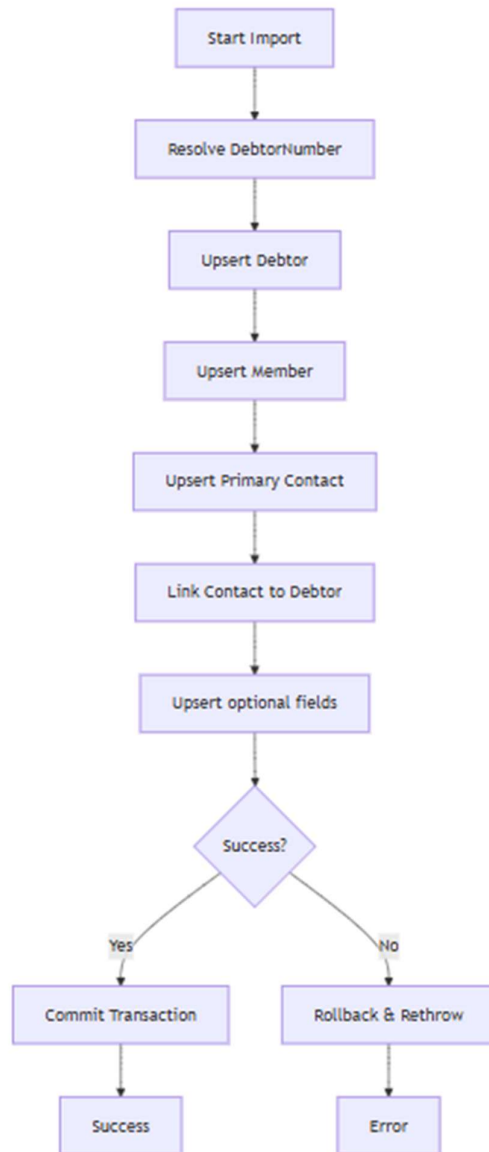
```

private CampaiImportDto MapTsvLineToDto(string[] headers, string[]
values)
{
    var dto = new CampaiImportDto();
    for (int i = 0; i < Math.Min(headers.Length, values.Length); i++)
    {
        switch (headers[i].Trim())
        {
            case "Vorname":      dto.Vorname      = values[i].Trim();
break;
            case "Nachname":      dto.Nachname      = val-
ues[i].Trim(); break;
            case "Mitgliedsnummer":  dto.Mitgliedsnummer  = val-
ues[i].Trim(); break;
            case "E-Mail":      dto.EMail      = values[i].Trim();
break;
            case "Mitgliedschaft":  dto.Mitgliedschaft  = va-
lues[i].Trim(); break;
            // ... remaining field mappings
        }
    }
    return dto;
}

```

Each parsed record passes through `ValidateCampaiDataAsync` before persistence. Validation failures are collected into `ImportResultDto.Errors` as **ImportErrorDto** objects carrying the row number, member number, error message, and entity type, allowing a complete error report to be returned in a single response rather than failing on the first invalid row. Records that pass validation are imported via `ImportSingleRecordAsync`, which creates the **Member**, associated **Contacts**, **PhoneNumbers**, **AdditionalEmails**, and **Debtor** records in the correct relational order within a shared **DatabaseContext** transaction. A `DebtorCounter` object thread-safely sequences new debtor numbers derived from the highest existing customer number in the database.

The returned **ImportResultDto** exposes TotalRecords, SuccessfulImports, FailedImports, Errors, ImportedMembers, and Duration, giving administrators a full audit of the operation.



6.4.6 Invoice Builder

The Invoice Builder is the programmatic subsystem responsible for constructing well-formed invoices from validated input data. It is implemented in **SchulvereinVerwaltungssystem.Services/InvoiceBuilderService/InvoiceBuilderService.cs** and exposed through **InvoiceTemplateController** (see chapter 6.4.8 for template-based generation). The direct invoice creation path runs through **InvoiceService** in **SchulvereinVerwaltungssystem.Services/InvoiceService/**.

An invoice is composed of a header record (**Invoice** entity) and one or more **InvoiceItem** line records. Each **InvoiceItem** carries a Description, Quantity, UnitPrice, and TaxRate. The total for each line is computed as $\text{Quantity} \times \text{UnitPrice}$, and the invoice's TotalAmount is the sum of all line totals. The TaxRate field on each item supports mixed-rate invoices where different items attract different VAT percentages — a requirement for invoices that bundle membership fees (exempt) with event tickets (standard rate).

The **CreateInvoiceDto** supplied by callers contains a DebtorId, DueDate, optional HeaderText and FooterText free-text fields, and a list of CreateInvoiceItemDto objects. If a ProductId is supplied on an item, **InvoiceService** resolves the referenced **Product** and inherits its DefaultPrice and TaxRate if the caller has not overridden them, ensuring catalogue prices are the default without preventing per-invoice customisation.

The invoice number is auto-generated at creation time using a sequential scheme: the service queries the highest existing invoice number within the current year and increments it, producing numbers in the format YYYY-NNNN (e.g., 2025-0047). This logic runs inside a database-level transaction to prevent duplicate numbers under concurrent creation.

After creation, the **PdfService** (see chapter 6.5.4) can immediately generate a printable invoice from the **InvoiceDto** returned by **InvoiceService**. The **QrCodeService** (see chapter 6.5.5) generates an EPC-compliant QR code at creation time, which is stored on the **InvoiceDto.QrCode** property as a Base64-encoded PNG and embedded in the PDF by **PdfService**.

Additional lifecycle operations exposed by **InvoiceController** (api/invoice) include mark-paid (sets PaidAt and flips Status to Paid), cancel (records a cancellation reason and sets Status to Cancelled), and batch/mark-paid (marks a list of invoice IDs paid in a single round-trip). A paginated search endpoint (POST api/invoice/search) accepts an **InvoiceSearchDto** with filters for debtor, status, date range, and overdue flag, returning both the matching invoice list and the total count for client-side pagination.

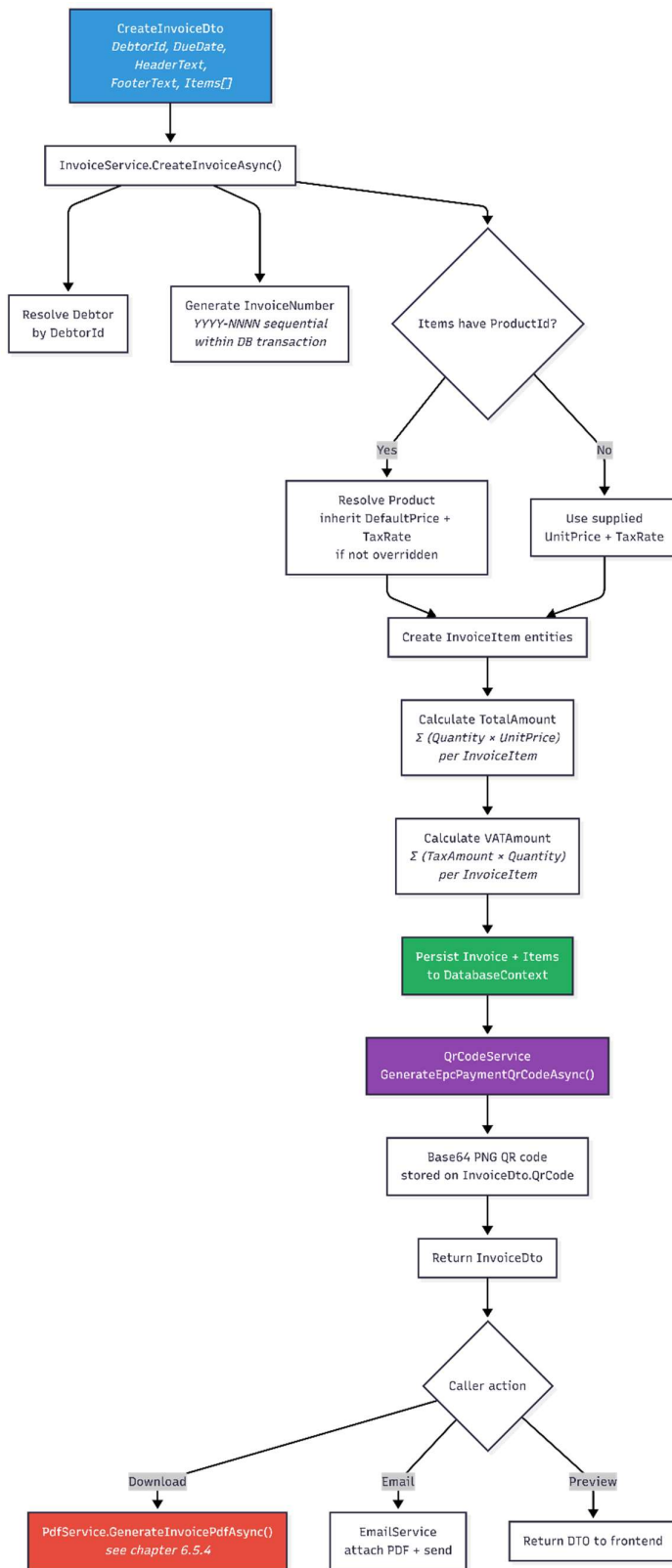


Figure 45: Invoice Construction Flow (Generated by Claude using NOVA Data)

6.4.7 Email System with Templates

The Email System in NOVA centralizes all outgoing communication (password reset, account confirmation, invoices, form invitations, calendar notifications) behind a single service abstraction built on MailKit's SMTP client (DOTNET Foundation, 2026). SMTP credentials, sender addresses and other transport options are provided by the Settings System via the *EmailSettings* class so they can be changed at runtime without redeploying the backend.

The chapter is structured as follows:

- Responsibilities and Layering
- Templates and Template Builder
- SMTP configuration
- Persistence and API surface

Responsibilities and layering

The central facade is *EmailService*, which orchestrates template resolution, message creation, and SMTP delivery. It delegates to three collaborators: *ITemplateService* (placeholder parsing and rendering), *IEmailFactory* (message construction per DTO type), and *IEmailSender* (SMTP transport). Higher-level modules such as authentication, the Formular System, invoicing, and event notifications never interact with MailKit (DOTNET Foundation, 2026) directly; instead, they pass typed DTOs to *EmailService.SendEmailAsync*, which selects the correct factory method:

```
public async Task SendEmailAsync<T>(T dto) where T : ISendableDto {
    EmailMessageDto email = dto switch {
        SendEmailDto emailDto => await emailFactory.CreateFromDto(emailDto, EmailTemplateEnum.EmailTemplate),
        SendPasswordResetDto resetDto => await emailFactory.CreatePasswordReset(resetDto),
        SendNotificationDto notifDto => await emailFactory.CreateNotification(notifDto),
        SendFormInvitationDto formDto => await emailFactory.CreateFormInvitation(formDto),
    };
}
```

```

    SendFormSubmissionDto submitDto => await emailFactory.Create-
    FormSubmission(submitDto),

    SendInvoiceDto invoiceDto => await emailFactory.CreateInvoice(in-
    voiceDto),

    _ => throw new NotSupportedException($"Email sending not imple-
    mented for type: {typeof(T).Name}")

};

await emailSender.SendAsync(email);
}

```

This pattern keeps controllers thin and aligns the Email System with the overall layered architecture. Figure 46 illustrates the high-level layering.

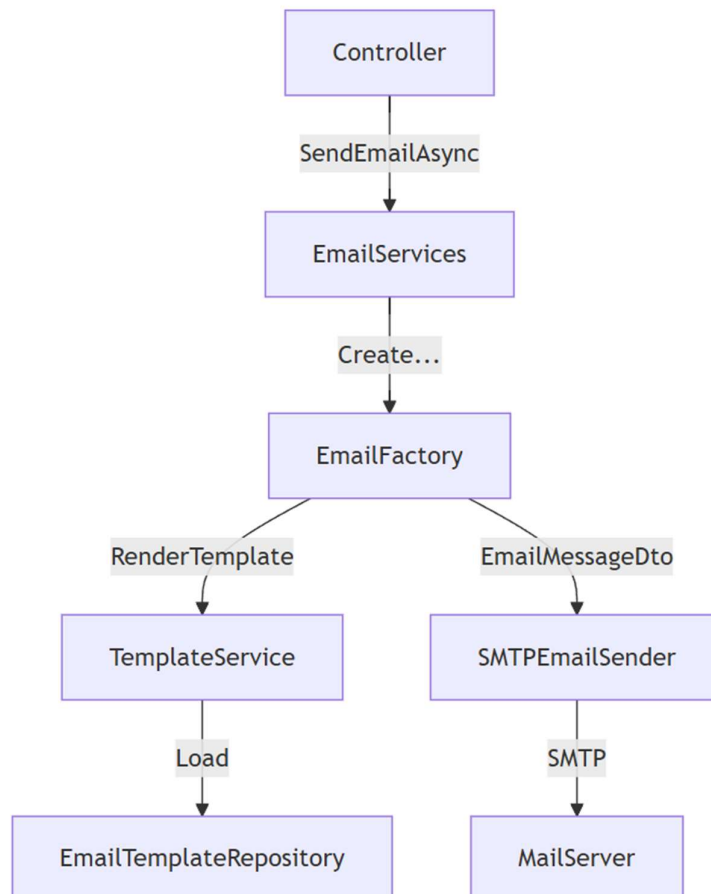


Figure 46: Email System Overview

Templates and Template Builder

All user-visible emails are backed by HTML templates stored in the database and accessed via *EmailTemplateRepository*. Each template is identified by an *EmailTemplate* enum value, for example *PasswordResetTemplate*, *FormInvitationTemplate*, *InvoiceTemplate*, and contains localized HTML with placeholder markers in the format *-*PlaceholderName**.

Template management is exposed via *EmailController* under *api/email/templates*. Administrators can list all templates including their required placeholders, update subject lines and HTML content, preview a template with sample data, and soft-delete templates.

TemplateService handles two responsibilities: discovering which placeholders a given template requires and rendering a final email body by replacing those placeholders with concrete values:

```
// Parse required placeholders from the template HTML
public async Task<List<string>> GetPlaceholders (EmailTemplateEnum
    template) {
    var dbTemplate = await _templateRepository.GetTemplateAsync (tem-
        plate)

    ?? throw new InvalidOperationException($"Template {template}
        not found");

    var matches = PlaceholderRegex.Matches (dbTemplate.ContentHtml);

    return matches.Select (m => m.Groups[1].Value).Distinct().Or-
        der().ToList();
}

// Replace all placeholders and return rendered HTML
public async Task<string> RenderTemplate (EmailTemplateEnum template,
    Dictionary<string, string>
    placeholders) {
    var dbTemplate = await _templateRepository.GetTemplateAsync (tem-
        plate)

    ?? throw new InvalidOperationException($"Template {template}
        not found");

    var content = dbTemplate.ContentHtml;
```

```
foreach (var (key, value) in placeholders) {  
    var marker = key.StartsWith("-*") ? key : $"-*{key}*-";  
    content = content.Replace(marker, value);  
}  
return content;  
}
```

On the Angular side, these endpoints form a template builder that allows privileged users to edit HTML content, see which placeholders the template requires, and render a live preview by calling `/templates/{id}/preview` with sample placeholder data. Figure 47 shows an example rendered password reset email.

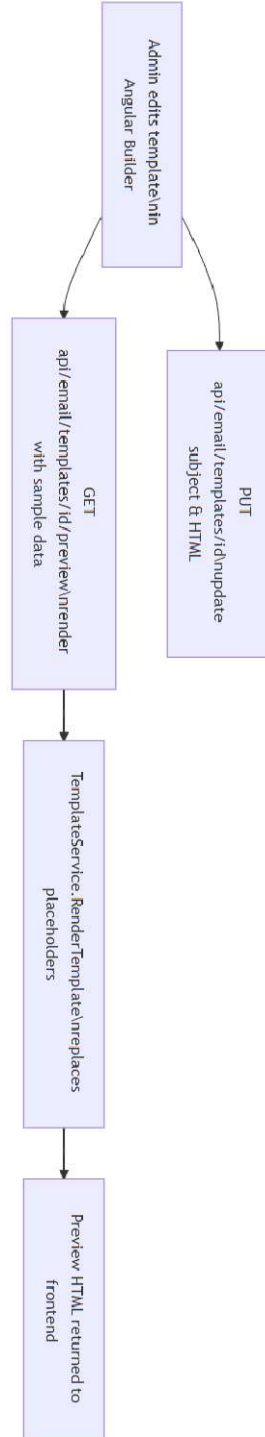


Figure 47:Email Template Flow (Generated by Claude using NOVA data)

Figure 48 shows an example of a rendered password reset email as it appears in the recipient's inbox after template processing.

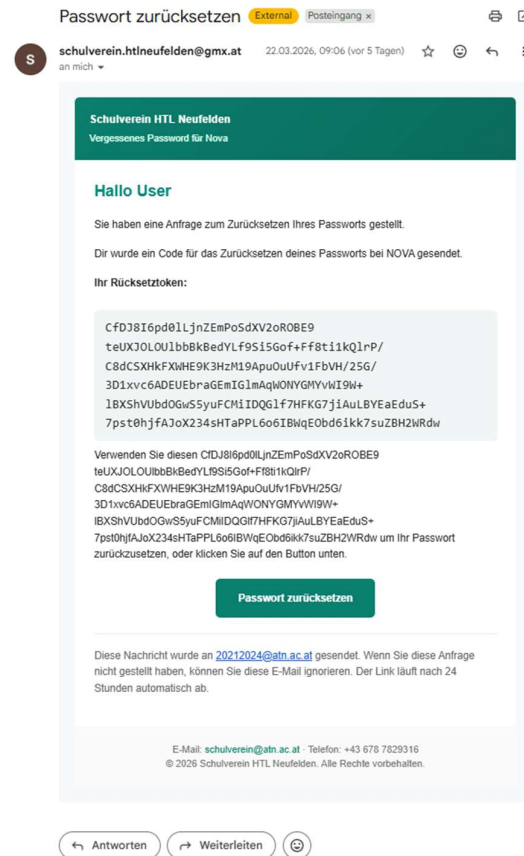


Figure 48: Email with Template

SMTP Configuration

SMTP configuration is provided by the *EmailSettings* typed settings class, which is populated from the Settings System at runtime via *IOptionsMonitor<EmailSettings>* (see chapter 6.4.10 Settings System). It contains the host, port, SSL flag, credentials, and sender address. Two configurations are used in practice:

Development uses a GMX test account with simple username and password credentials. This is sufficient for local development and the test server — developers can verify template changes by sending to a real mailbox without affecting external users.

Production uses the official school association Gmail account together with a Gmail App Password, ensuring emails appear to recipients as coming from the school association rather than a generic no-reply address, an explicit requirement from the client.

SMTPEmailSender reads these values at runtime and connects to MailKit accordingly:

```
public SMTPEmailSender(IOptionsMonitor<EmailSettings> settings,
                        ISmtpClient smtpClient,
                        ITemplateService templateService) {

    this.smtpClient      = smtpClient;
    this.settings        = settings.CurrentValue;
    this.templateService = templateService;
}

private async Task SendAsync(IEnumerable<string> recipients, string?
    subject,
                                string body, IEnumerable<Attachment>?
    attachments = null,
                                IEnumerable<string>? ccs = null,
                                IEnumerable<string>? bccs = null) {

    var mail = new MimeMessage { Subject = subject };
    mail.From.Add(MailboxAddress.Parse(settings.From));
    // recipients, CC/BCC and attachments omitted for brevity

    var secureOption = settings.EnableSsl ? SecureSocketOptions.Start-
        Tls : SecureSocketOptions.None;

    await smtpClient.ConnectAsync(settings.SmtpHost, settings.SmtpPort,
        secureOption);

    await smtpClient.AuthenticateAsync(settings.Username, settings.Password);

    await smtpClient.SendAsync(mail);

    await smtpClient.DisconnectAsync(true);
}
```

Switching from the test account to the production account requires only a settings change in the administration UI. No code changes or redeloys are required.

Persistence and API surface

Email template persistence is handled by *EmailTemplateRepository*, which extends the generic repository infrastructure. Templates support soft-delete via the *ISoftDeletable* interface and expose standard CRUD methods.

The public API surface is grouped under *api/email* in *EmailController* and exposes three functional areas:

Entity-centric sending targets a specific contact, member, debtor, or group by ID. The controller resolves the entity, builds a *SendEmailDto*, and delegates to *EmailService.SendEmailAsync*:

```
[HttpPost("send/contact/{contactId}")]
public async Task<IActionResult> SendToContact(int contactId, [FromBody] SendGenericEmailDto emailData) {
    var contact = await contactServices.GetContactAsync(contactId);
    if (contact == null || string.IsNullOrEmpty(contact.Email))
        return BadRequest("Contact has no email address.");

    await emailServices.SendEmailAsync(new SendEmailDto {
        Recipients = [contact.Email],
        Subject     = emailData.Subject,
        Body        = emailData.Body,
        Files       = emailData.Files
    });

    return Ok(new { Message = "Email sent successfully to contact." });
}
```

Generic sending via POST */api/email/email* and POST */api/email/notification* provides direct access for frontend-triggered emails and system notifications.

Template management exposes the CRUD and preview endpoints consumed by the Angular template builder, as described in the Templates section above.

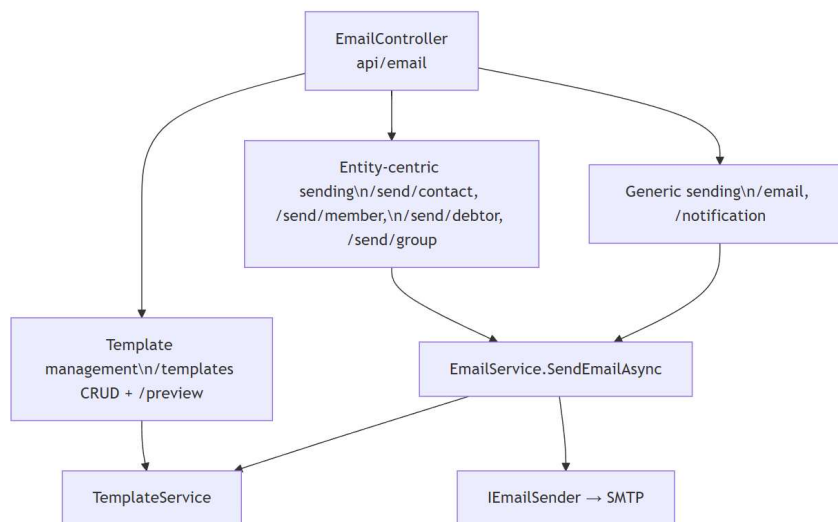


Figure 49:Email System Flow

6.4.8 Invoice system with Templates

The invoice template system enables administrators to define reusable invoice structures that can be instantiated as concrete invoices for one or many debtors with a single operation. The **InvoiceTemplate** entity in **Finance/InvoiceTemplate.cs** implements **ISoftDeletable** and stores a named template definition:

```
// InvoiceTemplate.cs - entity fields
public class InvoiceTemplate : ISoftDeletable
{
    [Key]
    public int TemplateId { get; set; }

    [Required, StringLength(200)]
    public string TemplateName { get; set; }

    public int? DefaultDebtorId { get; set; } // pre-fill
    target debtor
    public int PaymentTermsDays { get; set; } = 30;

    // Text templates with {{Placeholder}} syntax

```

```

    public string? SubjectTemplate      { get; set; }
    public string? HeaderTextTemplate  { get; set; }
    public string? FooterTextTemplate  { get; set; }
    public string? NotesTemplate        { get; set; }

    public bool      IsActive          { get; set; } = true;
    public bool      IsDeleted          { get; set; }
    public DateTime? DeletedAt          { get; set; }

    public virtual ICollection<InvoiceTemplateItem> TemplateItems {
        get; set; }
}

```

Each template contains zero or more **InvoiceTemplateItem** records that define the line items to be materialised into **InvoiceItem** instances. A template item may reference a catalogue **Product** by **ProductId**, in which case **InvoiceBuilderService** inherits the product's **DefaultPrice** and **TaxRate** at instantiation time if the template item does not specify its own values. The **DescriptionTemplate** field of each item supports placeholder tokens using **{{Variable}}** syntax.

Invoice generation from a template is handled by **GenerateInvoiceFromTemplateAsync** in **InvoiceBuilderService**:

```

// InvoiceBuilderService.cs – placeholder resolution and invoice construction
public async Task<InvoiceDto> GenerateInvoiceFromTemplateAsync(
    GenerateInvoiceFromTemplateDto generateDto, string userId)
{
    var template = await _templateRepository.GetByIdWithItemsAsync(generateDto.TemplateId);

    if (!template.IsActive)
        throw new InvalidOperationException("Template is not active");

    var debtor = await _debtorRepository.GetByIdAsync(generateDto.DebtorId);

```

```

// Build standard placeholder dictionary from debtor and date
context

var placeholders = generateDto.Placeholders ?? new Dictionary<string, string>();

placeholders["DebtorName"] = debtor.CompanyName ?? debtor.CustomerNumber;

placeholders["CustomerNumber"] = debtor.CustomerNumber;

placeholders["Year"] = DateTime.Now.Year.ToString();

placeholders["Month"] = DateTime.Now.ToString("MMMM");

placeholders["Date"] = DateTime.Now.ToString("dd.MM.yyyy");

var createDto = new CreateInvoiceDto
{
    DebtorId = generateDto.DebtorId,

    DueDate = generateDto.DueDate ??
DateTime.Now.AddDays(template.PaymentTermsDays),

    ReferenceNumber = ReplacePlaceholders(template.SubjectTemplate ?? "", placeholders),

    HeaderText = ReplacePlaceholders(template.HeaderTextTemplate ?? "", placeholders),

    FooterText = ReplacePlaceholders(template.FooterTextTemplate ?? "Vielen Dank!", placeholders),

    Items = template.TemplateItems
        .OrderBy(i => i.OrderIndex)
        .Select(item => new CreateInvoiceItemDto
        {
            ProductId = item.ProductId,

            Description = ReplacePlaceholders(item.DescriptionTemplate, placeholders),

            Quantity = item.Quantity,

            UnitPrice = item.UnitPrice
        }).ToList()
};

```

```

return await _invoiceService.CreateInvoiceAsync(createDto,
userId);
}

```

Bulk generation is available through `GenerateBulkInvoicesAsync`, which accepts a list of debtor IDs and runs the single-debtor generation loop for each, returning the list of created **InvoiceDto** objects together with a success rate metric. The **InvoiceTemplateController** at `api/invoice-template` exposes endpoints for template CRUD, soft delete, restore, paginated search with **InvoiceTemplateSearchDto**, preview generation (POST `api/invoice-template/{id}/preview`), single-debtor generation (POST `api/invoice-template/{id}/generate`), and bulk generation (POST `api/invoice-template/generate-bulk`). Template names are enforced as unique by `IsTemplateNameUniqueAsync` before creation and update.

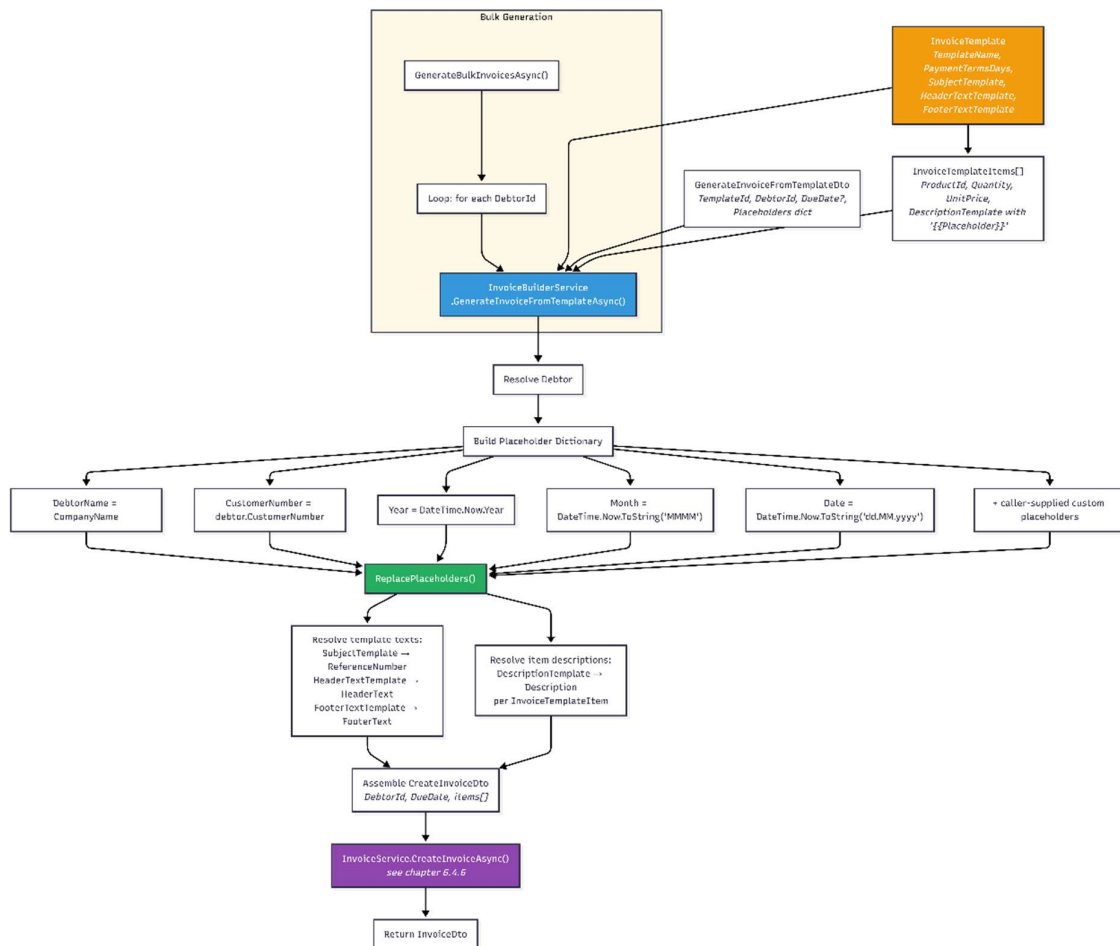


Figure 50: Template Instantiation Flow (Generated by Claude using NOVA Data)

6.4.9 Kalender System

The calendar system provides bidirectional event management between the NOVA internal store and Google Calendar. It is implemented across **CalendarController** (api/calendar), **GoogleCalendarService**, **CalendarEventService**, and **CalendarEventRepository**.

The **CalendarEvent** entity stores all metadata relevant to an event: Title, Description, StartDate, EndDate, IsAllDay, optional InvoiceId and MemberId foreign keys for contextual associations, SendInvitations, SendReminder, ReminderSent, GoogleEventId (populated once synced), and LastSyncedAt. The Invitees navigation property holds a collection of invitee records built from member and contact IDs.

GoogleCalendarService in **SchulvereinVerwaltungssystem.Services/** authenticates against the Google Calendar API using a service account key file whose path is supplied through GoogleCalendarSettings in **appsettings.json**. At construction time, **GoogleCredential.FromStream** reads the key file and scopes the credential to **CalendarService.Scope.Calendar**:

```
// GoogleCalendarService.cs - service account authentication
private CalendarService CreateCalendarService()
{
    using var stream = new FileStream(_settings.ServiceAccountKey-
        Path, FileMode.Open, FileAccess.Read);

    var credential = GoogleCredential.FromStream(stream)
        .CreateScoped(CalendarService.Scope.Calendar);

    return new CalendarService(new BaseClientService.Initializer
    {
        HttpClientInitializer = credential,
        ApplicationName       = _settings.ApplicationName
    });
}
```

If the key file is not present — as during local development without Google credentials — **CreateCalendarService** returns null and all sync methods return false or empty results gracefully, allowing the application to operate with local-only calendar data.

The **SyncEventAsync** method implements a create-or-update strategy: if **CalendarEvent.GoogleEventId** is empty the event is created in Google Calendar and the returned ID is stored back on the entity; if an ID already exists the event is updated in place. This idempotent pattern makes it safe to call **SyncEventAsync** after every create and update operation in **CalendarController** without risk of duplicate entries:

```
// GoogleCalendarService.cs - create-or-update sync logic
```

```

public async Task<bool> SyncEventAsync(CalendarEvent calendarEvent)
{
    if (string.IsNullOrEmpty(calendarEvent.GoogleEventId))
    {
        // First sync: create in Google and store the returned ID
        var googleId = await CreateEventAsync(calendarEvent);
        if (!string.IsNullOrEmpty(googleId))
        {
            calendarEvent.GoogleEventId = googleId;
            calendarEvent.LastSyncedAt = DateTime.UtcNow;
            return true;
        }
        return false;
    }
    else
    {
        // Subsequent syncs: update the existing Google event
        var success = await UpdateEventAsync(calendarEvent);
        if (success) calendarEvent.LastSyncedAt = DateTime.UtcNow;
        return success;
    }
}

```

The `ConvertToGoogleEvent` helper maps the NOVA **CalendarEvent** to the Google Event object, setting `TimeZone = "Europe/Vienna"` on both the start and end `EventDateTime` to ensure correct time handling in the Austrian locale.

CalendarController exposes the following operations:

```

GET api/calendar/events - retrieve all stored events.

GET api/calendar/events/{eventId} - retrieve a single event with its
invitees.

POST api/calendar/events - create an event, build invitees, persist,
attempt Google sync, and fire invitation emails asynchronously
via Task.Run.

PUT api/calendar/events/{eventId} - update an event, rebuild invitees,
and re-sync to Google.

```

```

DELETE api/calendar/events/{eventId} - delete the event from Google
(if synced) and then from the local database.

GET api/calendar/events/date-range - query events within a date range
(maximum 365-day window).

POST api/calendar/events/{eventId}/sync-to-google - explicitly trig-
ger a sync for a specific event.

POST api/calendar/sync/all-pending-to-google - batch-sync all events
lacking a GoogleEventId.

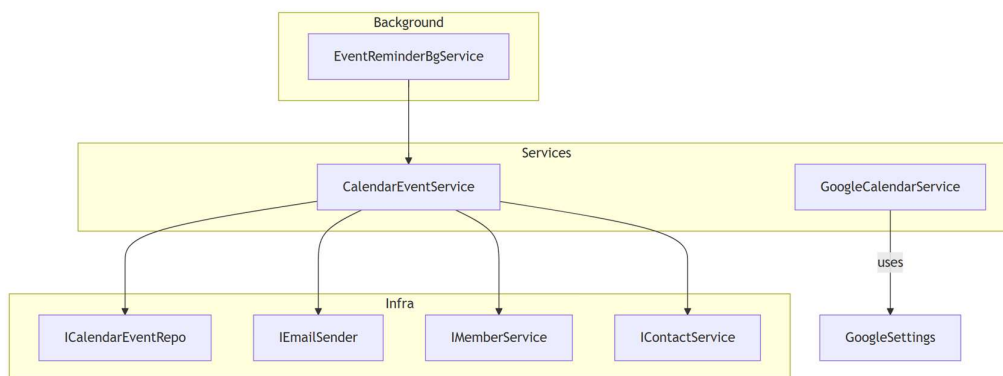
POST api/calendar/import/from-google - import Google Calendar events
not yet present in the local store.

POST api/calendar/sync/two-way - bidirectional sync: import new Google
events, remove local events deleted from Google.

POST api/calendar/events/create-invoice-reminder/{invoiceId} - auto-
create a reminder event 30 days in the future linked to a specific
invoice.

```

Invitation emails are dispatched in a fire-and-forget Task.Run block to avoid blocking the HTTP response, with any dispatch errors caught and logged at Error level. Reminder processing is handled by the **EventReminderBackgroundService** hosted service registered in **Program.cs**, which periodically calls ProcessPendingRemindersAsync on **CalendarEventService** to dispatch reminder emails for events approaching their start date.



6.4.10 Settings System

The Settings System forms the configuration backbone of NOVA and enables administrators to manage system-wide parameters such as SMTP credentials, JWT token lifetimes and external services accounts without redeploying the backend.

Configuration data is stored in dedicated database entities and loaded into strongly typed configuration objects during application startup and at runtime, providing a single, consistent source of truth for all modules.

Figure 51 shows an overview of the entire Settings System in the backend including persistence, runtime services and the **Settings* classes.

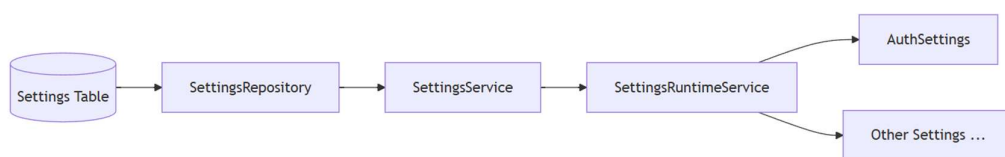


Figure 51: Settings System Overview

The Settings chapter is divided into the following sections:

- Settings
- Runtime Binding
- Runtime Changes
- Startup

Settings

This section describes settings persistence in the database. Typical examples include *AuthSettings* (JWT token lifetimes), *EmailSettings* (SMTP configuration) and invoice number configuration, all of which are irreplaceable for backend services.

Settings Entity

All configurable values are represented by the *Settings* entity, which captures both the raw value and metadata required for categorization and security.

```
public class Settings {
    [Key]
    public int Id { get; set; }
```

```

    [Required]
    [StringLength(100)]
    public string Name { get; set; }

    [StringLength(2500)]
    public string Description { get; set; }

    [Required]
    public SettingType Type { get; set; }

    [Required]
    public string Value { get; set; }

    [Required]
    public int SettingsCategoryId { get; set; }

    public SettingsCategory SettingsCategory { get; set; }

    public bool IsPassword { get; set; }
}

```

Each setting is grouped into a *SettingsCategory*, like “Auth” or “Email”, which allows the administrative UI to present related values together and make configuration navigation easier. The *IsPassword* flag marks sensitive entries, such as *TokenPrivateKey* or *SMTP passwords*, which must be masked in responses and user interfaces so they cannot be read even by administrators viewing configuration lists.

DTOs, Masking and Internal Representation

To decouple internal entities from API consumers, the Settings System uses two distinct DTO types:

- *SettingsDto* is used for external communication and masks password values
- *SettingsInternalDto* is used inside the backend and contains raw values without masking passwords

```

public static SettingsDto ToDto(this Settings entity)
=> new() {
    Id = entity.Id,
    CategoryId = entity.SettingsCategoryId,
    Name = entity.Name,

```

```

        Description = entity.Description,
        Type = (int)entity.Type,
        Value = entity.IsPassword ? $"*****" : entity.Value, //
        Hide if password
    };

    public static SettingsInternalDto ToInternalDto(this Settings entity)
    => new() {
        Name = entity.Name,
        Type = (int)entity.Type,
        Value = entity.Value
    };

```

SettingsDto guarantees that secrets are never exposed over the API, because callers always receive placeholders instead of the real values when *IsPassword* is set.

SettingsInternalDto preserves the original values and is therefore used exclusively in the backend by the runtime configuration loading mechanism described below.

Data and Service Layer

Persistence for settings is handled in *SettingsRepository*, which extends the generic repository and a specific *ISettingsRepository* (see chapters 4.2.1 Repository Pattern and 4.2.2 Generic Repository Pattern).

All higher-level operations are encapsulated by *ISettingsService* and *SettingsService*, which follow the same service pattern as in Member Management.

SettingsService uses *SettingsRepository* for CRUD operations and *ISettingsRuntimeService* to reload in-memory configuration after changes, so the detailed repository and service patterns are not repeated here.

Runtime Binding

Runtime binding covers how string-based values from the Settings table are converted into strongly typed configuration objects and injected into the application. The main building blocks are *ISettingsConverter*, *SettingsRuntimeService* and the common *ISetting* interface implemented by all typed settings classes.

Typed Conversions via *ISettingsConverter*

A dedicated *ISettingsConverter* interface and its implementation *SettingsConverter* encapsulate the logic of converting string values from the database into strongly typed configuration values.

```
public class SettingsConverter : ISettingsConverter {  
    public T GetSettingValue<T>(IEnumerable<SettingsInternalDto> settings, string settingName) {  
        return GetSettingValue<T>(settings.First(s => s.Name == settingName));  
    }  
    public T GetSettingValue<T>(SettingsInternalDto dto) {  
        // Directly cast Type with (T) (object)  
        return (SettingType)dto.Type switch {  
            SettingType.String => (T) (object)dto.Value,  
            SettingType.Integer => (T) (object)dto.Value,  
            SettingType.Boolean => (T) (object)Convert.ToBoolean(dto.Value),  
            SettingType.DateTime => (T) (object)Convert.ToDateTime(dto.Value),  
            _ => default!  
        };  
    }  
}
```

This converter allows the rest of the application to work with native types for configuration, for example *bool* for feature flags or *DateTime* for expiration settings, while persisting all values as strings in the database.

Because *SettingsConverter* operates on *SettingsInternalDto*, password masking has no impact on runtime behavior and secrets remain fully available to internal components while still being hidden from external clients.

Runtime Binding to Settings

ISettingsRuntimeService is responsible for loading internal settings DTOs into the strongly typed options classes that are registered in the dependency injection container.

```
public class SettingsRuntimeService : ISettingsRuntimeService
{
    ...

    public async Task LoadSettingAsync<T>(IEnumerable<SettingsInternalDto> settings) where T : ISetting
    {
        // Get service
        var monitor = serviceProvider.GetRequiredService<IOptionsMonitor<T>>();
        var instance = monitor.CurrentValue;

        // Configure setting object
        instance.Configure(settings);
    }

    public async Task LoadSettingsAsync(IEnumerable<SettingsInternalDto> settings)
    {
        // Load all settings
        await LoadSettingAsync<AppSettings>(settings);
        await LoadSettingAsync<AuthSettings>(settings);
        ...
    }
}
```

Each strongly typed settings class implements a common *ISetting* interface with a *Configure* method that receives the full list of *SettingsInternalDto* objects and extracts only the values relevant for that class.

This pattern keeps the Settings System generic, while each **Settings* class is responsible for its own mapping logic.

System Settings Configuration

Several system settings, such as *AuthSettings*, combine static configuration values with database-backed values.

AuthSettings illustrates how a typed configuration class uses the shared converter to read its values from the Settings System.

```
public class AuthSettings :ISetting {
    public string TokenPrivateKey { get; set; }
    public int TokenExpirationMinutes { get; set; }

    /// <inheritdoc/>
    public Func<IEnumerable<SettingsInternalDto>, string, object> Converter { get; set; }

    /// <inheritdoc/>
    public void Configure(IEnumerable<SettingsInternalDto> settings) {
        TokenExpirationMinutes = Convert.ToInt32(Converter(settings,
            "TokenExpirationMinutes"));
    }
}
```

The *TokenPrivateKey* is injected directly from configuration (for example environment variables), while *TokenExpirationMinutes* is resolved dynamically from the Settings table at runtime.

This pattern allows NOVA to combine static deployment-time configuration with flexible, administrator-managed values without mixing concerns.

Runtime Changes

Runtime changes describe how updates to settings made by administrators are propagated into the running application without a restart.

The central piece is *SettingsService*, which coordinates between the repository layer and *SettingsRuntimeService* after each change.

Applying Changes via Settings Service

All CRUD operations on settings are encapsulated by *ISettingsService* and its implementation *SettingsService*, which are injected wherever configuration management is required.

After each update or patch, *SettingsService* reloads all internal settings and triggers a runtime rebinding:

```
public async Task<PatchSettingsResponse> PatchAsync(int id, PatchSettingsDto dto) {  
    ...  
  
    await settingsRepository.UpdateAsync(existing);  
    await settingsRepository.SaveChangesAsync();  
  
    // Reload Settings  
    var settings = await GetAllInternalAsync();  
    await settingsRuntimeService.LoadSettingsAsync(settings);  
  
    return new PatchSettingsResponse {  
        Setting = existing.ToDto()  
    };  
}
```

By reloading the in-memory settings after each update or patch, the service guarantees that configuration changes are applied without restarting the application or recycling the web host.

From the perspective of the Angular frontend, administrators simply change values in a settings screen; in the background, the backend reloads all dependent configuration objects immediately.

Startup

The Startup section explains how settings are wired at application startup, how the converter is injected and how initial settings (such as invoice numbering) are seeded and hot-loaded.

ConfigureNovaSettings Extension

At application startup, NOVA configures all strongly typed settings classes and wires them to the shared converter through the *ConfigureNovaSettings* extension class.

```
public static class ConfigureNovaSettings
{
    public static IServiceCollection ConfigureSettings(this
        IServiceCollection services, IConfiguration configuration)
    {
        ...

        // Auth Settings

        services.Configure<AuthSettings>(config => {

            config.TokenPrivateKey = configuration.Get-
                Value<string>("AuthSettings:TokenPrivateKey")

            ?? throw new ArgumentNullException("Token key is null");

        });

        ...

        services.AddOptions<AuthSettings>()

            .Configure<ISettingsConverter>((settings, converter) =>

                settings.Converter = (settingsDtos, key) => converter.GetSet-
                    tingValue<object>(settingsDtos, key));

        ...

        return services;
    }

    ...
}
```

The *ConfigureSettings* method wires each **Settings* class so that its *Converter* field is resolved via dependency injection, giving every settings class access to *ISettingsConverter* without introducing direct coupling.

Static configuration values, for example *TokenPrivateKey*, are read from *IConfiguration*, while dynamic values, for example *TokenExpirationMinutes*, are read later from the *Settings System* using the injected converter.

Initial Hot-Loading of Settings

At runtime, *ConfigureSettingsAsync* is called during application startup to hot-load all settings after the database connection and schema are initialized.

```
public static async Task ConfigureSettingsAsync(this WebApplication
    service)
{
    // Load settingsruntime service
    using var scope = service.Services.CreateScope();
    var services = scope.ServiceProvider;

    // Seed InvoiceSettings if not present
    await SeedInvoiceSettingsAsync(services);

    var settingsRuntimeService = services.GetRequiredService<ISettingsRuntimeService>();
    var settingsService = services.GetRequiredService<ISettingsService>();

    var settings = await settingsService.GetAllInternalAsync();

    // Load all settings
    await settingsRuntimeService.LoadSettingsAsync(settings);
}
```

This method ensures that critical settings for invoice numbering exist in the database and then loads all available settings into the typed configuration objects via *SettingsRuntimeService*.

As a result, the application starts with a fully initialized configuration state, and subsequent changes to the *Settings* table can be applied at runtime without requiring manual restarts.

6.5 Secondary Features

In addition to the main functional modules, NOVA's backend implements a series of secondary features that complement and extend the system's capabilities. These components are not essential to the core business logic but significantly enhance user experience, operational efficiency, and integration with external systems.

Secondary features include supporting services such as group and dashboard management, as well as utilities for data export; filtering, document generation, and QR-code creation. Each feature is designed as an independent service within the backend architecture, allowing these enhancements to be maintained or extended without affecting the primary workflow modules.

Collectively, these secondary elements provide valuable auxiliary functionality that improves system usability and supports administrators in daily operation, data reporting, and communication tasks.

6.5.1 Dashboard Service

The dashboard provides a single, aggregated snapshot of the application's operational state, consumed by the Angular frontend to populate the start screen visible to authenticated users immediately after login. The aggregation is performed entirely server-side by **DashboardService** in **SchulvereinVerwaltungssystem.Services/DashboardService/DashboardService.cs**, which composes statistics from four domain services and repositories.

DashboardController at `api/dashboard` exposes a single endpoint:

```
// DashboardController.cs – single aggregation endpoint
[HttpGet]
[ProducesResponseType(typeof(DashboardDto), StatusCodes.Status200OK)]
public async Task<ActionResult<DashboardDto>> GetDashboard()
{
    var dashboard = await _dashboardService.GetDashboardAsync();
    _logger.LogInformation("Dashboard data retrieved successfully");
    return Ok(dashboard);
}
```

DashboardService receives **IMemberServices**, **IInvoiceService**, **IFormServices**, and **ICalendarEventRepository** via constructor injection and calls each in sequence within `GetDashboardAsync`:

```
// DashboardService.cs – aggregation of four data domains

public async Task<DashboardDto> GetDashboardAsync()
{
    var memberStatistics = await GetMemberStatisticsAsync();
    var invoiceStatistics = await _invoiceService.GetInvoiceStatisticsAsync();
    var formStatistics = await GetFormStatisticsAsync();
    var upcomingEvents = await GetUpcomingEventsAsync();

    return new DashboardDto
    {
        MemberStatistics = memberStatistics,
        InvoiceStatistics = invoiceStatistics,
        FormStatistics = formStatistics,
        UpcomingEvents = upcomingEvents
    };
}
```

The **DashboardDto** returned to the client, defined in **SchulvereinVerwaltungssystem.Core/DTOs/DashboardDto.cs**, contains four sub-objects:

- **MemberStatisticsDto** — TotalMembers, ActiveMembers, InactiveMembers, SuspendedMembers, and MembersByMembershipType (a dictionary mapping membership product name to count). These values are derived by calling GetMemberCountAsync, GetActiveMemberCountAsync, GetMemberCountByMemberStatusAsync, and GetMemberCountByMembershipProductAsync on **IMemberServices**.
- **InvoiceStatisticsDto** — total invoice count, paid/unpaid/overdue counts, and aggregate amounts. Delegated entirely to **IInvoiceService.GetInvoiceStatisticsAsync()**, which issues a single EF Core LINQ projection over the **Invoices** DbSet.
- **FormStatisticsDto** — TotalForms, PublishedForms, and TotalSubmissions. Computed by loading all forms via **IFormServices.GetFormsAsync()** and aggregating in memory, since the form count in typical Schulverein deployments is small.
- **UpcomingEvents** — the five next chronological events within the following 30 days, mapped from **CalendarEvent** entities to **UpcomingEventDto** objects (carrying EventId, Title, Description, StartDate, EndDate, IsAllDay). The repository query

uses `GetEventsByDateRangeAsync(now, now.AddDays(30))` and the result is ordered by `StartDate` and truncated with `Take(5)`:

```
// DashboardService.cs – upcoming events query and projection
private async Task<List<UpcomingEventDto>> GetUpcomingEventsAsync()
{
    var now = DateTime.UtcNow;
    var events = await _calendarEventRepository
        .GetEventsByDateRangeAsync(now, now.AddDays(30));

    return events
        .OrderBy(e => e.StartDate)
        .Take(5)
        .Select(e => new UpcomingEventDto
        {
            EventId = e.EventId,
            Title = e.Title,
            Description = e.Description,
            StartDate = e.StartDate,
            EndDate = e.EndDate,
            IsAllDay = e.IsAllDay
        })
        .ToList();
}
```

The `[Authorize]` attribute on **DashboardController** ensures that the aggregated metrics are not accessible to unauthenticated callers. All four data fetches execute sequentially within a single HTTP request; future performance optimisation could parallelise them with `Task.WhenAll`, but the current sequential approach ensures deterministic ordering for logging and simplifies error attribution.

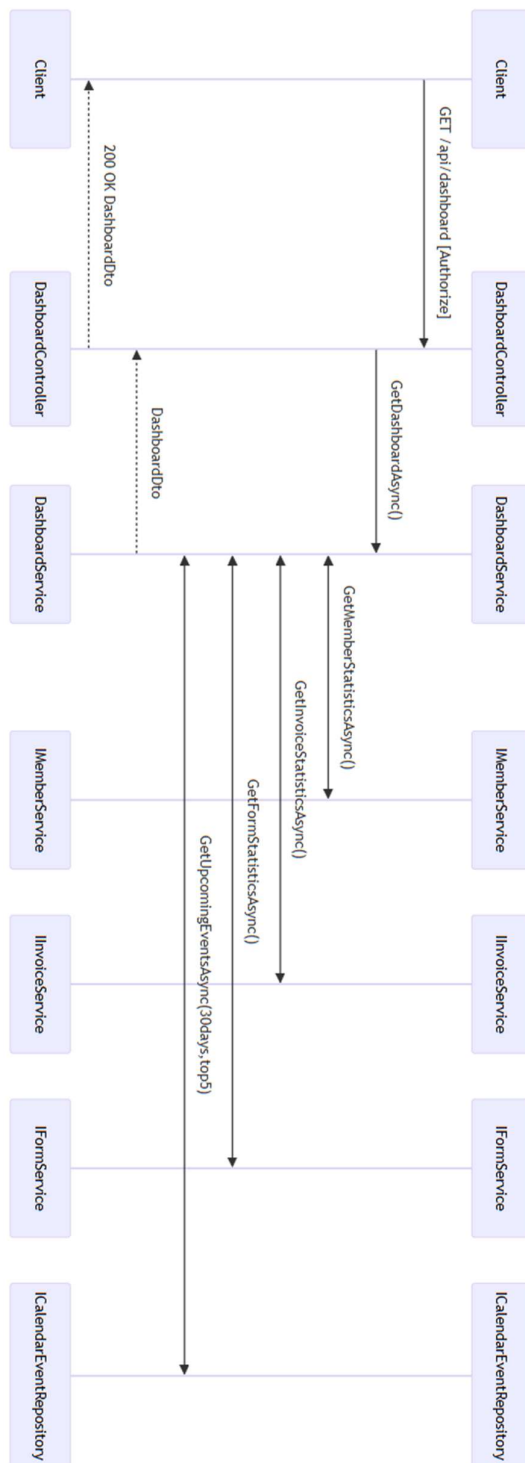


Figure 52: Dashboard Service Flow (Generated by Claude using NOVA Data)

6.5.2 Export System

The Export System provides a reusable, Excel-based export pipeline for all NOVA modules, including Member Management (above) and other systems. Instead of generating spreadsheets individually per use case, it uses a generic, attribute-driven model that turns arbitrary DTOs into one or more Excel worksheets with consistent layout, localization, and basic formatting via ClosedXML (ClosedXML, 2026).

Excel Export Architecture

From a backend perspective, an Excel export consists of three building blocks:

- Exportable DTOs,
- Worksheet Data Sources
- Sheet Processor.

Feature-specific services (for example member management or form management) collect data, configure the desired sheets, and delegate file creation to a central *ExportService*, which coordinates metadata and Excel generation.

Figure 53: Export System Overview **Fehler! Verweisquelle konnte nicht gefunden werden.** shows the simplified processing pipeline from controller to HTTP download.

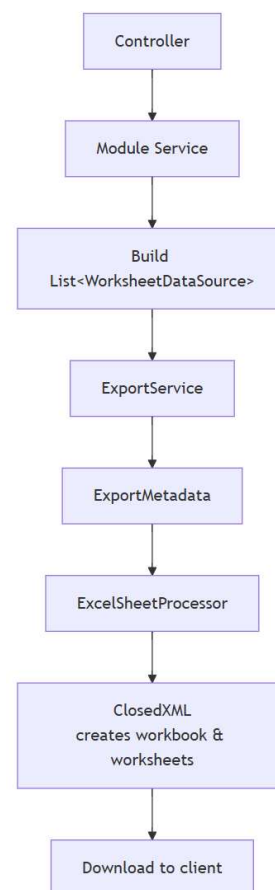


Figure 53: Export System Overview

In practice, a module service collects a list of exportable DTOs, wraps them into one or more *WorksheetDataSource* instances, and passes this list to *ExportService.ExportToExcel*, which returns a byte array for the Excel file. The controller simply exposes this as a *FileResult*, keeping HTTP concerns separate from export logic.

Exportable DTOs and Column Metadata

All exportable types implement the marker interface *IExportable*, which signals that a DTO can be written to Excel. Column definitions are placed directly on DTO properties using the *[ExportColumn]* attribute. This attribute defines the column header, the order, and optionally whether a property represents a comparison series.

```
public class FormSubmissionDto : IExportable {  
    [ExportColumn("Name", 0)]  
    public string SubmitterName { get; set; } = string.Empty;  
    [ExportColumn("Email", 1)]  
    public string SubmitterEmail { get; set; } = string.Empty;  
    [ExportColumn("Eingereicht am", 2)]  
    public DateTime SubmittedAt { get; set; }  
    // Internal fields without ExportColumn are not exported  
    public int FormId { get; set; }  
}
```

Only properties marked with *[ExportColumn]* appear in the Excel file, internal fields such as *FormId* are ignored until they are explicitly annotated. This keeps the export configuration close to the domain model and allows each module to add, remove, or reorder columns without changing the central export implementation.

ExportMetadata encapsulates the reflection logic: it scans an *IExportable* type for *ExportColumn* attributes, orders them by *ColumnOrder*, and returns a list of column descriptors (*ExportColumnInfo*) that contain the header text, order, comparison flag, and the underlying *PropertyInfo*. The sheet processor uses this metadata to decide which properties to render and in which order.

Worksheet Data Source and Alignment

The Export System separates data collection from sheet layout using *IWorksheetDataSource* and *WorksheetDataSource<T>*. A data source carries three important pieces of information: the sheet name, the list of *IExportable* items, and the desired alignment that controls whether data is written horizontally or vertically.

Horizontal alignment is used for classic tables in which each DTO becomes a row and each exported property becomes one or more columns, for example an overview of all submissions. Vertical alignment flips this perspective. One DTO becomes a “value pair” section with labels in the first column and values to the right, which is useful for detailed, per-record sheets.

Creating two differently aligned sheets for the same data looks as follows in simplified form:

```
var overview = new WorksheetDataSource<FormSubmissionDto>(
    submissions,
    $"Formular_{formId}_Übersicht",
    Alignment.Horizontal);

var detailsPerSubmission = submissions.Select(submission =>
    new WorksheetDataSource<FormSubmissionValueDto>(
        submission.Values,
        $"Einreichung_{submission.SubmitterName}_{submission.SubmissionId}",
        Alignment.Vertical));
```

By configuring multiple data sources, a module can combine an aggregated overview sheet with one or more detailed sheets in a single export without duplicating export code.

Excel Sheet Processor

The sheet processor is responsible for turning *ExportColumnInfo* metadata and *IExportable* instances into formatted Excel cells. It receives a worksheet instance from *ClosedXML*, the resolved columns from *ExportMetadata*, and the aligned data, and then performs three main tasks:

- It writes header cells for the selected alignment, either as a header row (horizontal) or header column (vertical) and can also mark comparison columns visually if *IsComparison* is set.
- It iterates over the items and writes property values into cells, expanding collection-type properties into multiple adjacent cells for comparison exports.
- It applies basic type-aware formatting, for example converting *bool* values into localized “Ja/Nein” strings and formatting *DateTime* values with the pattern *yyyy-MM-dd HH:mm*, while also setting the Excel date format so that spreadsheets remain sortable and filterable.

All low-level interactions with ClosedXML, such as setting cell values, styles, and number formats, are encapsulated inside *ExcelSheetProcessor*, so feature modules never depend on the Excel library directly. This keeps the export mechanism testable and allows the library to be swapped or upgraded with minimal impact.

Usage Across NOVA

The same Excel Export System is reused across NOVA wherever structured data must be exported, such as member lists or form submissions. Each module defines its own export DTOs and *WorksheetDataSource* configurations while relying on the shared *ExportMetadata* and *ExcelSheetProcessor* components for consistent output and behavior.

A detailed example of how the Formular System assembles multiple sheets (overview, detail overview, and per-submission sheets) using this infrastructure is described in chapter 1.1.1

Formular System, where the export of form submissions is explained from a feature perspective.

6.5.3 PDF Generation

PDF documents in NOVA are generated using the QuestPDF library, a .NET-native, code-first document layout engine that models page composition through a fluent C# API. This approach was chosen in preference to HTML-to-PDF conversion libraries (such as wkhtmltopdf or Puppeteer) for three reasons. First, QuestPDF operates entirely in-process without spawning external browser or rendering processes, eliminating the deployment complexity and OS-level dependencies that HTML converters introduce in Docker environments. Second, the layout model is deterministic: pixel-accurate column widths, borders, and font sizes are specified programmatically rather than inferred from CSS rendering, making the output reproducible across all environments. Third, QuestPDF's Community License permits commercial-equivalent use without royalty for open-source projects.

PdfService in **SchulvereinVerwaltungssystem.Services/PdfService/PdfService.cs** implements **IPdfService** and exposes two public methods: **GenerateInvoicePdfAsync(InvoiceDto)** and **GenerateBulkInvoicePdfAsync(BulkInvoiceDto)**. Both return **Task<byte[]>**, allowing the generated PDF to be streamed directly to the HTTP response or attached to an email without disk I/O. The QuestPDF license mode is set at construction time:

```
// PdfService.cs - constructor with QuestPDF Community licence
public PdfService(
    ILogger<PdfService> logger,
    IDebtorRepository debtorRepository,
    IQrCodeService qrCodeService,
    IOptionsMonitor<BankSettings> bankSettings)
{
    QuestPDF.Settings.License = LicenseType.Community;
    QuestPDF.Settings.CheckIfAllTextGlyphsAreAvailable = false;
    // ... field assignments
}
```

Document composition follows QuestPDF's container model, where **Document.Create** receives a delegate that defines one or more Page configurations. Each page declares its size, margins, and a default text style, then partitions content into three

zones: `page.Header()`, `page.Content()`, and `page.Footer()`. Nested layout elements are expressed through chained lambda calls on `Row`, `Column`, `Table`, `Text`, and `Image` containers:

```
// PdfService.cs - document structure declaration (excerpt)
var document = Document.Create(container =>
{
    container.Page(page =>
    {
        page.Size(PageSizes.A4);
        page.MarginVertical(1.5f, Unit.Centimetre);
        page.MarginHorizontal(2, Unit.Centimetre);
        page.DefaultTextStyle(x => x.Font-
Size(10).FontColor(DarkText));

        page.Header().Column(header =>
        {
            // Accent bar (HTL Neufelden brand colour #00a092)
            header.Item().Height(4).Background(AccentColor);

            header.Item().PaddingTop(15).Row(row =>
            {
                // Left column: organisation name and contact details
                row.RelativeItem(3).Column(col => { /* ... */ });
                // Right column: invoice number, date, due date
                row.RelativeItem(2).Column(col => { /* ... */ });
            });
        });

        page.Content().PaddingTop(15).Column(column =>
        {
            // Billing address block, optional HeaderText, item table,
            totals, QR code
        });
    });
});
```

```

        page.Footer().Column(footer =>
        {
            // Divider, ZVR/IBAN/BIC footer line, page numbers
        });
    });
});

return document.GeneratePdf(); // returns byte[]

```

The invoice item table is constructed using `column.Item().Table(...)`. Column definitions are declared with `ColumnsDefinition` using a mix of `ConstantColumn` (fixed pixel width) and `RelativeColumn` (proportional) values. The header row uses a shared `HeaderCellStyle` helper that applies the brand accent colour background and white semibold text; data rows alternate between white and the `LightAccent` colour (`#e6f7f5`) for legibility:

```

// PdfService.cs – alternating row style helper

private static IContainer DataCellStyle(IContainer container, string
    backgroundColor)
{
    return container
        .Background(backgroundColor)
        .PaddingVertical(6)
        .PaddingHorizontal(8)
        .BorderBottom(0.5f)
        .BorderColor(BorderColor);
}

```

The invoice's `TotalAmount` is displayed in a summary row below the item table, styled with the accent colour at a larger font size. Payment information (IBAN, BIC, reference) is rendered in the lower content section alongside the QR code (see chapter 6.5.5). If the invoice carries a `FooterText`, it is rendered centred below the payment block with an italic style, providing a customisable closing statement.

`page.Footer()` renders the Schulverein's ZVR number, bank coordinates, and page numbering (`CurrentPageNumber() / TotalPages()`) at the bottom of every page — including continuation pages for multi-item invoices. The `GenerateBulkInvoicePdfAsync` method applies the same structural approach for consolidated invoices, adding a summary table of referenced individual invoice numbers above the line-item detail section.

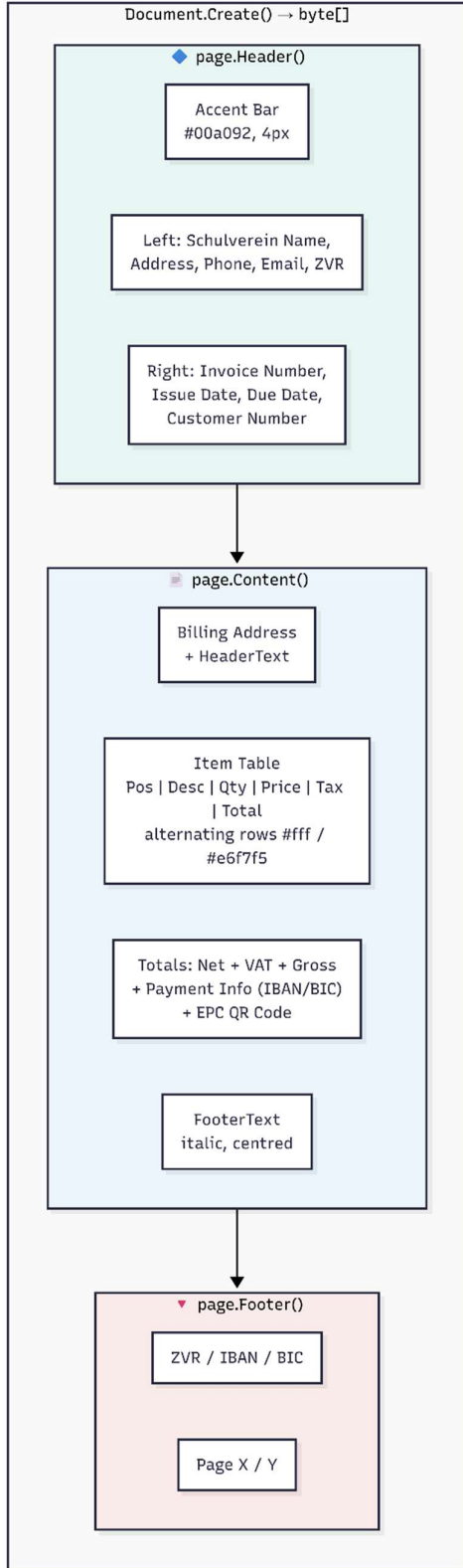


Figure 54: QuestPDF Document Model (Generated by Claude using NOVA Data)

6.5.4 QR-Code Generation

QR codes are embedded in every generated invoice PDF to enable scanning-based SEPA bank transfers. The integration uses the **QRCode** library (QRCode NuGet package) and is encapsulated in **QrCodeService** in **SchulvereinVerwaltungssystem.Services/QrCodeService/QrCodeService.cs**, which implements **IQrCodeService** registered as a scoped service in **ConfigureNovaServices.cs**.

Three generation modes are available:

1. **EPC payment QR code** (**GenerateEpcPaymentQrCodeAsync**) — produces a QR code encoding a SEPA Credit Transfer payload conforming to the European Payments Council EPC069-12 standard (GiroCode). The payload is a structured text block starting with BCD (service tag) and version 002, followed by the BIC, beneficiary name, IBAN, amount, and remittance reference.
2. **Invoice reference QR code** (**GenerateInvoiceReferenceQrCodeAsync**) — encodes a compact reference string of the form INVOICE:{number}|AMOUNT:{amount}|DUE:{date}|PAYEE:{name} for scanning by internal tooling.
3. **Payment link QR code** (**GeneratePaymentLinkQrCodeAsync**) — encodes a URL with query parameters for online payment redirection.

The EPC payload is built by **BuildEpcString**, which assembles the eleven-line block required by the standard:

```
// QrCodeService.cs - EPC SEPA payment string construction
private string BuildEpcString(
    string iban, string bic, string beneficiaryName,
    decimal amount, string purpose, string reference)
{
    var epc = new StringBuilder();
    epc.AppendLine("BCD"); // Service Tag
    epc.AppendLine("002"); // Version
    epc.AppendLine("1"); // Character set (UTF-8)
    epc.AppendLine("SCT"); // SEPA Credit Transfer
    epc.AppendLine(bic); // BIC of beneficiary
    bank
    epc.AppendLine(beneficiaryName); // Beneficiary name
    epc.AppendLine(iban); // Beneficiary IBAN
```

```

        epc.AppendLine($"EUR{amount:F2}");           // Amount in euros
        epc.AppendLine("");                          // Purpose code (optional)
        epc.AppendLine(reference);                   // Structured remittance
        reference
        epc.AppendLine(purpose);                      // Unstructured remittance
        information
        return epc.ToString().TrimEnd();
    }

```

All three modes feed the resulting string into the QRCode pipeline: `QRCodeGenerator.CreateQrCode` generates the internal QR data model at error correction level M, `PngByteQRCode` renders it to a PNG byte array at the specified pixel scale (10 pixels per module for EPC codes, 8 for the other variants), and `ConvertToBase64` encodes the byte array to a Base64 string for JSON serialisation and embedding in the **InvoiceDto**:

```

// QrCodeService.cs – QRCode render pipeline
var qrGenerator = new QRCodeGenerator();
var qrCodeData  = qrGenerator.CreateQrCode(epcString, QRCodeGenerator.ECCLevel.M);
var qrCode      = new PngByteQRCode(qrCodeData);
var qrCodeBytes = qrCode.GetGraphic(10);           // 10px per module
return await Task.FromResult(ConvertToBase64(qrCodeBytes));

```

The IBAN, BIC, and beneficiary name are not hard-coded in **QrCodeService** but injected through `BankSettings`, an options-pattern class read from `appsettings.json` under the `BankSettings` key. This makes the payment details configurable per deployment without code changes. Validation is performed at service construction time: if `BankSettings.IsValid()` returns false, an `InvalidOperationException` is thrown during DI resolution, preventing the application from starting with incomplete bank configuration.

PdfService accesses the QR code through the `InvoiceDto.QrCode` property (already populated as Base64 by the time PDF generation is called) and renders it directly as a raster image in the payment section of the PDF via `QuestPDF's .Image(Convert.FromBase64String(invoice.QrCode))` call. If QR code data is absent or malformed, the rendering block is guarded by a try/catch that logs a warning and omits the image without aborting PDF generation.

`ValidateQrCodeData` provides a lightweight structural check that can be called by callers prior to rendering: EPC strings are validated by checking for the BCD prefix and minimum line count; reference strings by the presence of AMOUNT: and DUE: tokens; and URL-based codes by `Uri.TryCreate` with HTTP scheme assertion.

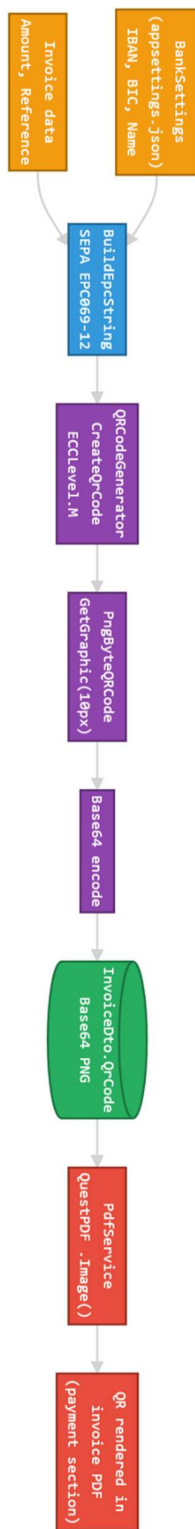


Figure 55: QR Code Generation Pipeline (Generated by Claude using NOVA Data)

7. Frontend

7.1 Introduction

The frontend of NOVA is designed as a high-performance administrative tool. Its primary goal is to replace the legacy *Campai* system with a streamlined, responsive interface that handles complex school association data without the overhead of traditional multi-page applications.

7.2 Technology

The choice of technologies was driven by the need for long-term maintainability and strict data consistency between the client and the .NET backend.

7.2.1 Angular and Typescript

NOVA utilizes Angular as its foundational framework. Unlike "view-only" libraries, Angular's "batteries-included" nature provides standardized solutions for routing, forms, and dependency injection. This was critical for ensuring that different team members produced consistent code across the Member, Invoice, and Form modules.

TypeScript is used to enforce type safety. By defining shared interfaces that mirror the backend DTOs (Data Transfer Objects), the team eliminated a common category of web development bugs: property name mismatches.

7.2.2 Reactive State with RxJS

Data flow within the application is handled reactively using RxJS. Instead of static variables, data is treated as asynchronous streams (Observables). This allows for efficient UI updates—for example, automatically refreshing the member list after a successful import without requiring a manual page reload.

7.2.3 Angular Material and Styling

To address the usability shortcomings of the previous *Campai* software, NOVA utilizes Angular Material 3 for its UI component library. This ensures that the application provides a modern, familiar, and highly accessible interface for administrative staff. Standardized components such as data tables, buttons, and modal dialogs are used throughout the application to provide a cohesive user experience.

The styling is implemented using SCSS, which allows for a sophisticated theming system. A central custom-theme.scss file is used to define the application's color palette, typography, and component-specific styles. By leveraging the Material 3 theming mixins, the application can maintain a consistent visual identity while being easily adaptable to future design

requirements. This modular approach to styling ensures that the CSS remains maintainable even as the application grows in complexity.

7.3 Design Patterns and Architecture

To maintain a clean separation of concerns and prevent the frontend from becoming a monolithic mess, several software engineering patterns were applied.

7.3.1 The Facade Design Pattern (State Management)

To manage application state without the complexity of a full NgRx implementation, NOVA employs the Facade pattern. Facades act as an intermediary abstraction layer between complex services and the actual UI components.

- Logic Flow:
 1. A UI Component (e.g., `MemberTableComponent`) subscribes *only* to the Facade (e.g., `MemberFacade`) via RxJS Observables.
 2. The Facade coordinates with the underlying Data Services (e.g., `MemberService`) to dispatch API calls.
 3. When the API returns data, the Facade updates its internal state, automatically pushing the update to all subscribed components via the RxJS stream.
- Justification: This provides a Single Source of Truth for data and decouples the view logic from the underlying communication protocols. This means UI components remain lightweight and completely agnostic about *how* data is fetched, significantly improving testability and code maintainability.

7.3.2 Modularization and Lazy Loading

The application structure strictly isolates feature modules from the core runtime.

- Core Module: Houses "singleton" services that must exist only once per session (e.g., `AuthService`, global error interceptors).
- Shared Module: Contains generic, context-agnostic UI elements used throughout the app (custom tables, date pipes, buttons).
- Feature Modules: (e.g., `InvoicingModule`, `FormBuilderModule`). These modules are Lazy Loaded via the Angular Router. The browser only downloads the bundle for the complex *Form Builder* when the user actually navigates to that section.
- Justification: This dramatically reduces the initial bundle size, ensuring that the critical "Time to Interactive" (TTI) for administrative staff remains optimized even as new feature modules are added to the system.

The physical organization of these modules is visualized in the project structure diagram below.

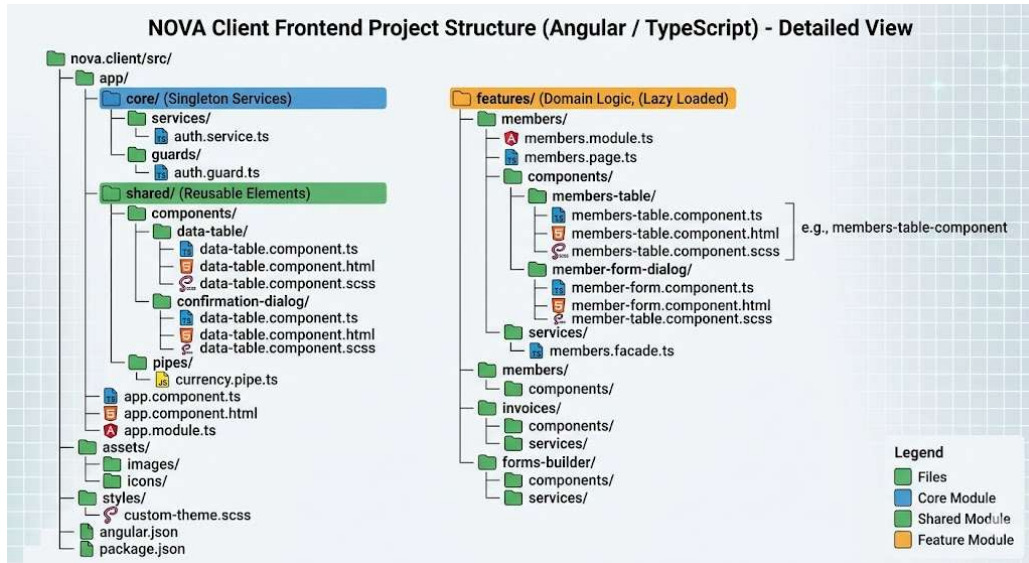


Figure 56: Graph displaying NOVA physical structure (Generated by Geminis Nano Banana 2 using NOVA Data)

7.4 Data Handling and Security

The bridge between the Angular client and the ASP.NET Core API is built on a "Service-Facade" pattern.

All HTTP communication is handled by specialized services using Angular's `HttpClient`. To ensure security, an `AuthInterceptor` is used to modify outgoing requests.

```
export function authenticationInterceptor(req: HttpRequest<unknown>, next: HttpHandlerFn): Observable<HttpEvent<unknown>> {
    const token = localStorage.getItem('authenticationJWT');

    if (token) {
        const authReq = req.clone({
            setHeaders: {
                Authorization: `Bearer ${token}`
            }
        });
    };

    return next(authReq);
}
```

```
}  
  
return next(req);  
  
}
```

7.4.1 Route Guards

To prevent unauthorized access to administrative views, **Route Guards** are implemented. If a user attempts to access `/admin/members` without a valid JWT, the AuthGuard cancels the navigation and redirects the user to the login page.

```
export class AuthGuard implements CanActivate {  
  constructor(  
    private authenticationFacade: AuthenticationFacade,  
    private router: Router  
  ) {}  
  
  canActivate(): boolean | UrlTree {  
    if (this.authenticationFacade.isLoggedIn()) {  
      return true;  
    }  
  
    return this.router.createUrlTree(['/login']);  
  }  
}
```

7.5 Routing and Navigation

The navigation structure of NOVA is designed to provide a seamless user experience while maintaining strict security standards. Angular's powerful routing engine is utilized to map browser URLs to specific functional components, allowing for a structured and intuitive flow through the application.

7.5.1 Global Routing Configuration

The central configuration for the application's navigation is defined in the `app-routing.module.ts`. This module acts as the "traffic controller" of the frontend, determining which component should be rendered based on the current path.

To ensure high performance and fast initial load times, NOVA heavily utilizes Lazy Loading. Instead of loading the entire application code at once, the global router only loads the core configuration. Feature-specific modules, such as the Invoicing or Member Management modules, are only fetched from the server when the user explicitly navigates to those sections. This significantly reduces the initial bundle size and improves the "Time to Interactive" (TTI) for the user.

7.5.2 Public vs. Protected Routes

A fundamental requirement for a school association management system is the strict separation between public areas and restricted administrative functions. NOVA implements a clear distinction between these two zones:

- **Public Routes:** These routes are accessible to any user without authentication. They include the Login Page, Password Reset requests, and public-facing Data Collection Forms. These pages are designed to be lightweight and accessible from any device.
- **Protected Routes:** All administrative modules—including the Dashboard, Member Lists, and Financial Reports—are considered protected. These routes are guarded by Angular Route Guards (CanActivate).

When a user attempts to access a protected route, the guard intercepts the request and checks the current authentication state via the `AuthService`. If no valid JSON Web Token (JWT) is found or the token has expired, the guard automatically cancels the navigation and redirects the user to the login page. This programmatic security layer ensures that internal association data remains confidential and inaccessible to unauthorized individuals.

7.6 Authentication and Authorization

Security is an important concern for the NOVA system, as it manages sensitive personal and financial data of the school association. The frontend implements an authentication and authorization system to ensure that only verified administrators can access the management modules.

7.6.1 NOVA Identity Integration

The frontend integrates seamlessly with the backend's identity system, which is based on JSON Web Tokens (JWT). When a user logs in via the authentication interface, the credentials are sent to the .NET API. Upon successful verification, the server returns a JWT containing the user's identity and assigned roles.

The NOVA client manages this identity through several specialized mechanisms:

- **Token Storage:** To maintain a persistent session across browser refreshes, the received JWT is securely stored in the browser's Local Storage.

Key	Value
authenticationJWT	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...

Figure 57: Screenshot of Local Storage containing token

- **Authentication State:** A central `AuthService` acts as the single source of truth for the user's current status. It provides an observable state that components can subscribe to—for example, to conditionally show or hide navigation elements based on whether a user is logged in.
- **Token Decoding:** The frontend utilizes libraries to decode the JWT payload on the fly. This allows the application to instantly retrieve the user's name and permissions without requiring an additional round-trip to the server.

7.6.2 Route Guards and Interceptors

To enforce the security policies defined by the identity integration, the application employs two powerful Angular features: Route Guards and HTTP Interceptors.

- **Route Guards (`AuthGuard`):** These act as a digital "gatekeeper" for the application's navigation. Before a user is allowed to navigate to a protected page (like the *Accounting* or *Member List* modules), the `AuthGuard` checks the `AuthService` for a valid, non-expired token. If the check fails, the navigation is cancelled, and the user is automatically redirected to the login page.
- **HTTP Interceptors (`AuthInterceptor`):** While guards protect the UI, interceptors protect the data. The `AuthInterceptor` is a middleware component that intercepts every outgoing HTTP request to the backend. It automatically injects the stored JWT into the Authorization header using the Bearer scheme. This ensures that every API call is authenticated by default, eliminating the need for developers to manually attach tokens to individual service calls and significantly reducing the risk of security gaps.

▼ Request Headers ☒ Raw	
Accept	application/json, text/plain, */*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-AT;q=0.9,de-AT;q=0.8,de;q=0.7,en-GB;q=0.6,en-US;q=0.5
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJvc2VySUQiOiIwYzIxOTIhN0YzQ2LTRiZTgtYjdiOC05MTQ0YWl0ZjZmODQlLCJ1bmlxdWVfbmFtZSI6I6lnNjaHVsdmVyZWluQW4uYXQlLCJ1bWVfbmFtZSI6I6lnNjaHVsdmVyZWluQW4uYXQlLCJvc2VyUm9sZSI6I6lkFkbWluliwibmJmljoxNzc0NDI1OTgzLCJleHAiOiE3NzQ0MzE5ODMsImhhdCI6MTc3NDQyNTk4M30.waFKeU8lZtINiFC7j96pbjMDZdlF21gWq0QQ94IL5Pw

Figure 58: Screenshot of HTTP-Request with injected Authorization Token (Bearer-Scheme)

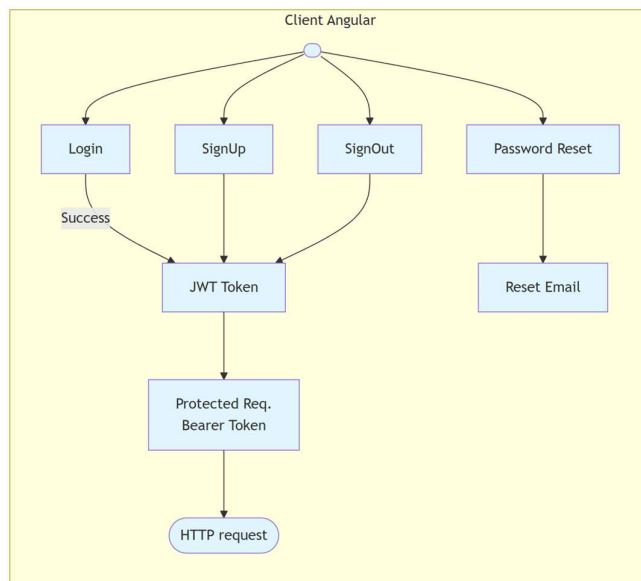


Figure 59: Authentication Flow - Frontend

7.7 Core Feature Implementation

This section details the technical implementation of the system's most critical features.

7.7.1 Unified Entity Management

Member, Contact, and Debtor management are handled within a single route using mat-tabs. This allows administrators to view a member's personal data and financial status simultaneously. Relationships between these entities are managed via Angular Material Dialogs, which allow for "Entity Linking" (e.g., assigning an existing contact profile to a new member record).

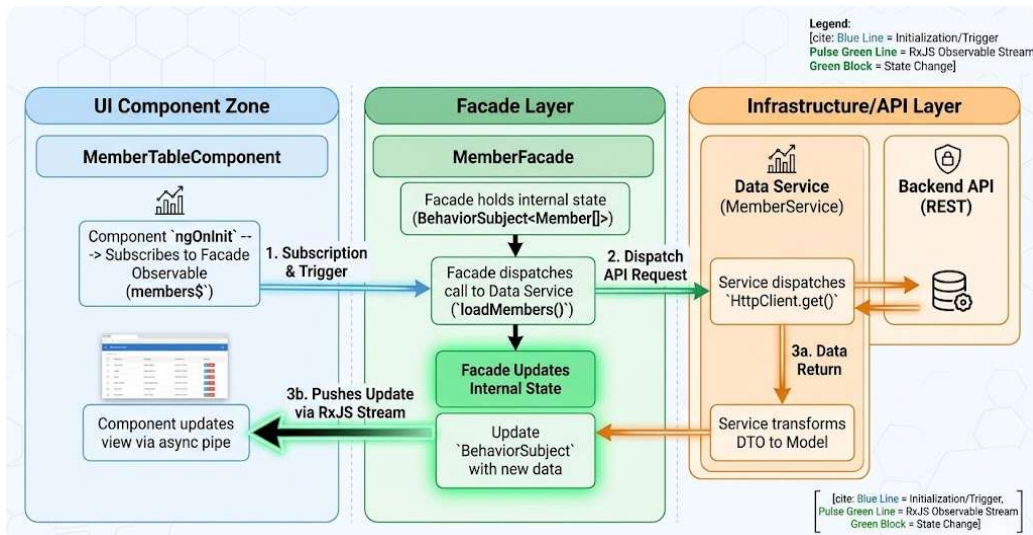


Figure 60: Logic flow for populating/updating components in angular (Generated with Gemini Nano Banana 2 using NOVA data)

7.7.2 Table Personalization and Persistent Tables

A major "Quality of Life" feature in NOVA is the advanced table management. Each table allows for dynamic sorting and custom column ordering.

To improve efficiency, the system remembers the user's preferred layout. This is achieved by serializing the column state and storing it in the browser's **LocalStorage**. When the user returns to the member list, the component retrieves this state during the `ngOnInit` lifecycle hook.

```
// Example: Persisting column order in LocalStorage

saveTableSettings(columns: string[]) {
  const settingsKey = 'member_table_config';
  localStorage.setItem(settingsKey, JSON.stringify(columns));
}

loadTableSettings(): string[] {
  const saved = localStorage.getItem('member_table_config');
  return saved ? JSON.parse(saved) : this.defaultColumns;
}
```

7.7.3 Dynamic Form Builder UI

The Form Builder is the most complex frontend module. It allows administrators to create custom data collection forms. Technically, this is implemented as a Dynamic Reactive Form.

The UI generates a JSON schema based on the user's "drag-and-drop" actions, which is then sent to the backend for storage and later rendered for public users.

7.7.4 Invoice Management and PDF Generation

The financial module automates the association's billing process by bridging the gap between database records and official documents. When an administrator initiates the generation of a single or bulk invoice, the frontend sends a request to the .NET backend, which utilizes iText7 to assemble the document. The resulting binary stream is then rendered directly in the browser. This allows for immediate verification of totals and formatting before the document is finalized or dispatched via email.



Figure 61: Image of Invoice Generated by the System

7.7.1 Dynamic Email System and Templates

To streamline communication, NOVA features a template-based email system. The frontend integrates a custom form control to allow for personalization of email templates for example for invoices, form submission confirmations, etc. A critical technical detail is the implementation of dynamic placeholders (e.g., `{{firstName}}`). During the dispatch phase, the frontend sends the raw HTML and the recipient IDs to the backend, which performs the final string interpolation before sending the mail via the association's SMTP server.

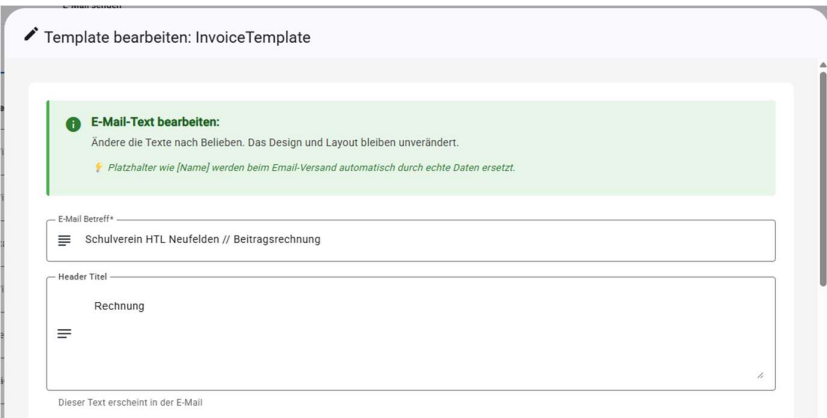


Figure 62: Screenshot of email-editor (only upper partial as to illustrate the general layout)

7.7.1 Product and Fee Management

Product management serves as the configuration layer for the association's revenue streams (e.g., annual membership fees). This module is technically linked to the Debtor and Member entities. From a UI perspective, this is handled through a reactive catalog where products can be assigned to members with specific price overrides. This relationship is managed through a complex Reactive FormArray, allowing administrators to add multiple products to a member profile in a single transaction, which the backend then persists as relational entries for future automated billing cycles.

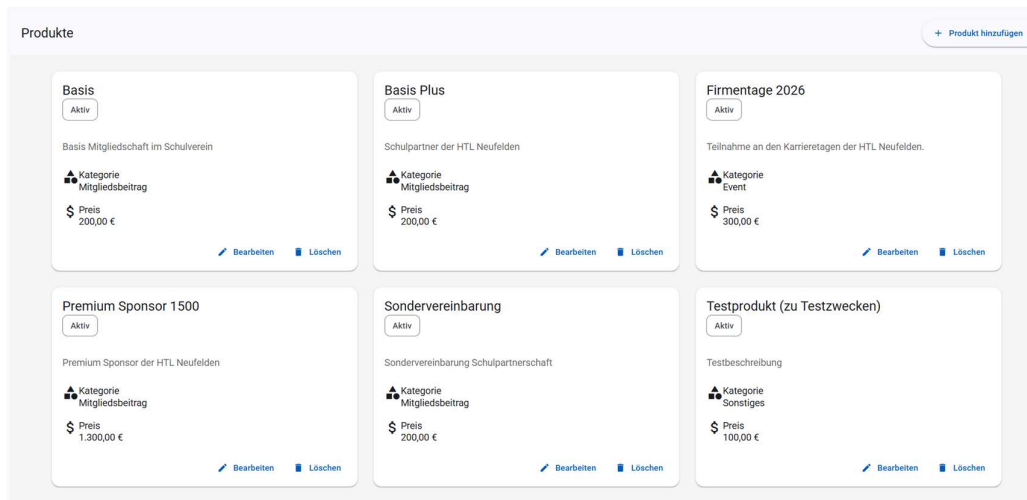


Figure 63: Screenshot of Product-Selection Page from the NOVA System

7.8 General UI Layout and Interface Patterns

The user interface of the NOVA client is designed to maximize administrative efficiency. Leveraging the architectural capabilities of a Single-Page Application (SPA), the layout ensures that primary navigation remains persistent while only the main content area dynamic updates, providing a seamless workflow that mitigates context-switching overhead.

7.8.1 Three Viewing Zone Skeleton Layout

To provide a consistent and predictable user experience, the system implements a standardized three-zone layout strategy across the entire application viewport:

- **Zone 1: Persistent Navigation Sidebar:** The sidebar acts as the central router control. It provides direct, single-click access to all primary feature modules (Members, Invoicing,

Forms, Events). To optimize screen real estate during complex data management tasks, the sidebar is implement as collapsible, shrinking to an icon-only view upon user interaction.

- **Zone 2: Contextual Topbar:** The topbar provides immediate situational awareness. It displays the active module title (e.g., "Member Management") and hosts global action elements, such as the user profile dropdown and logout functionality. This zone is designed to ensure that critical session status is always visible.
- **Zone 3: Dynamic Content Surface:** This is the primary workspace, housed within an Angular Material Card container. This area dynamically renders the components associated with the active route. By adhering to a consistent card layout, data remains visually organized and computationally separate from the navigation logic.

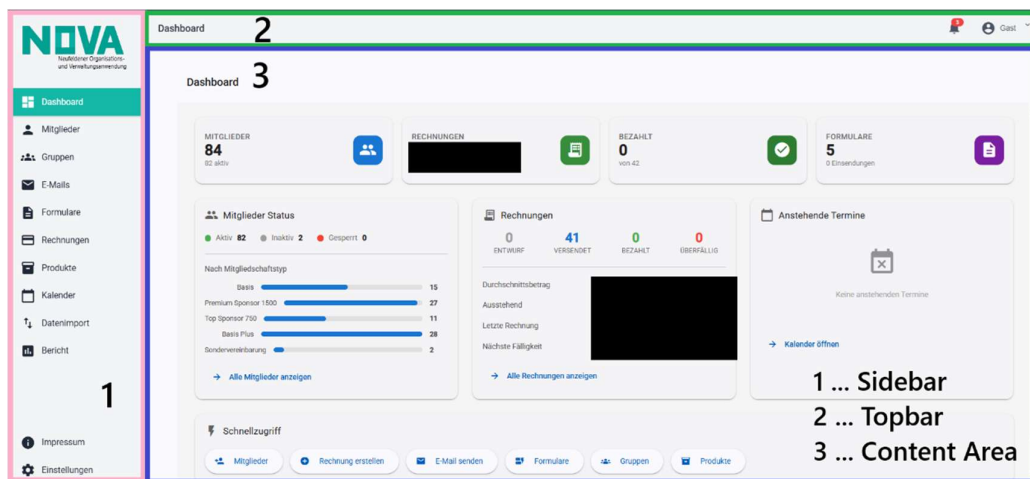


Figure 64: Screenshot of Dashboard with Areas of Interest outlined. (Redacted)

8. Deployment

The deployment strategy for NOVA underwent a significant transformation during the project lifecycle. Initially, the system relied on manual deployment processes typical of legacy environments. However, to increase development velocity and ensure a reliable production state, the team transitioned to a modern, container-based CI/CD (Continuous Integration/Continuous Deployment) pipeline.

8.1 Evolution of the Deployment Strategy

8.1.1 Legacy Deployment: Manual IIS Provisioning

In the early stages of the project, the application was hosted on the school's internal servers using **Internet Information Services (IIS)**. This approach presented several operational challenges:

- **Administrative Bottlenecks:** Access to the production server was strictly limited to the school's system administrator. Any update or configuration change required a TeamViewer session facilitated by the admin, leading to significant delays.
- **Manual Build Errors:** The process of manually building the .NET and Angular projects and transferring the binaries was prone to "human error," where environment-specific configurations could easily be mismatched.
- **Lack of Autonomy:** The development team could not verify deployment success in real-time, making hotfixes and rapid iterations nearly impossible.

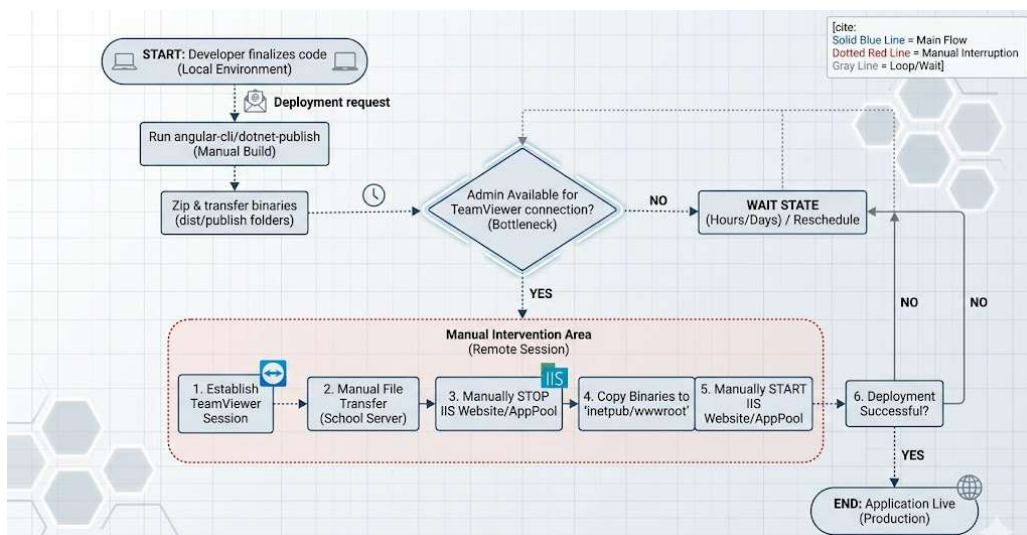


Figure 65: Flowchart Depicting Legacy Deployment Process (Generated with Gemini Nano Banana 2 using NOVA Data)

8.1.2 8.1.2 Transition to Containerization (Portainer)

To address these inefficiencies, the school proposed a new infrastructure standard for diploma projects based on Portainer, a container management platform. This shift required the NOVA architecture to be fully containerized (as detailed in the earlier chapters on Docker).

The transition to a container-based environment offered immediate benefits:

- **Environment Parity:** The application runs in the exact same Docker container on the school server as it does on the developer's local machine.
- **Isolation:** The database and web services are isolated from the host OS, preventing conflicts with other school projects.
- **Self-Service Management:** Using the Portainer web interface, the team regained the ability to restart services, view logs, and monitor resource usage without constant administrative intervention.

8.2 CI/CD Pipeline Implementation

To fully leverage the new infrastructure, a automated CI/CD pipeline was implemented. This ensures that the production server is always running the latest, verified version of the code without manual effort.

8.2.1 Pipeline Workflow

The pipeline, triggered by every "push" to the main branch of the GitHub repository, follows a strict sequence of automated stages:

1. **Continuous Integration (Build & Test):** The runner environment pulls the latest source code. It executes the .NET test suite and Angular linter to ensure that no "breaking changes" are introduced.
2. **Containerization:** Upon successful testing, the pipeline uses the project's Dockerfiles to build new production-ready images for both the frontend and the backend.
3. **Image Distribution (Docker Hub):** These images are tagged with a version number and pushed to a private repository on **Docker Hub**. This acts as the central registry for the project's artifacts.
4. **Continuous Deployment:** The school's Portainer instance is configured to monitor the Docker Hub registry. When a new image is detected, the stack is automatically updated, pulling the latest image and replacing the old containers with zero-to-minimal downtime.

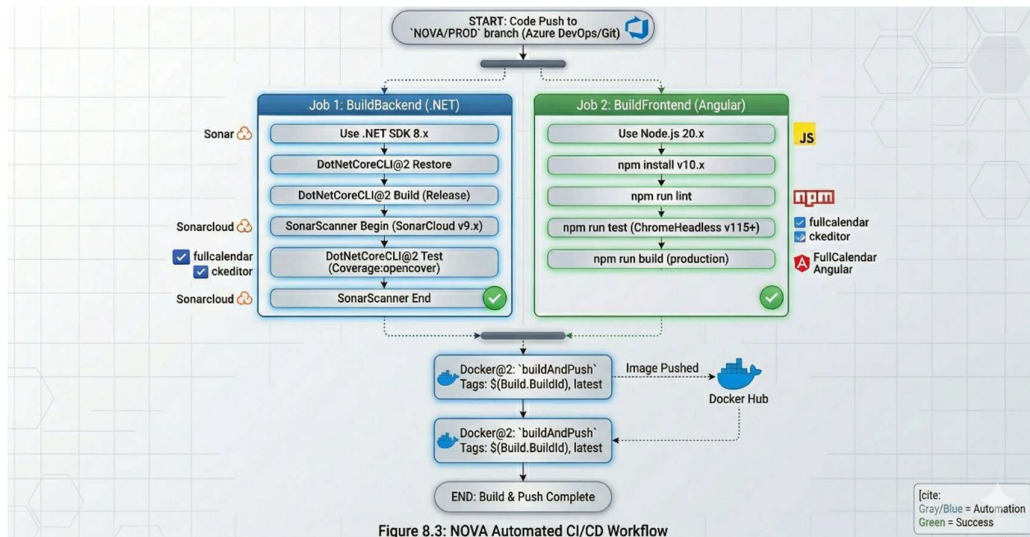


Figure 66: Flowchart of Improved Deployment Process (Generated with Gemini Nano Banana 2 using NOVA Data)

8.3 Conclusion on Infrastructure

By shifting from a manual IIS setup to a containerized CI/CD workflow, the NOVA project achieved a professional-grade deployment standard. This modernization did more than just save time; it established a "fail-safe" environment where code is automatically tested before reaching production. For the development team, this meant shifting focus away from server administration and back toward feature development, resulting in a more stable and frequently updated system for the school association.

9. Results and Outlook

The NOVA project reaches its first production milestone as a functionally complete, multi-module web application deployed to the internal server infrastructure of HTL Neufelden at `nova.htl-neufelden.at`. Over the course of eleven months — from the initial requirements gathering in April 2025 to the final revision cycles in March 2026 — the system evolved from a rough set of competing priorities into a coherent platform covering member management, email campaigns, dynamic forms, invoice generation, calendar integration, and GDPR compliance. This chapter evaluates the outcome against the original brief: section 9.1 measures the system in quantitative and architectural terms; section 9.2 traces the evolution of the client relationship through the recorded meeting history; section 9.3 assesses the automated test suite and its integration into the CI pipeline; and section 9.4 identifies concrete areas for improvement, drawing on recurring themes from the project protocols.

9.1 Benchmarks

Scope and Structural Metrics

The delivered system comprises 16 API controllers, 31 domain entities managed through the `DbContext` (see chapter 5.2), and 28 EF Core migrations capturing the full schema evolution from the initial Identity tables to the bulk invoice and invoice template subsystems introduced in the later development phases. The service layer in `SchulvereinVerwaltungssystem.Services` contains 83 source files, including 19 interface/implementation pairs for the core business logic (e.g., `IMemberServices / MemberService`, `IInvoiceService / InvoiceService`, `IFormServices / FormServices`) and auxiliary implementations for email composition, export formatting, and entity mapping extensions (see chapter 6.2.2).

The test project (`SchulvereinVerwaltungssystem.Tests`) contains 45 test files distributed across four categories: repository tests, service tests, controller tests, and extension method tests. Of the approximately 253 test method declarations present in the project, 194 are active; the remaining 59 are commented out, primarily within `InvoiceServiceTests` (disabled in full) and isolated methods in `EmailServiceTests` (see chapter 9.3 for a detailed assessment).

Deployment Target and Concurrency Profile

NOVA is deployed on a Windows Server host within the HTL Neufelden network, accessible at `nova.htl-neufelden.at` over HTTPS via IIS. The production database runs on the iWeb4 SQL Server instance using Windows Integrated Security (see chapter 5.6.2). The expected operational load is five to ten simultaneous users, consistent with the staffing structure of the Schulverein HTL Neufelden. No virtualisation or container orchestration is used in production; the Docker Compose configuration (see chapter 5.4.1) serves exclusively as a portable development and testing environment.

At this concurrency level, the sequential aggregation strategy in `DashboardService` (see chapter 6.5.2) and the per-request EF Core `DbContext` lifetime are architecturally sufficient without introducing caching or connection pooling beyond the defaults provided by the SQL

Server driver. The Serilog file sink retains 30 days of rolling log history capped at 10 MB per file (see chapter 6.2.6), providing adequate operational visibility for a small administrative team without requiring a dedicated log aggregation system.

Architectural Quality Indicators

The strict four-layer dependency graph — NOVA.Server → Services → Infrastructure / Core, with Core carrying zero project references — ensures that domain models and service contracts are testable in isolation and that any layer can be replaced without cascading changes. The consistent application of the Soft Delete pattern across eight entity types (see chapter 5.2), the centralised exception-to-HTTP mapping in `ExceptionHandlerMiddleware` (see chapter 6.2.6), and the options-pattern configuration for SMTP credentials and bank settings collectively keep environment-specific concerns out of business logic. These structural properties remain sound regardless of the user volume; they would also support a controlled scale-out if the Schulverein's operational needs were to grow beyond the current profile.

9.2 Feedback from Client

Early Phase: Requirements Gathering and Shifting Priorities

The project was initiated on 4 April 2025 with a brief from the client (Dipl.-Ing. Rainhard Prieschl) to replace the existing Campai membership management platform with a web-based tool tailored to the Schulverein's operational needs. The initial requirements conversation on 7–8 April 2025 produced a broad wish list spanning member management, email campaigns, dynamic forms, invoice generation, and calendar integration, but without a defined priority order between modules. An important clarification from that meeting — that invoicing should be treated as a dependent downstream feature rather than the starting point — shaped the early implementation sequence.

Throughout April 2025, the team assembled the technical foundation: project structure, solution layout, and the working name NOVA (Neufeldner Organisations- und Verwaltungs-Anwendung). Wireframing in Figma was completed over late June and early July 2025, and a team responsibility shuffle on 16 July 2025 realigned backend and frontend ownership. This structural instability in the early months meant that the first functional deliverables did not reach the client for review until October 2025, a lag that the project protocols attribute to the breadth of the initial scope rather than to any single technical obstacle.

Mid Phase: Structured Feedback and Iterative Refinement

By the time of the first substantive client review session on 7 October 2025, a working prototype was available and the feedback became significantly more specific. The client identified a series of concrete usability requirements: member and debtor number formatting (a year-prefixed sequential scheme), configurable membership product names in Settings, confirmation dialogs for destructive actions, and a firm deadline for the dynamic forms module — the Firmentag 2026 invitation forms were required to be operational by 1 November 2025. The meeting notes from this session also introduced a recurring pattern: the client's feedback arrived in dense clusters, mixing bug reports, design preferences, and new feature requests

within the same conversation. Managing this without derailing the implementation schedule required the team to triage requests against the agreed delivery target.

The biweekly cadence of client meetings — established by agreement at this point and maintained through to March 2026 — provided the structural feedback loop that the project had lacked in its opening months. Meetings on 23 October, 11 November, 17 November, and 24 November 2025 each yielded a prioritised correction list that the team resolved before the next session. Deployment to the production server at nova.htl-neufelden.at was coordinated with Dipl.-Ing. Werner Furtmüller on 13 October 2025 and executed in a deploy-test-refine cycle: the team published a build, the client tested it against real operational data, and corrections were applied over the following weekend.

The 11 November 2025 meeting formalised several operational details that had been deferred — the official email address (schulverein@htl-neufelden.at), the legal name for all outward-facing documents ("Schulverein HTL Neufelden"), and the Gmail application password for SMTP relay. These details, while not technically complex, had visible downstream effects: invoices and emails sent before this meeting carried incorrect sender identity and required manual correction.

Problems and Resolution

The most significant operational failure recorded during the project occurred on 3 December 2025, when a bulk email dispatch to 84 recipients resulted in only 20 to 21 confirmed deliveries. The error surface presented to the client was a generic "Fehler" message with no delivery count, no per-recipient status, and no retry path. The 5 December meeting placed this failure as the primary agenda item and produced a concrete requirement list: per-campaign delivery statistics, a scrollable submission view, Excel export of form submissions, and more informative error messaging throughout the application. These requirements were addressed in the following two weeks.

The 24 November 2025 meeting stands in contrast: the client explicitly expressed satisfaction with the state of the system and confirmed that the Firmentag invitations would be dispatched that same week. This milestone — a live, client-approved form campaign reaching external recipients — represented the first end-to-end operational use of NOVA and validated the core form, email, and submission pipeline in production conditions.

Subsequent meetings in December 2025 and January–March 2026 continued to refine the invoice builder, email template management, and member-contact data model. The tone of these sessions shifted from reactive bug-fixing to incremental improvement, reflecting a client who had integrated NOVA into daily operations and was identifying enhancement opportunities from practical use rather than abstract requirements.

9.3 Unit testing

Framework and Organisation

The automated test suite is implemented in **SchulvereinVerwaltungssystem.Tests** using xUnit as the test runner, Moq for mock object creation in service and controller tests, FluentAssertions for expressive assertion syntax, and the EF Core InMemory provider for repository tests that require a functioning **DatabaseContext** without a live SQL Server instance. The project is organised into four test categories, each mirroring a corresponding layer of the application:

- **RepositoryTests/** — nine test classes covering **MemberRepository**, **InvoiceRepository**, **GroupRepository**, **FormRepository**, **DebtorRepository**, **AuditLogRepository**, **SettingRepository**, **DocumentRepository**, and **CalendarEventRepository**. This category contains 57 active test methods, which verify CRUD operations, soft delete behaviour, and specialised queries against an in-memory database.
- **ServiceTests/** — test classes for **AuthService** (sign-up, sign-in, sign-out, password reset, token verification), **EmailService**, **GroupService**, **MemberService**, **InvoiceService**, **SettingsService**, **TokenService**, and **GoogleCalendarService**. This category accounts for 85 active test methods and uses mocked repository interfaces to isolate business logic from data access.
- **ControllerTests/** — six test classes (Invoice, Auth, Calendar, CalendarTwoWaySync, Settings, Form) containing 73 active test methods. Controller tests use mocked service interfaces and an injected ClaimsPrincipal to simulate authenticated HTTP calls, asserting on the HTTP result type and response body without requiring a running web server.
- **ExtentionTests/** — nine test classes covering the entity-to-DTO mapping extensions in **SchulvereinVerwaltungssystem.Services/Extentions/** (see chapter 6.2.4), totaling 38 active test methods.

Representative Test Patterns

Chapter 6.2.5 presents the repository test pattern in full, including the in-memory **DatabaseContext** setup and the **IDisposable** teardown that guarantees per-class database isolation. The controller test pattern, not covered in that chapter, follows the same structural convention but substitutes mocked services for a live context. The **InvoicesControllerTests** class illustrates the approach:

```
// InvoicesControllerTests.cs — controller test with mocked service and injected user context
public class InvoicesControllerTests
{
    private readonly Mock<IInvoiceService> _mockInvoiceService;
    private readonly Mock<IEmailServices> _mockEmailService;
    private readonly Mock<IPdfService> _mockPdfService;
    private readonly InvoiceController _controller;

    public InvoicesControllerTests()
    {
```

```

        _mockInvoiceService = new Mock<IInvoiceService>();
        _mockEmailService = new Mock<IEmailServices>();
        _mockPdfService = new Mock<IPdfService>();
        _controller = new InvoiceController(
            _mockInvoiceService.Object,
            _mockPdfService.Object,
            _mockEmailService.Object,
            new Mock<ILogger<InvoiceController>>().Object);
    }

    private void SetupUserContext(string userId = "test-user-id")
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.NameIdentifier, userId),
            new Claim(ClaimTypes.Email, "test@example.com")
        };
        _controller.ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext
            {
                User = new ClaimsPrincipal(new ClaimsIdentity(claims, "TestAuth"))
            }
        };
    }
}

```

SetupUserContext injects a synthetic ClaimsPrincipal directly into the controller's ControllerContext, enabling tests of endpoints that extract the user identity from HttpContext.User without a full authentication middleware stack.

Coverage and Limitations

The 194 active test methods cover the primary CRUD paths and a selection of error paths for the most critical domain areas. Repository tests are the most systematically complete category: all nine specialised repository classes have corresponding test files that exercise create, retrieve, update, and soft-delete operations. Service test coverage is uneven: **AuthService** and **GroupService** are tested across multiple files including separate exception and success sub-directories, while **InvoiceService** test coverage is absent — the **InvoiceServiceTests** class exists but its content is entirely commented out, leaving the invoice number generation logic, TotalAmount calculation, and bulk-generation path without automated verification. The **EmailServiceTests** file is similarly incomplete, with its primary test method disabled.

The CI pipeline definition in **azure-pipelines.yml** configures the test stage with the project pattern ****/*UnitTests.csproj**. The actual test project file is named **SchulvereinVerwaltungssystem.Tests.csproj**, which does not match this glob. As a consequence, the dotnet test step

in the **BuildAndTest** stage does not execute the test suite on the build agent. SonarCloud code coverage analysis therefore receives no coverage data from the pipeline, and coverage-gated quality gates are not enforced. This configuration discrepancy is the single most consequential gap in the quality assurance setup.

9.4 Points for Improvement

The following areas for improvement are drawn from recurring themes in the client meeting protocols and from the architectural assessment of the codebase. They are presented as future development opportunities rather than deficiencies in the delivered system.

Invoice Workflow Completeness

The invoice status model, as noted in the 10 March 2026 meeting, supports mark-paid as the only user-initiated state transition and treats it as irreversible. A practical improvement would be to expand `InvoiceStatus` transitions to allow cancellation and reinstatement, and to introduce a configurable overdue detection threshold. The bulk generation feature, requested on the same date, is partially implemented via `GenerateBulkInvoicesAsync` in **InvoiceBuilderService** (see chapter 6.4.8), but the client requirement to filter bulk generation by membership product type — creating annual invoices only for Premium members, for instance — was identified as missing. The email dispatch dialog for sending invoices to debtors, also raised in January 2026, represents a high-visibility gap: the current flow requires the user to navigate out of the invoice view to compose an email, rather than triggering a send dialog inline.

Email Template Visibility and Management

The 10 March 2026 meeting noted that email templates are displayed in the frontend using their raw database names, which are in English and do not indicate which application feature uses each template. A straightforward improvement would be to introduce a `DisplayName` and `UsageContext` field on **EmailTemplate**, populated during seeding, so that the settings page can show administrators which template governs invitation emails, payment reminders, and form submission confirmations respectively.

User Interface Persistence and Drag-and-Drop

Column visibility and sort order preferences for the member, contact, debtor, and invoice table views are stored locally per browser session. The 17 November 2025 and 10 March 2026 meetings both raised the expectation that these preferences should persist across devices — either via a user-specific settings entry in the **Settings** table or through a server-side preference store. The product and form card ordering, introduced as a drag-and-drop requirement in March 2026, similarly relies on `localStorage`, which is neither synchronised across devices nor backed up.

Test Coverage and CI Integration

The most immediate corrective action for the test suite is to align the Azure Pipelines glob pattern with the actual project file name, changing `**/*UnitTests.csproj` to

******/SchulvereinVerwaltungssystem.Tests.csproj. This single change would activate SonarCloud coverage reporting and bring the CI gate into effect. Beyond the configuration fix, the disabled **InvoiceServiceTests** and **EmailServiceTests** should be completed: the invoice number generation algorithm and the bulk generation logic contain non-trivial state-dependent behaviour that is currently unverified. Extending service test coverage to **DebtorService**, **FormServices**, **BulkInvoiceService**, and **ImportService** would bring the most operationally critical paths under automated regression protection.

Database Backup and Recovery

Database backup strategy was raised by the client on 17 November 2025 in the context of form submission data loss risk. The current production setup relies on the default SQL Server recovery model configured by the school's infrastructure team, without application-level backup triggers or automated export jobs. For a system that stores legally relevant financial records and GDPR consent histories, a scheduled backup job — even a nightly BACKUP DATABASE to a network share — would materially reduce recovery risk.

Mobile Support and Accessibility

The original requirements specification (7 April 2025) explicitly excluded mobile device support, and the current Angular UI is built for desktop viewport widths. As NOVA moves from an internal administration tool to a platform accessed intermittently by a broader set of Schulverein stakeholders, a responsive layout pass on the high-traffic pages (member list, invoice view, dashboard) would reduce friction without requiring a full design overhaul. Formal WCAG 2.2 conformance was also deferred in the initial brief; the European Accessibility Act obligations that apply to public-sector web services by 2025 warrant a structured accessibility review before any planned public-facing feature additions.

Two-Factor Authentication

The 11 November 2025 meeting noted 2FA as an open question, with no implementation decision recorded. The current authentication model uses JWT bearer tokens issued by **AuthService** (see chapter 6.3). ASP.NET Core Identity's built-in TOTP support (ITwoFactorTokenProvider<NovaUser>) would enable a standard authenticator-app second factor without replacing the existing authentication flow, making this a self-contained addition with limited cross-cutting impact.

Caching and Cloud Migration

At the current user scale, in-process caching of the **Settings** table — already loaded at startup via **SettingsRuntimeService** — is the only caching concern with measurable effect. Should the system be extended to serve a wider user base, an IMemoryCache layer on the most frequent read-only queries (product catalogue, group list, dashboard statistics) would reduce database round-trips without introducing distributed cache infrastructure. A migration from the on-premises SQL Server instance to Azure SQL Database would simplify the backup, high-availability, and credential management concerns raised throughout the project, at the cost of a dependency on external cloud services — a trade-off that the school's infrastructure policy would need to evaluate.

10. Glossary

Technical vocabulary with description here.

A

Angular – *“Angular is an open-source web application framework maintained by Google and a community of developers. It is designed to build dynamic and interactive single-page applications (SPAs) efficiently.”* (Geeks For Geeks, 2026).

API – *“An API, or application programming interface, is a set of rules or protocols that enables software applications to communicate with each other to exchange data, features and functionality.”* (IBM, 2026)

ASP.NET Core – *“ASP.NET Core is a cross-platform, high-performance, open-source framework for building modern web apps using .NET.”* (Microsoft, 2026)

B

Bearer Scheme – *“Bearer authentication (also called token authentication) is an HTTP authentication that involves security tokens called bearer tokens.”* (Swagger, 2026)

C

C# – *“C# is a general-purpose, object-oriented programming language with applications across many areas in software development.”* (Jalli, 2026)

CRUD – *“CRUD is an acronym from the world of computer programming and refers to the four basic operations necessary to implement a persistent storage application: create, read, update and delete.”* (sumo logic, 2026)

Controllers – *“A Controller is a special class in the ASP.NET Core Application with .cs (for C# language) extension. In the ASP.NET Core MVC Application, the controller class must be inherited from the Controller base class. In ASP.NET Core MVC, the Controller base class provides many properties, methods, and features that handle HTTP requests and produce responses in our application. It provides Action Result Methods, Model State Management, Validation, TempData, ViewBag, ViewData, etc.”* (Dot Net Tutorials, 2026)

D

Database Context – *“The central component of EF Core functions as a bridge between the domain (entity classes) and the database. It represents the database session and provides an abstraction for querying and saving data.”* (Devart, 2026)

Docker / Containerization – *“Docker Containerization is a technology that enables creating, deploying, and running applications in isolated environments called containers. Containerization using Docker allows running applications with all their dependencies in a portable and efficient manner, regardless of the environment in which they are run.”* (EITT, 2026)

E

Entity – *“Entity classes are plain C# classes (POCO - Plain Old CLR Objects) that represent the application's data structure. They map to database tables, and their properties map to table columns.”* (Devart, 2026)

Entity Framework Core (EF Core) – *“Entity Framework (EF) Core is a lightweight, extensible, open source and cross-platform version of the popular Entity Framework data access technology.”* (Microsoft, 2026)

Environment Variables – *“An environment variable is a variable whose value is set outside the program, typically through functionality built into the operating system or microservice. An environment variable is made up of a name/value pair, and any number may be created and available for reference at a point in time.”* (Medlock, 2026)

F

Figma – *“Figma is a powerful design tool that lets you create and collaborate on user interfaces and prototypes for websites, apps, and much more. Unlike traditional design software that requires installation, Figma runs entirely in your web browser, making it accessible on any platform, whether you're using Windows, Mac, or Linux. This platform independence allows everyone to access and use it easily.”* (Geeks For Geeks, 2026).

H

HTL (Höhere Technische Lehranstalt) – An Austrian higher technical college that provides secondary vocational education with a strong focus on engineering and applied sciences.

HTTP – *“HTTP (HyperText Transfer Protocol) is an application-layer protocol used for transferring data over the web.”* (Geeks For Geeks, 2026)

HTTPS – *“HTTPS stands for Hyper Text Transfer Protocol Secure. HTTP Secure (HTTPS), could be a combination of the Hypertext Transfer Protocol with the SSL/TLS convention to supply encrypted communication and secure distinguishing proof of an arranged web server.”* (Geeks For Geeks, 2026)

JSON – *"JSON (JavaScript Object Notation) is a text-based format for storing and exchanging data in a way that's both human-readable and machine-parsable."* (Oracle, 2026)

JWT – *"JSON Web Token (JWT) is a compact, URL-safe means of representing claims to be transferred between two parties."* (RFC, 2026)

M

MSSQL – *"Microsoft SQL Server is a relational database management system (RDBMS). Applications and tools connect to a SQL Server instance or database, and communicate using Transact-SQL (T-SQL)."* (Microsoft, 2026)

N

NuGet – *"An essential tool for any modern development platform is a mechanism through which developers can create, share, and consume useful code"* (Microsoft, 2025)

.NET – *".NET is an open-source platform for building desktop, web, and mobile applications that can run natively on any operating system. The .NET system includes tools, libraries, and languages that support modern, scalable, and high-performance software development. An active developer community maintains and supports the .NET platform."* (Amazon AWS)

R

RBAC (Role-Based Access Control) – *"Role-based access control (RBAC) is a model for authorizing end-user access to systems, applications and data based on a user's predefined role."* (IBM, 2026)

REST (Representation State Transfer) – *"A REST API is an application programming interface API that conforms to the design principles of the representational state transfer (REST) architectural style, a style used to connect distributed hypermedia systems. REST APIs are sometimes referred to as RESTful APIs or RESTful web APIs."* (IBM, 2026)

Repository Pattern – *"The Repository pattern is a Domain-Driven Design pattern intended to keep persistence concerns outside of the system's domain model. One or more persistence abstractions - interfaces - are defined in the domain model, and these abstractions have implementations in the form of persistence-specific adapters defined elsewhere in the application."* (Microsoft, 2023)

S

SPA – “An SPA (Single-page application) is a web app implementation that loads only a single web document, and then updates the body content of that single document via JavaScript APIs such as Fetch when different content is to be shown.” (Mozilla, 2026)

Serilog – “[...] - a powerful, extensible, and highly performant logging library designed specifically for .NET applications. With its rich ecosystem of sinks, enrichers, and support for structured logging, Serilog has become the go-to choice for developers building robust and maintainable systems in ASP.NET Core.” (Suthar, 2025)

T

TypeScript – “TypeScript is a programming language developed by Microsoft that adds static typing to JavaScript, making it more robust, maintainable, and easier to scale. It is an open-source language and has gained immense popularity among developers due to its seamless integration with existing JavaScript projects and its numerous features that enhance the development experience. ” (Williams, 2026)

TOTP – “From a user’s perspective, TOTP works extremely simple. It involves three components: (1) An authenticator app on your smartphone; (2) a shared key that only your app and the server know; (3) a six-digit code, or token, that is regenerated every 30 seconds.” (Hendrik, 2024)

U

UI (User Interface) – “The user interface (UI) is the point of human-computer interaction and communication in a device. This can include display screens, keyboards, a mouse and the appearance of a desktop.” (Hashemi-Pour, 2026)

UX (User Experience) – “User experience, often shortened to UX, refers to the overall experience a customer or user has when interacting with a product, system or service.” (IBM, 2026)

V

Version Control – “Version control, also known as source control, is the practice of tracking and managing changes to software code. Version control systems are software tools that help software teams manage changes to source code over time. ” (Atlassian, 2026)

W

Wireframe – “Wireframing is a way to design a website service at the structural level. A wireframe is commonly used to layout content and functionality on a page which takes into account user needs and user journeys. ” (ExperienceUX, 2026)

WCAG – “Web Content Accessibility Guidelines (WCAG) 2 is developed through the W3C process in cooperation with individuals and organizations around the world, with a goal of providing a single shared standard for web content accessibility that meets the needs of individuals, organizations, and governments internationally.” (Henry, 2025)

X

xUnit – “xUnit.net is a free, open source, community-focused unit testing tool for C#, F#, and Visual Basic. xUnit.net v3 supports .NET 8.0 or later, and .NET Framework 4.7.2 or later.” (xUnit, 2026)

11. Table Collection

NuGet Packages with Versions, Purpose and Licenses

Table 2: NuGet Packages with Versions, Purpose and Licenses (Refined by Perplexity.ai using existing table/ NOVA's data)

Package	Version(s)	Purpose	License
ClosedXML	0.105.0	Excel file manipulation	MIT
EPPlus	8.2.1; 7.4.2	Spreadsheet (.xlsx) library	Polyform Noncommercial
Coverlet.collector	6.0.0	Code coverage collection	MIT
DnsClient	1.8.0	DNS resolution and queries	Apache-2.0
FluentAssertions	8.6.0	Fluent assertions for TDD/BDD testing	Proprietary (Non-comm.)
Google.Apis.Auth	1.68.0	Google API authentication	Apache-2.0
Google.Apis.Calendar.v3	1.68.0.3536	Google Calendar API client	Apache-2.0
Google.Apis.Core	1.68.0	Core utilities for Google APIs	Apache-2.0
Humanizer.Core	2.14.1	Human-readable string formatting	MIT

Package	Version(s)	Purpose	License
MailKit	4.13.0	Email sending/receiving (SMTP/IMAP)	MIT
Microsoft.AspNetCore.Authentication.JwtBearer	8.0.18	JWT token authentication middleware	MIT
Microsoft.AspNetCore.Identity.EntityFrameworkCore	8.0.18	Identity management with EF Core	MIT
Microsoft.AspNetCore.SpaProxy	8.0.18	Proxy for SPA dev server	MIT
Microsoft.EntityFrameworkCore	9.0.10; 8.0.18	ORM for database operations	MIT
Microsoft.EntityFrameworkCore.Design	9.0.10	Design-time EF tools	MIT
Microsoft.EntityFrameworkCore.InMemory	9.0.0	In-memory DB provider (testing)	MIT
Microsoft.EntityFrameworkCore.SqlServer	9.0.10	SQL Server EF provider	MIT
Microsoft.EntityFrameworkCore.Tools	9.0.7	EF migrations and scaffolding	MIT
Microsoft.NET.Test.Sdk	17.8.0	.NET test SDK and runner	MS .NET Library
Microsoft.VisualStudio.Azure.Containers.Tools.Targets	1.23.0	Azure container build tools	MS Proprietary

Package	Version(s)	Purpose	License
Moq	4.20.72	Mocking framework for unit tests	BSD-3-Clause
QrCoder	1.6.0	QR code generation	MIT
QuestPDF	1.6.0	PDF document generation	Community MIT
Serilog.AspNetCore	9.0.0	Structured logging for ASP.NET Core	Apache-2.0
Serilog.Sinks.Console	6.1.1	Console sink for Serilog	Apache-2.0
Serilog.Sinks.File	7.0.0	File sink for Serilog	Apache-2.0
Swashbuckle.AspNetCore	6.6.2	Swagger/OpenAPI documentation	MIT
xunit	2.5.3	xUnit testing framework	Apache-2.0
xunit.runner.visualstudio	2.5.3	VS test runner for xUnit	Apache-2.0

Angular Depedencies

Table 3: Angular Dependencies (Generated with Perplexity.ai using NOVA's data)

Package	Version	Purpose	License
@angular/core	^20.0.0	Core framework: lifecycle, DI, change detection	MIT

Package	Version	Purpose	License
@angular/common	^20.0.0	Pipes, directives (NgIf/NgFor), HTTP client	MIT
@angular/forms	^20.0.0	Reactive/template-driven forms with validation	MIT
@angular/router	^20.0.0	Client-side routing and navigation	MIT
@angular/platform-browser	^20.0.0	Browser rendering platform	MIT
@angular/material	^20.1.1	Material Design UI (tables, dialogs, buttons)	MIT
@angular/cdk	^20.1.1	Low-level primitives (overlays, a11y, drag-drop)	MIT
rxjs	~7.8.0	Async data streams via Observables	Apache-2.0
ngx-quill	^26.0.0	Angular wrapper for Quill rich-text editor	MIT
quill	^2.0.2	Modular rich-text editor	BSD-3-Clause
@tailwindcss/postcss	^4.1.11	Tailwind CSS PostCSS integration	MIT
tslib	^2.3.0	TypeScript runtime helpers	BSD Zero Clause
zone.js	~0.15.0	Change detection execution context	MIT

License Analysis

Table 4: License Analysis (Refined by Perplexity.ai using NOVA's data)

License Name	Use Case	License URL
MIT	Free use/mod/distribution (attribution optional)	MIT
Apache-2.0	Free use with attribution/patent grants	Apache-2.0
BSD-3-Clause	Free use with attribution/no endorsement	BSD-3-Clause
OBSD	Free use (public domain equivalent)	OBSD
Microsoft .NET Library	Free install/use/redistribute in apps	MS-NET-Lib
Microsoft Proprietary	Free use (similar to MIT for containers/tools)	MS-Prop
QuestPDF Community (MIT)	Free for non-profits/orgs <\$1M revenue	QuestPDF
FluentAssertions Proprietary	Free for non-commercial use (paid for comm.)	FluentAssertions
Polyform Noncommercial 1.0.0	Free for non-commercial/personal projects	EPPlus

12. List of Figures

Figure 1: Campai Logo	11
Figure 2: NOVA-Logo	11
Figure 3: Task Overview.....	13
Figure 4: System Architecture Overview (<i>Generated by Perplexity with NOVA's data</i>).....	16
Figure 5: Database overview (Generated by Gemini's Nano Banana 2 using NOVA-Data) ...	17
Figure 6: Wireframe of NOVA-Dashboard.....	19
Figure 7: Wireframe of NOVA-Members-Page.....	20
Figure 8: Wireframe of NOVA-Login-Page.....	20
Figure 9: NOVA-Form-Builder	21
Figure 10: Gantt Chart – Timeline (<i>Generated by Perplexity with NOVA's data</i>)	24
Figure 11: Tech Stack (Generated by Perplexity.ai with NOVA's data)	25
Figure 12: Project- Solution in Visual Studio	26
Figure 13: Repository Pattern (The Blue Book, 2026)	30
Figure 14: Service Layer Pattern (LinkedIn, 2026)	31
Figure 15: Facade Pattern (Peiretti, 2026).....	32
Figure 16: Data Transfer Objects Pattern (Baeldung, 2026).....	33
Figure 17: Middleware Pattern (Microsoft, 2026).....	33
Figure 18: Template Method Pattern (Heise, 2026).....	34
Figure 19: Strategy Pattern (Dofactory, 2026)	35
Figure 20: Dependency Injection (Sanjay, 2026)	35
Figure 21: Entity Relationship Diagram (Generated by Claude using NOVA Data)	42
Figure 22: Person domain (Generated by Claude using NOVA Data).....	48
Figure 23: Finance domain (Generated by Claude using NOVA Data)	50
Figure 24: Form domain (Generated by Claude using NOVA Data)	53
Figure 25: Domain Grouping Overview (Generated by Claude using NOVA Data)	56
Figure 26: Flowchart Displaying Database Startup Process (Generated with Geminis Nano Banana 2 using NOVA Data)	58
Figure 27: Startup Sequence (Generated by Claude using NOVA Data)	61
Figure 28: Controller Layer Interaction Diagram (Generated by Claude using NOVA Data) ..	68
Figure 29: Test Project Structure (Generated by Claude using NOVA Data).....	75
Figure 30: Middleware Pipeline Order (Generated by Claude using NOVA Data)	79
Figure 31: Backend Workflow - Part 1	80
Figure 32: Backend Workflow - Part 2	81
Figure 33: JWT Structure (Geeks For Geeks, 2026)	83
Figure 34: Authentication Flow – Backend	84
Figure 35: Password Reset Flow	87
Figure 36: Member Object States.....	89
Figure 37: Member System Flow.....	91
Figure 38: Product Management Flow (Generated by Claude using NOVA Data)	96
Figure 39: Form Entity Hierarchy.....	98
Figure 40: FormController	100
Figure 41: Form Posting Flow	102

Figure 42: Form Submission Flow	103
Figure 43: Submission Export - Part1.....	106
Figure 44: Submission Export - Part2.....	106
Figure 45: Invoice Construction Flow (Generated by Claude using NOVA Data)	112
Figure 46: Email System Overview	113
Figure 47:Email Template Flow (Generated by Claude using NOVA data).....	116
Figure 48: Email with Template.....	117
Figure 49:Email System Flow	120
Figure 50: Template Instantiation Flow (Generated by Claude using NOVA Data).....	123
Figure 51: Settings System Overview	127
Figure 52: Dashboard Service Flow (Generated by Claude using NOVA Data)	139
Figure 53:Export System Overview.....	140
Figure 54: QuestPDF Document Model (Generated by Claude using NOVA Data).....	148
Figure 55: QR Code Generation Pipeline (Generated by Claude using NOVA Data).....	150
Figure 56: Graph displaying NOVA physical structure (Generated by Geminis Nano Banana 2 using NOVA Data)	153
Figure 57: Screenshot of Local Storage containing token	156
Figure 58: Screenshot of HTTP-Request with injected Authorization Token (Bearer-Scheme)	157
Figure 59: Authentication Flow - Frontend	157
Figure 60: Logic flow for populating/updating components in angular (Generated with Geminis Nano Banana 2 using NOVA data).....	158
Figure 61: Image of Invoice Generated by the System.....	159
Figure 62: Screenshot of email-editor (only upper partial as to illustrate the general layout)	159
Figure 63: Screenshot of Product-Selection Page from the NOVA System	160
Figure 64: Screenshot of Dashboard with Areas of Interest outlined. (Redacted)	161
Figure 65: Flowchart Depicting Legacy Deployment Process (Generated with Geminis Nano Banana 2 using NOVA Data)	162
Figure 66: Flowchart of Improved Deployment Process (Generated with Geminis Nano Banana 2 using NOVA Data)	164

13. List of Tables

Table 1: Project Responsibilities	27
Table 2: NuGet Packages with Versions, Purpose and Licenses (Refined by Perplexity.ai using existing table/ NOVA's data)	177
Table 3: Angular Dependencies (Generated with Perplexity.ai using NOVA's data).....	179
Table 4: License Analysis (Refined by Perplexity.ai using NOVA's data)	181

14. Prompt Directory

Prompts submitted to Perplexity AI

Mermaid Diagram — Group Feature

Prompt: [Generate a metaprompt to generate a Mermaid diagram for the GitHub repository "NOVA". The Mermaid should visualize NOVA's "Group" feature in the backend. The Mermaid structure should be slim, to fit documentation format.](#)

Response Summary: A comprehensive metaprompt was crafted for a software documentation expert specializing in C# .NET backend architecture. The resulting Mermaid diagram visualizes the Group feature of the NOVA backend — a C# .NET school association management system (Schulverein) — in a slim, documentation-ready format fitting 1–2 A4 pages.

Used in: Chapter 1.1.1 — Group Management System (Mermaid diagram)

Documentation Points — Formular System

Prompt: [Vaguely suggest points to write for the formular system of the backend of the GitHub repository "NOVA" — Chapter 1.1.1 Formular System.](#)

Response Summary: Based on the NOVA repository, the AI identified and described the core formular entity hierarchy: a Form as the top-level entity with metadata fields (Name, Description, CreatedAt, CreatedBy, form image, MaxSubmissionCount), section and field structures, and submission logic.

Used in: Chapter 1.1.1 — Formular System (content ideation and structure)

Mermaid Diagrams — Form Module

Prompt: [You are a technical documentation engineer. Your task is to generate concise, clean Mermaid diagrams for the NOVA backend — a C# ASP.NET project for school association management. The diagrams must document the Form module.](#)

Response Summary: Four Mermaid diagrams were generated covering the Form module: (1) Core Entity Data Model (Form, FormSection, FormField, etc.), (2) Submission Flow, (3) Publishing Lifecycle, and (4) Relationship overview. The diagrams were produced based on uploaded project documentation files.

Used in: Chapter 1.1.1 — Formular System (Mermaid diagrams)

Chapter Introductions — Email, Settings, and Group Systems

Prompt: [Formulate brief introductions for the GitHub project "NOVA" for the following diploma chapters: 1.1.1 Email System with Templates, 1.1.1 Settings System, 1.1.1 Group Management.](#)

Response Summary: Short, precise introductions were generated for each subsystem. NOVA's email system is built on MailKit (SMTP), enabling administrators to send individual and group emails using configurable templates. The settings system and group management module introductions were similarly derived, a C# web application with an ASP.NET Core backend and Angular frontend built as a diploma project for HTL Neufelden.

Used roughly in: Chapters 1.1.1 Email System, 1.1.1 Settings System, 1.1.1 Group Management (section introductions)

Additional prompts and conversations related to the NOVA project can be found in the dedicated Perplexity Space: [NOVA — Perplexity Space](#)

Other submitted Prompts

Prompt: "Here is a description of a legacy deployment process, please generate a technical flowchart diagram based on it."

Response Summary: "Generated a technical flowchart diagram visualizing the manual legacy deployment process." Used Model: "Gemini 3 Flash Image"

Prompt: "Here is an Azure DevOps YAML pipeline script, please generate a detailed flowchart diagram visualizing the parallel jobs, and add a highlighted manual Portainer update step at the end."

Response Summary: "Generated a comprehensive flowchart visualizing the automated CI/CD pipeline and manual Portainer deployment gate." Used Model: "Gemini 3 Flash Image"

Prompt: "Here is a request to document a flowchart, please explain the visual structure in great detail to create a generation script, and then generate the image anew based on that context."

Response Summary: "Created a detailed textual description of the flowchart's structure and generated a new image based on those specifications." Used Model: "Gemini 3.1 Pro & Gemini 3 Flash Image"

Prompt: "Here are visual adjustments for the pipeline flowchart (change GitHub to Azure Repos, remove the figure label at the bottom, ensure high resolution), please regenerate the image."

Response Summary: "Applied the requested visual adjustments and regenerated the high-resolution flowchart image." Used Model: "Gemini 3 Flash Image"

Prompt: "Here are new constraints for the CI/CD pipeline (remove Grafana, remove Infrastructure-as-Code, end the flow directly after the Docker push), please write a new context script and generate the updated flowchart."

Response Summary: "Updated the structural script to reflect the simplified pipeline and generated the corresponding flowchart image." Used Model: "Gemini 3.1 Pro & Gemini 3 Flash Image"

Prompt: "Here is a draft glossary and thesis chapters containing quotes, please find and update the research sources by providing an author, release date, and specific URL for each citation."

Response Summary: "Researched and provided formatted citations (Author, Date, URL) for the glossary terms based on the provided text." Used Model: "Gemini 3.1 Pro"

Prompt: "Here are the official guidelines and requirements for a diploma thesis poster, please provide the exact text content and structural layout so it can be used directly in a layout program or as a prompt for an AI."

Response Summary: "Structured a detailed layout template and content guide for the A1 diploma thesis poster." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a request for poster layout advice, please suggest which UI screenshots and technical diagrams should be included, and provide short project descriptions/pitches."

Response Summary: "Recommended specific UI screenshots and technical diagrams, and provided two variations of a short project pitch." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a request for the poster's 'Task Description' (Aufgabenstellung) and 'Project Goals', please formulate professional yet easily understandable texts for a general audience."

Response Summary: "Drafted professional, easy-to-understand texts for the poster's task description and project goals in two length variations." Used Model: "Gemini 3.1 Pro"

Prompt: "Here are the drafted goals and solutions for the poster, please shorten them significantly so they are readable in passing."

Response Summary: "Condensed the project goals and solutions into highly readable, bulleted summaries suitable for a poster." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a request regarding poster screenshots (already possessing the main dashboard), please suggest 1-2 additional features and 1 technical graphic to complete the poster."

Response Summary: "Suggested specific application features (e.g., Live PDF, Form Builder) and technical graphics (CI/CD pipeline) to highlight on the poster." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is an introductory text about the NOVA system, please condense it and make it less technical for the poster."

Response Summary: "Provided three shortened, non-technical variations of the NOVA system introduction for quick reading." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a drafted sentence referring readers to the glossary for high-level technical terminology, please refactor it into a formal academic tone."

Response Summary: "Refactored the sentence into several formal, academic options suitable for a thesis." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a paragraph describing the team's initial division of labor and their later transition into full-stack development, please refine it for a professional thesis."

Response Summary: "Refined the paragraph to professionally describe the team's role transition and upskilling process." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a sentence introducing a table of NuGet packages along with an AI usage attribution, please refine it so it flows seamlessly between the main text and the table caption."

Response Summary: "Separated the AI attribution from the main text and integrated it smoothly into the formal table caption." Used Model: "Gemini 3.1 Pro"

Prompt: "Here is a request to generate a realistic image displaying a potential poster mounted on a wall."

Response Summary: "Explained the inability to generate the requested physical mockup image and offered alternative text-based assistance." Used Model: "Gemini 3.1 Pro"

Prompts submitted to Claude (Claude Opus 4.6)

Prompt: "Generate a high-level Mermaid entity relationship diagram showing all NOVA entities grouped by domain with cross-domain foreign key relationships." Response Summary: Produced a graph LR diagram with nine coloured domain subgraphs and solid/dashed arrows for required and optional foreign key relationships. Used in: Figure 21 — Entity Relationship Diagram

Prompt: "Generate a Mermaid erDiagram for the NOVA Person domain showing Member, Contact, AdditionalEmail, and PhoneNumber with their fields and relationships." Response Summary: Generated an erDiagram with field-level detail, cardinalities, and the exclusive-ownership check constraint on AdditionalEmail and PhoneNumber. Used in: Figure 22 — Person domain

Prompt: "Generate a Mermaid erDiagram for the NOVA Finance domain covering Debtor, Invoice, InvoiceItem, BulkInvoice, Product, InvoiceTemplate, and PartnershipContract." Response Summary: Generated an erDiagram with all finance entities, the BulkInvoice join table, and template-to-product references. Used in: Figure 23 — Finance domain

Prompt: "Generate a Mermaid erDiagram for the NOVA Form domain showing the Form, FormSection, FormField, FormSubmission hierarchy with their fields." Response Summary: Generated an erDiagram visualising the strict parent-child hierarchy from Form down through FormSubmissionValue. Used in: Figure 24 — Form domain

Prompt: "Generate a Mermaid overview of all 31 NOVA DbSet entities grouped by their nine functional domains. Layout as a 3x3 grid where each domain is a single box containing its entity names." Response Summary: Generated a graph TB with nine large coloured nodes, each listing its entities, arranged as a 3x3 grid using invisible links. Used in: Figure 25 — Domain Grouping Overview

Prompt: "Generate a Mermaid sequenceDiagram showing the NOVA application startup sequence — service registration, database connection check, migration, and pipeline

activation in Program.cs." Response Summary: Generated a sequenceDiagram with three coloured phases covering DI registration, InitializeDatabaseAsync with CanConnectAsync and MigrateAsync, and final pipeline activation. Used in: Figure 27 — Startup Sequence

Prompt: "Generate a Mermaid sequenceDiagram showing the NOVA controller layer interaction — HTTP request flow through middleware, controller, service, and repository, plus an error path via ExceptionHandlerMiddleware." Response Summary: Generated a sequenceDiagram with a successful request path and a separate exception path showing 404 propagation through the middleware. Used in: Figure 28 — Controller Layer Interaction Diagram

Prompt: "Generate a Mermaid diagram of the NOVA test project structure showing RepositoryTests and ServiceTests directories, the frameworks used (xUnit, Moq, FluentAssertions, EF Core InMemory), and Azure Pipelines CI integration." Response Summary: Generated a coloured flowchart grouping the seven repository test classes, eight service test sub-directories, testing frameworks, and CI stages. Used in: Figure 29 — Test Project Structure

Prompt: "Generate a Mermaid diagram of the NOVA middleware pipeline order including Serilog request logging, CORS, authentication, authorization, and ExceptionHandlerMiddleware, with the full exception-to-HTTP-status-code mapping." Response Summary: Generated a top-down flowchart showing the ordered middleware stages and a sub-block mapping each custom exception type to its corresponding HTTP status code. Used in: Figure 30 — Middleware Pipeline Order

Prompt: "Generate a compact Mermaid flowchart showing the product management flow — creation validation, persistence, usage in invoice items, and lifecycle operations (soft delete, restore, hard delete)." Response Summary: Generated a left-to-right flowchart with eight phase-coloured nodes covering validation, creation, usage paths, and lifecycle operations. Used in: Figure 38 — Product Management Flow

Prompt: "Generate a Mermaid flowchart of the NOVA invoice construction flow — CreateInvoiceDto, product resolution, InvoiceItem aggregation, TotalAmount calculation, QR code generation, and downstream actions (PDF, email, preview)." Response Summary: Generated a top-down flowchart tracing the transformation from DTO to persisted invoice with QR code and follow-up output paths. Used in: Figure 45 — Invoice Construction Flow

Prompt: "Generate a Mermaid flowchart of the NOVA invoice template instantiation flow showing InvoiceTemplate and TemplateItems, placeholder dictionary construction, ReplacePlaceholders application, CreateInvoiceDto assembly, and InvoiceService delegation." Response Summary: Generated a flowchart visualising placeholder resolution and the template-to-concrete-invoice transformation, including the bulk generation loop. Used in: Figure 50 — Template Instantiation Flow

Prompt: "Generate a Mermaid diagram of the NOVA DashboardService aggregation flow — showing how MemberStatistics, InvoiceStatistics, FormStatistics, and UpcomingEvents are collected from four separate sources and merged into a single DashboardDto." Response

Summary: Generated a flowchart showing the four-domain aggregation converging into a single DTO returned to the Angular client. Used in: Figure 52 — Dashboard Service Flow

Prompt: "Generate a compact Mermaid diagram of the NOVA QuestPDF document model — page header, content, and footer zones with the invoice table structure, totals, payment info, and QR code placement." Response Summary: Generated a diagram with three horizontally arranged page zones, each summarising the key layout elements inline. Used in: Figure 54 — QuestPDF Document Model

Prompt: "Generate a linear Mermaid flowchart of the NOVA QR code generation pipeline — BankSettings, EPC SEPA string construction, QRCode render, Base64 encoding, InvoiceDto storage, and PdfService embedding." Response Summary: Generated a left-to-right flow of nine phase-coloured nodes tracing the full pipeline from configuration input to PDF embedding. Used in: Figure 55 — QR Code Generation Pipeline

Prompt: "Here are my draft chapters for the diploma thesis. Please review them against the established NOVA writing style — formal academic tone, third person, technological justification followed by architectural patterns and implementation details with code snippets. Correct grammar and phrasing, tighten sentence structure, and refine the technical terminology where needed. Keep the content and intent intact."

Response Summary: Reviewed the author's draft chapters and returned refined versions with corrected grammar, tightened phrasing, and consistent academic tone. Suggested improvements to sentence structure and terminology while preserving the original content, code references, and argumentative structure.

Used in: Chapters 5, 5.1, 5.2, 5.6, 6.2, 6.4, 6.5, 9 (text refinement and proofreading)

15. Bibliography

- Amazon AWS. (n.d.). *Amazon AWS*. Retrieved from Amazon AWS.
- Atlassian. (2026, March 20). *What is version control?* Retrieved from What is version control?: <https://www.atlassian.com/git/tutorials/what-is-version-control>
- Baeldung. (2026, March). *The DTO Pattern (Data Transfer Object)*. Retrieved from <https://www.baeldung.com/java-dto-pattern>
- campai. (2026, 3 11). *campai*. Retrieved from campai: <https://www.campai.com/>
- ClosedXML. (2026, March 27). *ClosedXML*. Retrieved from ClosedXML: <https://docs.closedxml.io/en/latest/>
- Devart. (2026, March 20). *What is Entity Framework Core?* Retrieved from What is Entity Framework Core?: <https://www.devart.com/dotconnect/what-is-entity-framework-core.html>
- DevOps, A. (2026, 03 30). Retrieved from <https://azure.microsoft.com/en-us/products/devops>
- Discord. (2026). *Discord*. Retrieved from <https://discord.com/>
- Dofactory. (2026, March). *C# Chain of Responsibility Design Pattern*. Retrieved from <https://www.dofactory.com/net/chain-of-responsibility-design-pattern>
- Dofactory. (2026, March). *C# Strategy Design Pattern*. Retrieved from <https://www.dofactory.com/net/strategy-design-pattern>
- Dot Net Tutorials. (2026, March 20). *Controllers in ASP.NET Core MVC*. Retrieved from Controllers in ASP.NET Core MVC: <https://dotnettutorials.net/lesson/controllers-asp-net-core-mvc/>
- DOTNET Foundation. (2026, March 27). *MailKit - Working with Emails*. Retrieved from MailKit - Working with Emails: <https://dotnetfoundation.org/news-events/detail/mailkit-working-with-emails>
- DOTNET Foundation. (2026, March 27). *MailKit - Working with Emails*. Retrieved from MailKit - Working with Emails: <https://dotnetfoundation.org/news-events/detail/mailkit-working-with-emails>
- EITT. (2026, March). *Docker Containerization*. Retrieved from <https://eitt.academy/glossary/docker-containerization/>
- ExperienceUX. (2026, March 20). *What is wireframing?* Retrieved from What is wireframing?: <https://www.experienceux.co.uk/faqs/what-is-wireframing/>
- Geeks For Geeks. (2026, March 20). *Difference between http:// and https://*. Retrieved from Difference between http:// and https://: <https://www.geeksforgeeks.org/computer-networks/difference-between-http-and-https/>

Geeks For Geeks. (2026, March 22). *JSON Web Token (JWT)*. Retrieved from JSON Web Token (JWT): <https://www.geeksforgeeks.org/web-tech/json-web-token-jwt/>

Geeks For Geeks. (2026, March 20). *What is Angular?* Retrieved from What is Angular?: <https://www.geeksforgeeks.org/angular-js/what-is-angular/>

Geeks For Geeks. (2026, April 7). *What is Figma?* Retrieved from What is Figma?: <https://www.geeksforgeeks.org/websites-apps/what-is-figma/>

geralt. (2020). *Pixabay*. Retrieved 06 08, 2020, from <https://pixabay.com/photos/baby-learn-laptop-question-2709666/>

Hashemi-Pour, C. (2026, March 20). *user interface (UI)*. Retrieved from user interface (UI): <https://www.techtarget.com/searchapparchitecture/definition/user-interface-UI>

Heise. (2026, March). *Patterns in der Softwareentwicklung: Die Template-Methode*. Retrieved from <https://www.heise.de/blog/Patterns-in-der-Softwareentwicklung-Die-Template-Methode-7365112.html>

Hendrik. (2024, 06 2). *Hendrik* . Retrieved from Hendrik : <https://www.hendrik-erz.de/post/understanding-totp-two-factor-authentication-eli5>

Henry, S. L. (2025, 10 20). *w3*. Retrieved from w3: <https://www.w3.org/WAI/standards-guidelines/wcag/>

HTL-Neufelden-Calendar. (n.d.). Retrieved from <https://calendar.google.com/calendar/embed?src=verwaltung%40atn.ac.at&ctz=Europe%2FVienna>

IBM. (2026, April 07). *What is an API?* Retrieved from What is an API?: <https://www.ibm.com/think/topics/api>

IBM. (2026, March 20). *What is role-based access control (RBAC)?* Retrieved from What is role-based access control (RBAC)?: <https://www.ibm.com/think/topics/rbac>

IBM. (2026, March 20). *What is user experience (UX)?* Retrieved from What is user experience (UX)?: <https://www.ibm.com/think/topics/user-experience>

Jalli, A. (2026, March 20). *What Is C#?* Retrieved from What Is C#?: <https://builtin.com/software-engineering-perspectives/c-sharp>

LinkedIn. (2026, March). *The Controller-Service-Repository Pattern: A Comprehensive Guide*. Retrieved from <https://www.linkedin.com/pulse/controller-service-repository-pattern-comprehensive-guide-shikder-azicc>

Medlock, J. (2026, March 20). *An Introduction to Environment Variables and How to Use Them*. Retrieved from An Introduction to Environment Variables and How to Use Them: <https://medium.com/chingu/an-introduction-to-environment-variables-and-how-to-use-them-f602f66d15fa>

Microsoft. (2023, 02 21). *Microsoft Learn*. Retrieved 03 27, 2026, from Microsoft Learn: <https://learn.microsoft.com/en->

us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design

Microsoft. (2025, 03 03). *Microsoft Learn*. Retrieved from Microsoft Learn: <https://learn.microsoft.com/en-us/nuget/what-is-nuget>

Microsoft. (2026, March). *ASP.NET Core Middleware*. Retrieved from <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0>

Microsoft. (2026, March). *Entity Framework Core*. Retrieved from <https://learn.microsoft.com/en-us/ef/core/>

Microsoft. (2026, March 21). *Introduction to Identity on ASP.NET Core*. Retrieved from Introduction to Identity on ASP.NET Core: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-10.0&tabs=visual-studio>

Microsoft. (2026, March). *What is ASP.NET Core?* Retrieved from <https://dotnet.microsoft.com/en-us/learn/aspnet/what-is-aspnet-core>

Microsoft. (2026, March 20). *What is SQL Server?* Retrieved from What is SQL Server?: <https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server?view=sql-server-ver17>

Mozilla. (2026, March 20). *SPA (Single-page application)*. Retrieved from SPA (Single-page application): <https://developer.mozilla.org/en-US/docs/Glossary/SPA>

Oracle. (2026, March 20). *What Is JSON?* Retrieved from What Is JSON?: <https://www.oracle.com/europe/database/what-is-json/>

Peiretti, G. (2026, March). *Facade Design Pattern*. Retrieved from <https://gustavopeiretti.com/facade-design-pattern/>

RFC. (2026, March). *JSON Web Token (JWT)*. Retrieved from <https://www.rfc-editor.org/rfc/rfc7519.html>

Sanjay. (2026, March 19). *Pro Code Guide*. Retrieved from Dependency Injection in ASP.NET Core 3.1 – Beginner’s Guide: <https://procodeguide.com/programming/dependency-injection-in-aspnet-core/>

sumo logic. (2026, March 20). *Create, read, update and delete*. Retrieved from <https://www.sumologic.com/glossary/crud>

Suthar, K. (2025, 04 22). *Medium*. Retrieved from Medium : <https://medium.com/@kirteshsuthar2605/serilog-the-powerful-and-game-changer-library-for-structured-and-scalable-logging-dd20e65f9994>

Swagger. (2026, March 20). *Bearer Authentication*. Retrieved from Bearer Authentication: https://swagger.io/docs/specification/v3_0/authentication/bearer-authentication/

The Blue Book. (2026, March). *Repository Pattern*. Retrieved from https://lyz-code.github.io/blue-book/architecture/repository_pattern/

Wikipedia. (2026, March). *ASP.NET*. Retrieved from <https://en.wikipedia.org/wiki/ASP.NET>

Wikipedia. (2026, March). *REST*. Retrieved from <https://en.wikipedia.org/wiki/REST>

Williams, P. (2026, March 20). *WHAT IS TYPESCRIPT?* Retrieved from WHAT IS TYPESCRIPT?: <https://medium.com/@pryncwill819/what-is-typescript-35f4e6f37bd5>

xUnit. (2026, March 20). *About xUnit.net*. Retrieved from About xUnit.net: <https://xunit.net/?tabs=cs>